Cloud Computing

CAT7002 Course Notes

Servers, virtualization, data centres, cloud fundamentals, AWS storage and delivery, and Microsoft Azure.

by Bhuvnesh Verma

Contents

Unit 1: Virtualization Concepts and Hypervisors 8

1.1 Introduction to Servers 8
1.1.1 Types of servers . . . 8

1.2 The Traditional Server Model 8
1.2.1 Advantages . . . 9
1.2.2 Disadvantages . . . 9

1.3 Introduction to Virtualization 10
1.3.1 Architecture diagram . . . 10
1.3.2 Reasons for using virtualization . . . 11

1.4 Hypervisor 12
1.4.1 Architecture diagram . . . 12
1.4.2 Type 1 hypervisor . . . 12
1.4.3 Type 2 hypervisor . . . 12
1.4.4 Type 1 versus Type 2 . . . 13

1.5 Full Virtualization 14
1.5.1 Architecture diagram . . . 14

1.6 Paravirtualization 14
1.6.1 Architecture diagram . . . 14

1.7 Hardware Based Virtualization 15
1.7.1 Architecture diagram . . . 15

1.8 Types of Virtualization 16
1.8.1 Overview . . . 16
1.8.2 Desktop virtualization . . . 16
1.8.3 Network virtualization . . . 16
1.8.4 Server and machine virtualization . . . 17
1.8.5 Storage virtualization . . . 17
1.8.6 Operating system level virtualization . . . 17
1.8.7 Application virtualization . . . 17

1.9 Levels of Virtualization 18
1.9.1 Before and after virtualization . . . 18
1.9.2 Implementation levels . . . 18
1.9.3 Instruction set architecture level . . . 19
1.9.4 Hardware abstraction level . . . 19
1.9.5 Operating system level . . . 19
1.9.6 Library support level . . . 19
1.9.7 User-application level . . . 19

1.10 Containers 20
1.10.1 Containers versus virtual machines . . . 20
1.10.2 Comparison . . . 21

1.11 Container Orchestration **21**
1.11.1 Why it is needed . . . 21

Unit 2: Virtualization Technologies and Data Center Management **22**

2.1 Virtualization Technologies **22**

2.2 Software Virtualization **23**
2.2.1 Components . . . 23
2.2.2 Types of software virtualization . . . 23

2.3 Operating System Virtualization **24**
2.3.1 Architecture diagram . . . 24
2.3.2 Types of virtual disk . . . 24
2.3.3 Advantages . . . 25
2.3.4 Disadvantages . . . 25

2.4 Application Virtualization **26**
2.4.1 Architecture diagram . . . 26
2.4.2 Benefits . . . 26

2.5 Service Virtualization **27**
2.5.1 Architecture diagram . . . 27

2.6 Hardware Virtualization **28**
2.6.1 Benefits . . . 28

2.7 Storage Virtualization **29**
2.7.1 Types of storage virtualization . . . 29
2.7.2 Benefits . . . 29

2.8 The Data Center **30**
2.8.1 Need for a data centre . . . 30
2.8.2 Objectives of a data centre . . . 30
2.8.3 Components . . . 30

2.9 High Availability of a Data Center **31**
2.9.1 Architecture diagram . . . 31
2.9.2 Need for high availability . . . 31
2.9.3 Causes of data centre downtime . . . 31

2.10 Data Center Maintenance **32**
2.10.1 Why it matters . . . 32

2.11 Planned Maintenance **33**
2.11.1 Objectives . . . 33
2.11.2 Characteristics . . . 33
2.11.3 Activities . . . 34
2.11.4 Advantages . . . 34
2.11.5 Disadvantages . . . 34

2.12 Unplanned Maintenance **35**
2.12.1 Causes . . . 35
2.12.2 Common activities . . . 35
2.12.3 Steps followed . . . 35
2.12.4 Challenges . . . 36
2.12.5 Best practices to reduce it . . . 36
2.12.6 Planned versus unplanned maintenance . . . 36

2.13 Server Migration **37**
2.13.1 Benefits . . . 37
2.13.2 Types of server migration . . . 38
2.13.3 Migration considerations . . . 39
2.13.3.1 Business considerations . . . 39
2.13.3.2 Hardware considerations . . . 39
2.13.3.3 Software compatibility . . . 39
2.13.3.4 Data integrity . . . 39

2.14 Planning a Server Migration **40**
2.14.1 Phase 1: Discovery . . . 40
2.14.2 Phase 2: Assessment . . . 40
2.14.2.1 Migration strategies . . . 41
2.14.3 Phase 3: Migration . . . 42
2.14.4 Best practices . . . 42
2.14.5 Advantages of proper planning . . . 42

2.15 Networking Concepts: Masks and CIDR **43**

2.16 Subnetting **44**
2.16.1 Levels of hierarchy . . . 44
2.16.2 Finding the subnetwork address . . . 44

2.17 Supernetting **46**

2.18 Classless Addressing **47**
2.18.1 Rules for allocating a block . . . 47
2.18.2 Slash notation . . . 47
2.18.3 Extracting information from an address . . . 47
2.18.4 Address mask . . . 47

2.19 Practice Problems **50**

Unit 3: Cloud Computing Fundamentals **80**

3.1 Importance of Cloud Computing **80**

3.2 Cloud Service Models **80**
3.2.1 Overview . . . 80

3.3 Infrastructure as a Service (IaaS) **80**
3.3.1 Architecture diagram . . . 81
3.3.2 Benefits . . . 81
3.3.3 Issues . . . 81
3.3.4 Characteristics . . . 81

3.4 Platform as a Service (PaaS) 82
3.4.1 Architecture diagram . . . 82
3.4.2 Benefits . . . 82
3.4.3 Issues . . . 82
3.4.4 Characteristics . . . 82

3.5 Software as a Service (SaaS) 83
3.5.1 Characteristics . . . 83
3.5.2 Benefits . . . 83
3.5.3 Issues . . . 84

3.6 Cloud Deployment Models 84
3.6.1 Overview diagram . . . 84
3.6.2 Public Cloud . . . 84
3.6.3 Private Cloud . . . 85
3.6.4 Hybrid Cloud . . . 85
3.6.5 Community Cloud . . . 86

3.7 Regions, Availability Zones and Edge Locations 86
3.7.1 Region . . . 86
3.7.2 Availability Zone . . . 87
3.7.3 Edge Location . . . 87
3.7.4 Relationship between Region, AZ and Edge Location . . . 88
3.7.5 Difference between Region, AZ and Edge Location . . . 89

3.8 Cloud Computing: Benefits and Challenges 90
3.8.1 Benefits, in detail . . . 90
3.8.2 The Challenges . . . 91

3.9 Fundamentals of Pricing: Total Cost of Ownership 92
3.9.1 Overview diagram . . . 93
3.9.2 Need for TCO . . . 93

3.10 On-Premises TCO 94
3.10.1 CAPEX and OPEX . . . 94

3.11 Cloud TCO 94
3.11.1 Components of cloud TCO . . . 94

3.12 Basic TCO Formulas 95

3.13 On-Premises versus Cloud TCO 95

3.14 TCO Numericals 95

Unit 4: Amazon Web Services (AWS) 101

4.1 The AWS Storage Portfolio 101

4.2 Amazon S3: Simple Storage Service 101
4.2.1 Overview . . . 101
4.2.2 Access methods . . . 102
4.2.3 Fundamental constructs . . . 102

4.2.3.1 Buckets . . . 102
4.2.3.2 Objects . . . 103
4.2.3.3 Keys . . . 103
4.2.4 Storage classes . . . 103
4.2.4.1 S3 Standard . . . 103
4.2.4.2 S3 Standard-IA (Infrequent Access) . . . 103
4.2.4.3 S3 RRS (Reduced Redundancy Storage) . . . 103
4.2.4.4 Amazon Glacier . . . 104
4.2.4.5 Comparison . . . 104
4.2.5 Lifecycle policies . . . 104
4.2.6 Data ingress and egress . . . 105
4.2.7 Security . . . 106

4.3 Amazon EBS: Elastic Block Store 106
4.3.1 Introduction . . . 106
4.3.2 EBS lifecycle . . . 107
4.3.3 Volume types . . . 107
4.3.4 EBS snapshots . . . 108
4.3.5 Features and benefits . . . 108

4.4 Amazon EFS: Elastic File System 108
4.4.1 Simple, elastic, scalable . . . 109
4.4.2 File systems and mounting . . . 109
4.4.3 How it fits together . . . 110
4.4.4 Securing a file system . . . 110
4.4.5 Availability and consistency . . . 111

4.5 Content Delivery Networks (CDN) 111
4.5.1 Why a CDN is needed . . . 111
4.5.2 Passive and active CDNs . . . 112
4.5.3 CDN challenges . . . 112
4.5.4 Directing clients to a replica . . . 112

4.6 Amazon CloudFront 113
4.6.1 How CloudFront gives higher performance . . . 113
4.6.2 Pricing . . . 114
4.6.3 Edge locations . . . 114
4.6.4 Speeding up an entire website . . . 115
4.6.5 How long files stay at the edge . . . 115
4.6.6 Distributions . . . 115

4.7 AWS Snowball and Snowmobile 116
4.7.1 The problem: moving large data sets . . . 116
4.7.2 AWS Snowball and Snowball Edge . . . 116
4.7.3 How Snowball works . . . 117
4.7.4 Types of Snowball jobs . . . 117
4.7.5 Features, advantages and limitations . . . 118
4.7.6 AWS Snowmobile . . . 118

4.8 Amazon Route 53: DNS Service 120
4.8.1 DNS background . . . 120
4.8.2 How a name is resolved . . . 121
4.8.3 Routing policies . . . 121
4.8.3.1 Simple and weighted . . . 122
4.8.3.2 Failover . . . 122
4.8.3.3 Latency and geolocation . . . 123

Unit 5: Introduction to Microsoft Azure 124

5.1 Introduction to Microsoft Azure 124
5.1.1 Why we need Azure . . . 124

5.2 Cloud Service Models in Azure 124

5.3 Major Azure Services 125

5.4 Azure Subscription 125
5.4.1 Key functions . . . 125

5.5 Azure Resource Group 125
5.5.1 Key functions . . . 125
5.5.2 Example: college management system . . . 126
5.5.3 Important points . . . 126

5.6 Azure Resource Deployment Models 126
5.6.1 Azure Resource Manager (ARM) . . . 126
5.6.2 Classic deployment model . . . 127
5.6.3 ARM versus Classic . . . 127
5.6.4 Cloud deployment models with Azure examples . . . 128

5.7 Methods of Creating Resources in Azure 128
5.7.1 Method 1: Azure Portal . . . 128
5.7.2 Method 2: Azure Cloud Shell . . . 129
5.7.3 Method 3: ARM templates . . . 129

5.8 Virtual Networking in the Cloud 129
5.8.1 Main components . . . 130
5.8.2 Architecture diagram . . . 130

5.9 Compute Services in AWS and Azure 130
5.9.1 Amazon EC2 (Elastic Compute Cloud) . . . 131
5.9.2 Creating a Linux VM in Azure . . . 131
5.9.3 AWS EC2 versus Azure Virtual Machines . . . 132

5.10 Remote Desktop Protocol (RDP) 132
5.10.1 How RDP works . . . 132

5.11 Secure Shell (SSH) 134
5.11.1 How SSH works . . . 134
5.11.2 Important features . . . 135
5.11.3 RDP versus SSH . . . 135

5.12 Load Balancer **135**
5.12.1 How a request is handled . . . 136
5.12.2 Objectives . . . 136
5.12.3 Types of load balancers . . . 136
5.12.4 Load-balancing algorithms . . . 137
5.12.5 Health checks . . . 138

5.13 Auto Scaling **138**
5.13.1 Why we need it . . . 139
5.13.2 Types of scaling . . . 139
5.13.3 Load balancing and Auto Scaling together . . . 140

5.14 Load Balancing and Auto Scaling Numericals **140**

Unit 1: Virtualization Concepts and Hypervisors

1.1. Introduction to Servers

Short description: A server is a computer or software system that provides services, resources, or data to other computers, called clients, over a network.

Everyday examples:

- Opening a website: the browser sends a request to a **web server**.
- Checking Gmail: a **mail server** delivers the messages.
- Using online banking: a **database server** stores and retrieves the account.

What a server does: Stores files, hosts websites, manages databases, runs applications, sends and receives email, shares printers and other resources, and manages users and security.

1.1.1 Types of servers

Server type	Purpose
Web server	Hosts websites
File server	Stores and shares files
Database server	Stores databases
Mail server	Sends and receives emails
Application server	Runs business applications
Print server	Manages printers
DNS server	Converts domain names into IP addresses

1.2. The Traditional Server Model

Short description: Before virtualization and cloud computing, organizations typically deployed **one physical server for one application**. Each application had its own dedicated hardware.

Application	Physical server
Website	Server 1
Database	Server 2
Email	Server 3
ERP	Server 4

1.2.1 Advantages

- **High performance.** Applications have direct access to hardware resources.
- **Better security.** Each application runs on its own physical machine.
- **Reliability.** No interference from other applications sharing the hardware.
- **Simple architecture.** Easy to install, configure and troubleshoot.
- **Full hardware control.** Administrators control the server hardware completely.

1.2.2 Disadvantages

- **Low resource utilization.** Most servers use only 10–20% of CPU and memory, leaving the rest idle.
- **High cost.** Each application requires separate hardware.
- **High power consumption.** More servers draw more electricity and need more cooling.
- **More physical space.** Multiple servers occupy valuable rack space.
- **Difficult scalability.** A new application often means buying new hardware.
- **Higher maintenance.** Every server needs updates, monitoring, backups and repairs.
- **Single point of failure.** If a physical server fails, its application becomes unavailable.

Why a new technology was needed

As organizations expanded they faced too many physical servers, rising hardware costs, increased electricity and cooling expenses, underutilized resources, complex management, and slow deployment of new services. These limitations led to **virtualization**, which allows multiple virtual servers to run on a single physical machine, and virtualization in turn became the foundation for **cloud computing**.

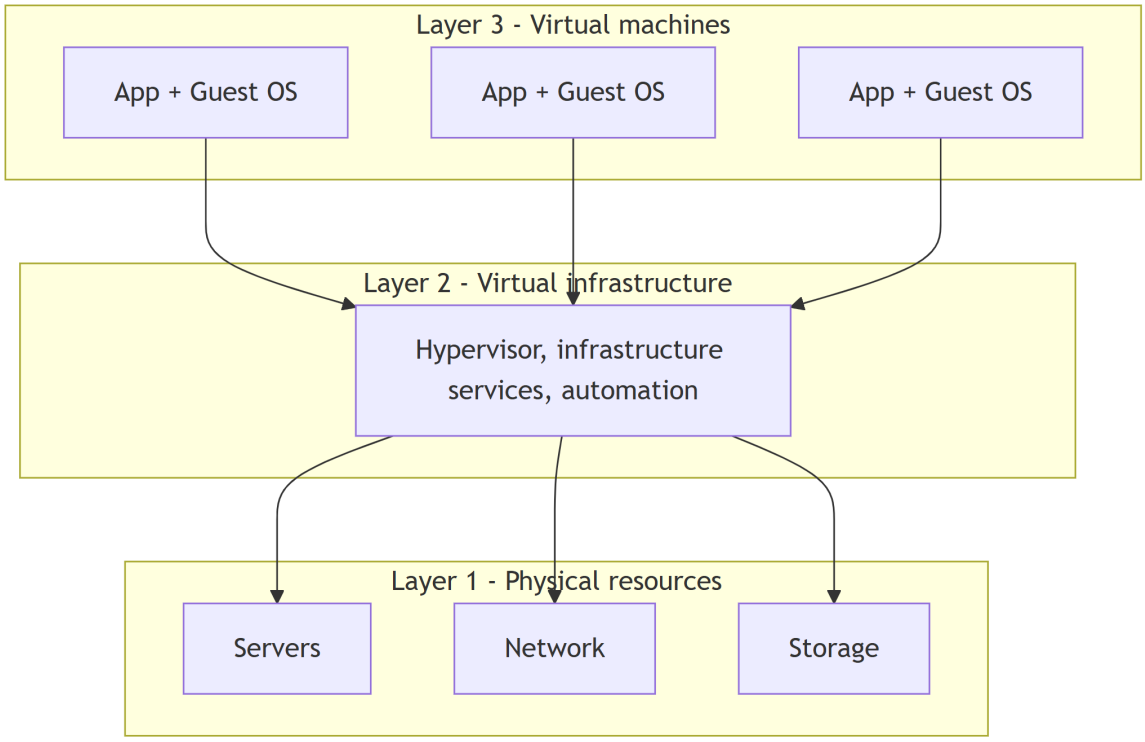
1.3. Introduction to Virtualization

Short description: Virtualization is a methodology for dividing computer resources into more than one execution environment, applying concepts such as partitioning, time-sharing, machine simulation and emulation.

Theory: Virtualization is a method in which multiple independent operating systems run on one physical computer. It maximizes the usage of available physical resources, so one can achieve high server usage: it increases total computing power while decreasing overhead. Increasing the number of logical operating systems on a single host reduces hardware acquisition and maintenance cost for an organization.

How it works: The architecture has three layers. Layer 1 is the physical infrastructure of servers, networks and storage. Layer 2 is the virtual infrastructure, consisting of hypervisors, virtual infrastructure services and automated solutions that optimize the IT process. Layer 3 holds the virtual machines, where different operating systems and applications are deployed. A single virtual infrastructure supports more than one virtual machine, so the physical resources of the whole infrastructure are shared across many virtual machines for maximum efficiency.

1.3.1 Architecture diagram



Notes

- Virtualization is a trend in IT that includes autonomic computing and utility computing: the environment manages itself, and every resource is seen as a utility that a client pays per use.
- It reduces the burden of workloads on users by centralizing administrative tasks and improving scalability.

1.3.2 Reasons for using virtualization

- Virtual machines consolidate the workloads of under-utilized servers, saving on hardware, environmental costs and management.
- Legacy applications can keep running inside a VM.
- A VM provides a secure sandbox for running an untrusted application.
- A VM helps in building a secured computing platform.
- A VM provides an illusion of hardware.
- A VM simulates networks of independent computers.
- A VM supports running distinct operating systems with different versions.

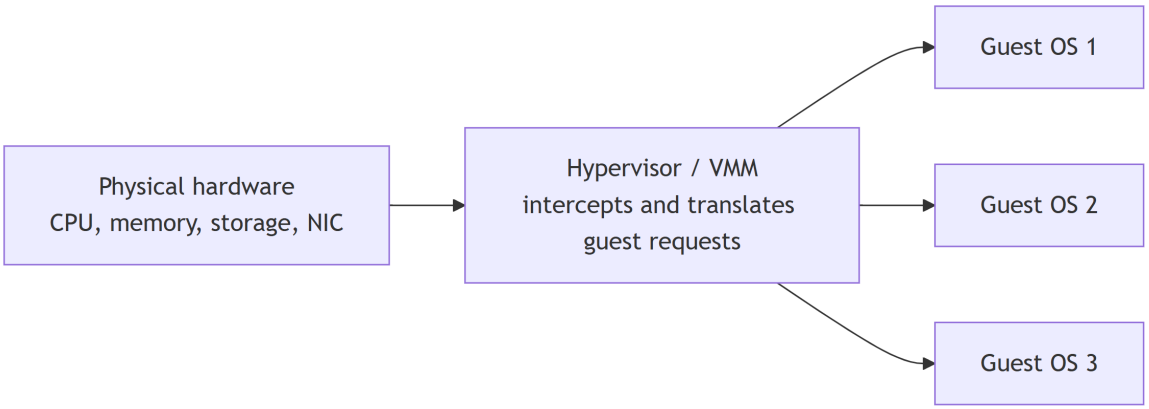
1.4. Hypervisor

Short description: A hypervisor, or Virtual Machine Monitor (VMM), is software that lets multiple operating systems run on a single physical machine.

Theory: It manages hardware resources such as CPU, memory and storage and allocates them to virtual machines without interference. This improves hardware utilization, reduces costs, and provides flexibility in cloud and server environments.

How it works: A hypervisor runs on hardware or on a host OS to create and manage virtual machines, each with its own virtual CPU, memory, storage and network. It intercepts guest OS requests and translates them to physical hardware, ensuring isolation, security and stability.

1.4.1 Architecture diagram



1.4.2 Type 1 hypervisor

A Type 1 hypervisor runs directly on the host's hardware and does not rely on a host operating system. This architecture offers better performance and security because there is no intermediary OS. It is the standard for enterprise-level data centres and for cloud providers such as Amazon Web Services and Microsoft Azure.

Examples: VMware ESXi, Microsoft Hyper-V, KVM (Kernel-based Virtual Machine), Xen.

- **Pros:** high performance through direct hardware access; strong security with no intermediate OS layer; suitable for mission-critical workloads.
- **Cons:** requires dedicated hardware; setup and management are complex compared to Type 2.

1.4.3 Type 2 hypervisor

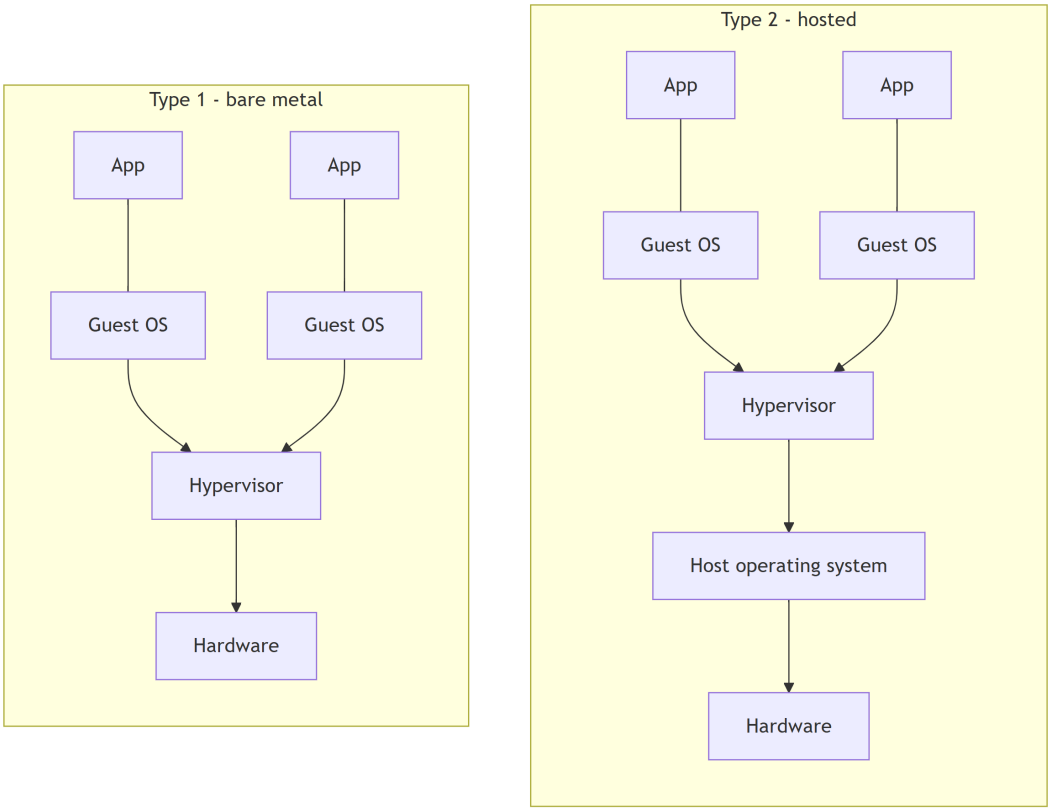
A Type 2 hypervisor runs on top of a conventional operating system such as Windows, macOS or Linux. It is essentially an application within the host OS. This type is generally used for desktop virtualization, development and testing, where a user needs to run multiple operating systems on a personal computer. Performance is slightly lower than Type 1 because of the overhead of the host OS.

Examples: Oracle VM VirtualBox, VMware Workstation, Parallels Desktop.

- **Pros:** easy to install and use; useful for development, testing and malware analysis; good host–guest integration features.

- **Cons:** slower performance with no direct hardware access; security depends on the host OS, so a compromise of the host may affect the guests.

1.4.4 Type 1 versus Type 2



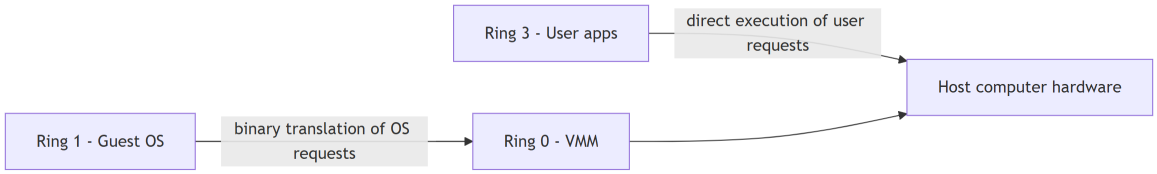
1.5. Full Virtualization

Short description: Full virtualization was introduced by IBM in 1966. It was the first software solution for server virtualization and uses binary translation together with direct execution.

How it works: The virtual machine completely isolates the guest OS from the virtualization layer and the hardware. Kernel code is translated to replace non-virtualizable instructions with new sequences of instructions that have the intended effect on the virtual hardware, while user level code is executed directly on the processor for high performance.

Examples: Microsoft and Parallels systems.

1.5.1 Architecture diagram



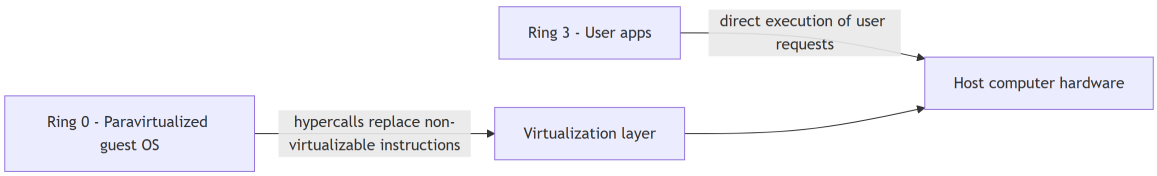
1.6. Paravirtualization

Short description: Paravirtualization is the category of CPU virtualization that uses hypercalls for operations handled at compile time.

How it works: The guest OS is not completely isolated, only partially isolated by the virtual machine from the virtualization layer and hardware. The OS kernel is modified to replace non-virtualizable instructions with hypercalls that communicate directly with the virtualization layer, which improves performance and efficiency.

Examples: VMware and Xen.

1.6.1 Architecture diagram

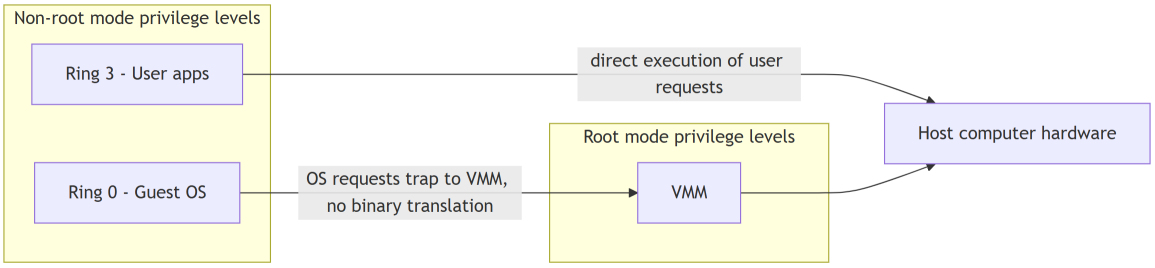


1.7. Hardware Based Virtualization

Short description: Hardware vendors added CPU features that simplify virtualization directly in silicon.

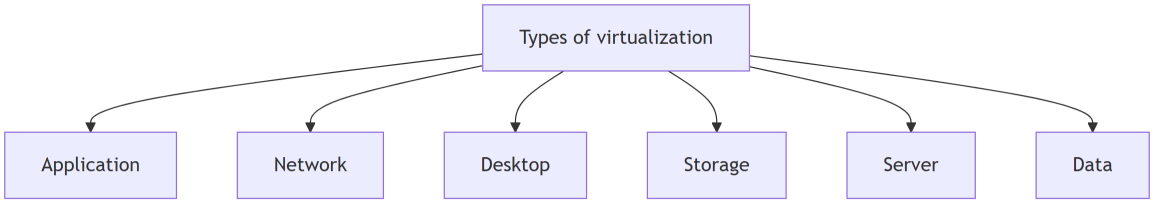
How it works: First generation enhancements include Intel Virtualization Technology (VT-x) and AMD's AMD-V. Both target privileged instructions with a new CPU execution mode that allows the VMM to run in a root mode below ring 0. Privileged and sensitive calls are set to trap automatically to the hypervisor, removing the need for either binary translation or paravirtualization. The guest state is stored in Virtual Machine Control Structures (VT-x) or Virtual Machine Control Blocks (AMD-V).

1.7.1 Architecture diagram



1.8. Types of Virtualization

1.8.1 Overview

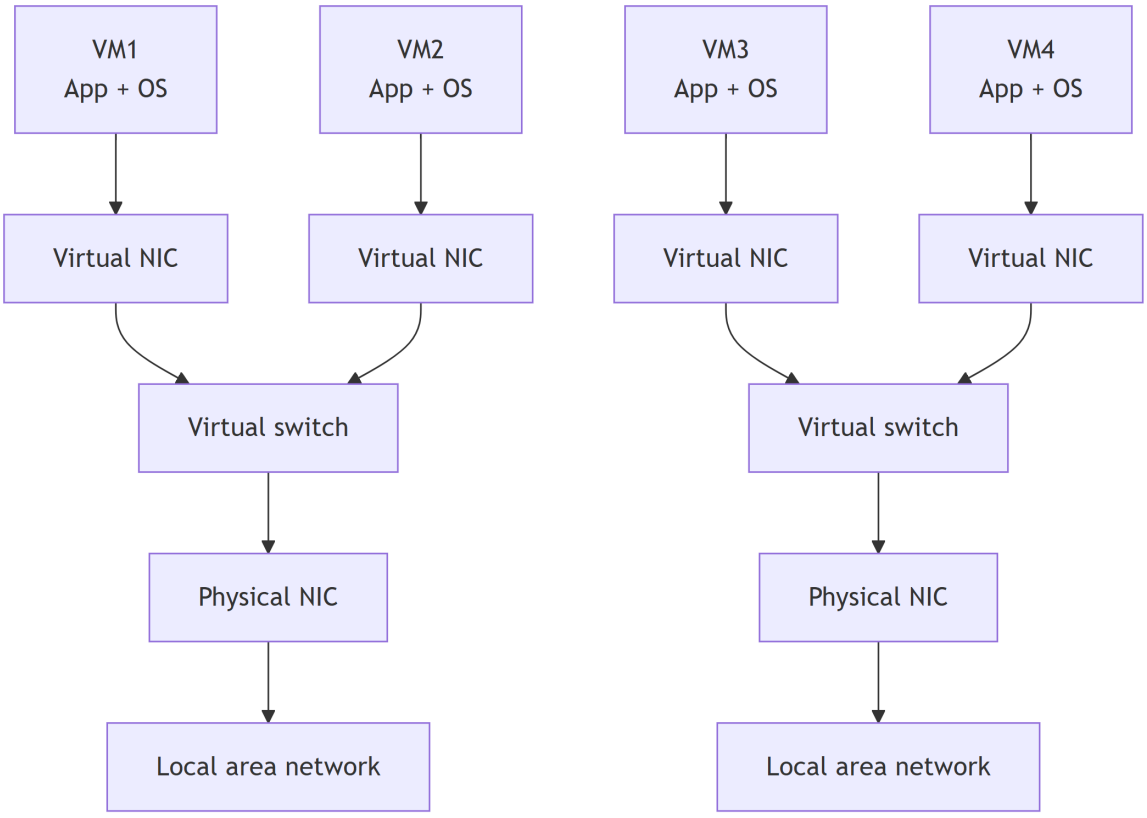


1.8.2 Desktop virtualization

Also called client virtualization. The virtualization happens on the user's site: desktops are replaced with thin clients that use datacentre resources.

1.8.3 Network virtualization

Part of the virtualization infrastructure, used especially when virtualizing servers. It allows multiple switches, VLANs and NAT-ing to be created in software.

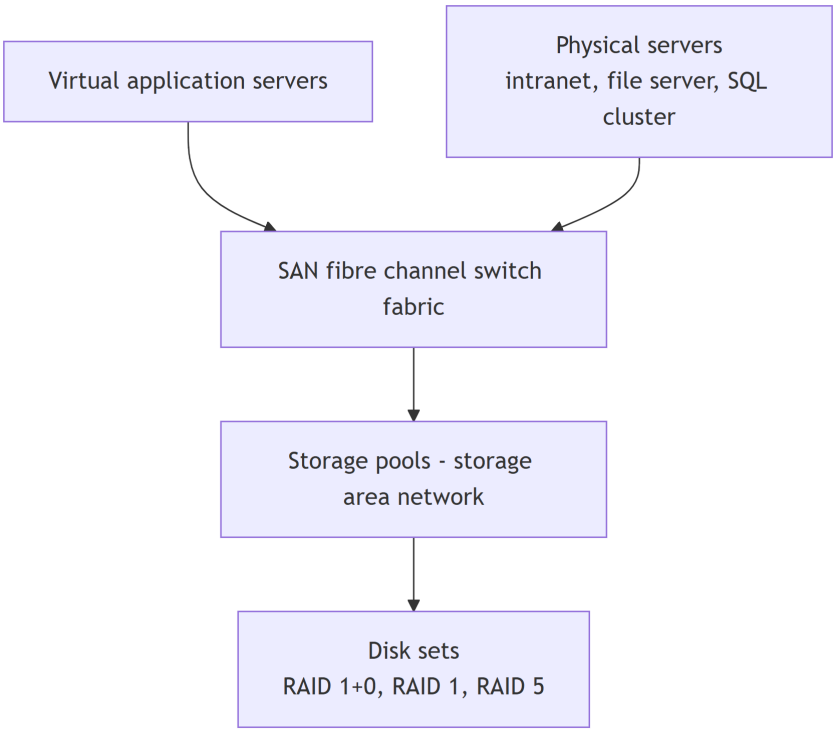


1.8.4 Server and machine virtualization

Virtualizing the server infrastructure, so no further physical servers are needed for different purposes.

1.8.5 Storage virtualization

Widely used in datacentres with large storage. It helps create, delete and allocate storage to different hardware, over a network connection. The leading technology here is the SAN.



1.8.6 Operating system level virtualization

An abstraction layer between the traditional OS and user applications. OS-level virtualization creates isolated **containers** on a single physical server, using one OS instance to utilize the hardware and software in data centres. The containers behave like real servers. It is commonly used to create virtual hosting environments that allocate hardware resources among a large number of mutually distrusting users, and to a lesser extent to consolidate server hardware by moving services on separate hosts into containers or VMs on one server.

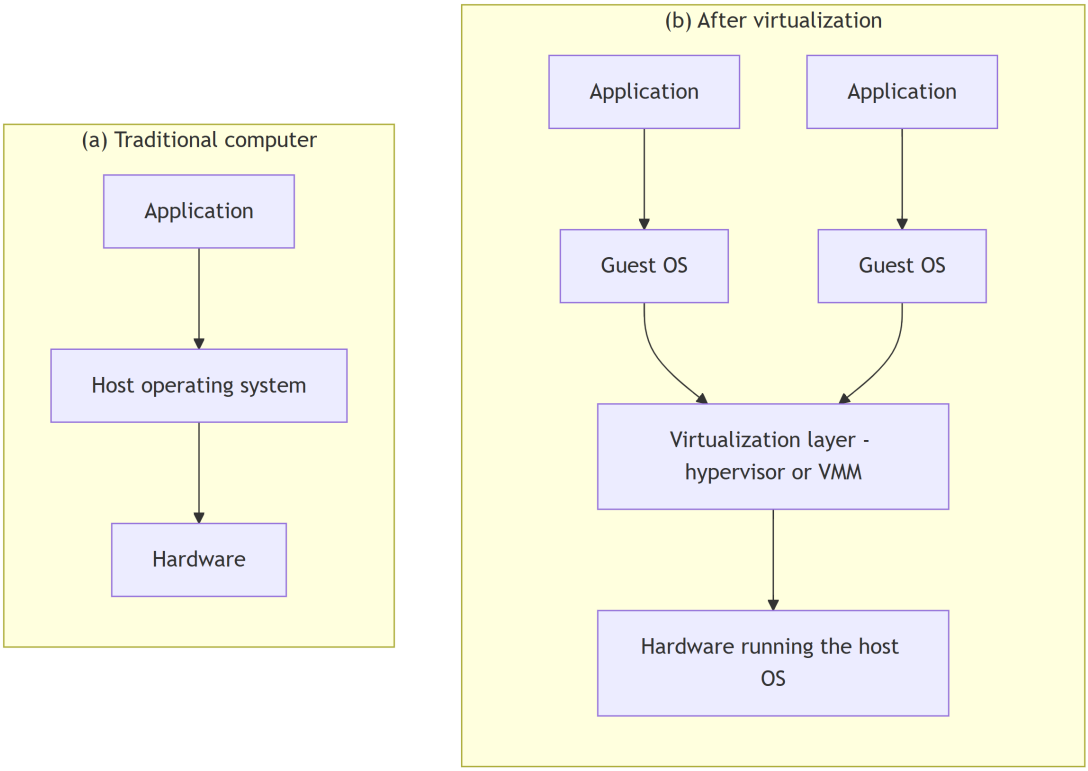
1.8.7 Application virtualization

The technology isolates applications from the underlying operating system and from other applications, to increase compatibility and manageability. Docker can be used for this purpose.

1.9. Levels of Virtualization

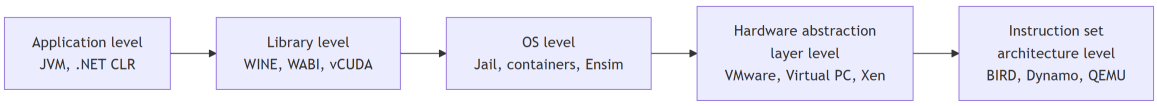
Short description: A traditional computer runs a host operating system tailored to its hardware architecture. After virtualization, different user applications managed by their own guest operating systems can run on the same hardware, independent of the host OS. This is done by adding a software layer called the virtualization layer, known as the hypervisor or virtual machine monitor.

1.9.1 Before and after virtualization



1.9.2 Implementation levels

Virtualization can be implemented at different levels within a computing system, offering varying degrees of isolation and flexibility.



1.9.3 Instruction set architecture level

- Virtualization is performed by emulating a given ISA with the ISA of the host machine.
- For example, MIPS binary code can run on an x86-based host machine through ISA emulation.
- It becomes possible to run a large amount of legacy binary code written for various processors on any given new hardware host.

1.9.4 Hardware abstraction level

- Hardware-level virtualization is performed right on top of the bare hardware. It generates a virtual hardware environment for a VM, and the process manages the underlying hardware through virtualization.
- The idea is to virtualize a computer's resources, such as its processors, memory and I/O devices, to upgrade the hardware utilization rate by multiple concurrent users.
- It was implemented in the IBM VM/370 in the 1960s. More recently the Xen hypervisor has been applied to virtualize x86-based machines running Linux or other guest OS applications.

1.9.5 Operating system level

- An abstraction layer between the traditional OS and user applications.
- It creates isolated containers on a single physical server, with the OS instances utilizing the hardware and software in data centres.
- The containers behave like real servers, and are commonly used for virtual hosting environments allocating hardware among mutually distrusting users.

1.9.6 Library support level

- Most applications use APIs exported by user-level libraries rather than lengthy system calls, so a well-documented API becomes another candidate for virtualization.
- Virtualization with library interfaces controls the communication link between applications and the rest of the system through API hooks.
- WINE implements this approach to support Windows applications on top of UNIX hosts.
- vCUDA allows applications executing within VMs to leverage GPU hardware acceleration.

1.9.7 User-application level

- Virtualization at the application level virtualizes an application as a VM. On a traditional OS an application runs as a process, so this is also known as process-level virtualization.
- The most popular approach is to deploy high level language (HLL) VMs. The virtualization layer sits as an application program on top of the operating system and exports the abstraction of a VM that runs programs compiled to a particular abstract machine definition.
- The Microsoft .NET CLR and the Java Virtual Machine are two good examples.

1.10. Containers

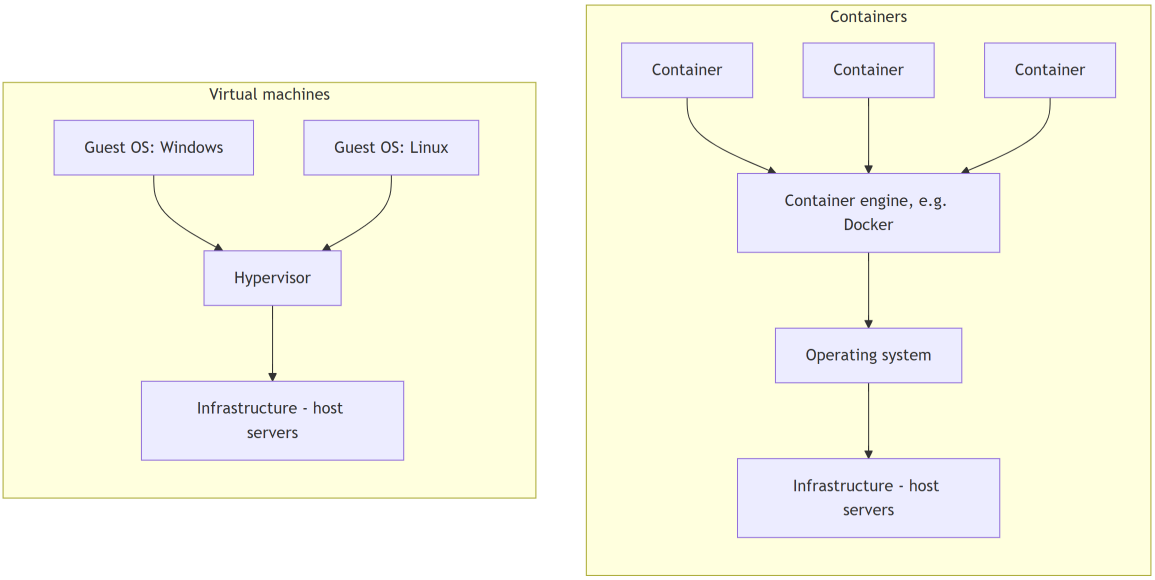
Short description: A container is an isolated, stand-alone unit that encapsulates an application and all its dependencies. It runs consistently in any environment, independent of the host system.

Theory: It is a light, executable software package that wraps everything an application needs: libraries, configuration files and binaries. The application behaves the same way on a developer's laptop, in a testing environment, or on a production server. Unlike traditional virtual machines that carry a full OS, containers pack only what is required, which makes them faster and more efficient.

How it works: Containerization is the process of packing an application together with all its dependencies into a container so it runs consistently from one computing environment to another. In simple terms it uses the host OS kernel to run many isolated instances of applications on the same machine, which makes it very lightweight and efficient.

1.10.1 Containers versus virtual machines

Both containers and VMs run applications in isolated environments, but they differ in architecture, performance, resource usage and startup time.



1.10.2 Comparison

	Containers	**Virtual machines**
Architecture	All containers share the host OS kernel; the running user spaces are isolated, which makes them lightweight.	A hypervisor running on the host OS includes a full guest OS with virtualized hardware.
Boot time	Much less, typically seconds, as no full OS has to boot.	Longer, because the full OS in the VM must be initialized.
Isolation	At the process level, less strong than a VM, though for many use cases this does not matter.	Very good, because each VM is a system on its own with its own OS.
Resource usage	Fewer resources: only the necessary binaries and libraries, not an entire OS.	Very high, as the full OS overhead is incurred for each instance.

1.11. Container Orchestration

Short description: Container orchestration is the process of automating the deployment, management, scaling and networking of containers throughout their lifecycle.

Theory: Orchestration platforms such as Kubernetes create a smooth, self-managing operation and coordinate microservices held in separate containers. Kubernetes eliminates many of the manual processes involved in deploying and scaling containerized applications, and Red Hat creates essential tools for securing, simplifying and automatically updating container infrastructure.

1.11.1 Why it is needed

When many containers run across different machines, managing them manually becomes difficult. An orchestration platform automates:

- **Deployment:** starts and distributes containers across servers.
- **Scaling:** increases or decreases the number of container instances based on demand.
- **Load balancing:** distributes incoming traffic among healthy containers.
- **Self-healing:** restarts failed containers or moves them to healthy nodes.
- **Service discovery:** allows containers to find and communicate with each other.
- **Rolling updates and rollbacks:** updates applications with minimal downtime and reverts changes if needed.
- **Resource management:** efficiently allocates CPU, memory and storage.

Benefits

Orchestration platforms automate the entire lifecycle of containers, transforming what would be a non-stop maintenance operation for IT teams into a smooth, self-managing one, and bring advantages in development methods, costs and security.

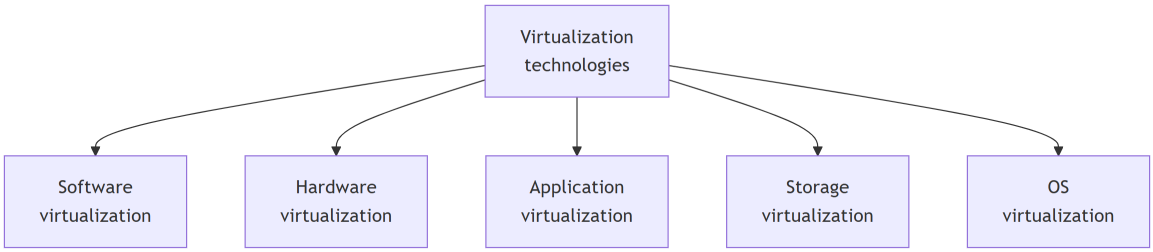
Unit 2: Virtualization Technologies and Data Center Management

2.1. Virtualization Technologies

Short description: Unit 1 covered what virtualization is and how a hypervisor works. This unit covers the technologies that implement it, the data centre those technologies run in, and the addressing scheme that ties the machines together.

Five technologies are studied in this unit.

1. Software virtualization
2. Hardware virtualization
3. Application virtualization
4. Storage virtualization
5. Operating system virtualization



2.2. Software Virtualization

Short description: Software virtualization is a technique that allows a server to run multiple operating systems and applications by creating a virtual environment.

Theory: Software virtualization develops a virtual environment in which software, applications and operating systems can be installed. A virtualized software is simply an application that has been installed inside such an environment. It lets a user run more than one operating system on the same machine. VMware and VirtualBox are common examples.

2.2.1 Components

- **Hypervisor:** a piece of software that manages and allocates hardware resources to the virtual machines.
- **Virtual machines:** independent computers, each running its own operating system on a portion of the physical machine's hardware.
- **Abstraction layer:** the layer of separation between the physical hardware and the virtual machines.

2.2.2 Types of software virtualization

The three forms studied in this unit are operating system virtualization, application virtualization and service virtualization. Each is covered in its own section below.

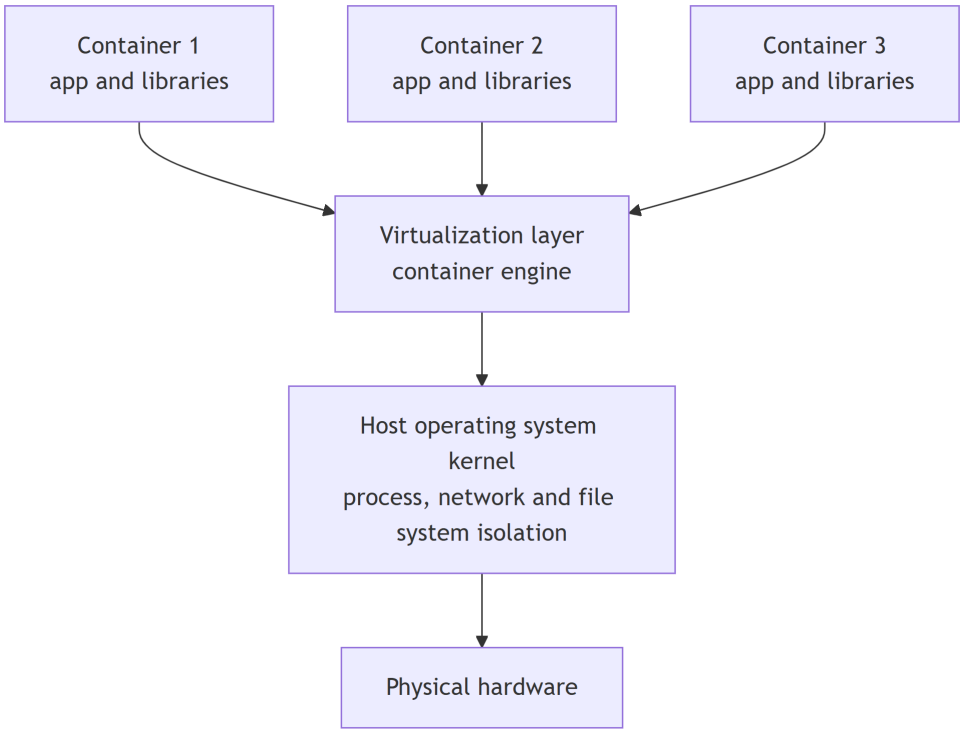
2.3. Operating System Virtualization

Short description: A virtualization technique in which the virtualization software runs on top of a host operating system and creates isolated environments called containers.

Theory: Multiple applications or services run independently on the same physical hardware while sharing the underlying OS kernel. The virtualization layer creates an abstraction between the hardware and the applications, which improves resource utilization and isolation. Unlike traditional virtualization, which creates complete virtual machines, OS-based virtualization shares the host kernel among the containers, so it is lighter and more efficient.

How it works: The host operating system kernel manages the containers through specialised features that control resource access and allocation. Each container operates as an isolated environment with its own process space, network interfaces and file systems, while sharing the underlying kernel. The kernel enforces limits on CPU time, memory usage, disk space and network bandwidth for each container, so one container cannot affect the others and no conflicts or security breaches arise between them.

2.3.1 Architecture diagram



2.3.2 Types of virtual disk

- **Private virtual disk:** dedicated to a single container. It keeps data and configuration across restarts, much like a local hard disk.
- **Shared virtual disk:** used by several containers at the same time. Changes go to a temporary cache that is cleared on restart, which gives every container the same baseline configuration.

2.3.3 Advantages

- **Resource efficiency:** containers share the host kernel, so memory and storage overhead is lower than with full virtualization.
- **Fast startup:** a container starts quickly because no full OS boot is needed.
- **High density:** more containers fit on the same hardware than virtual machines.
- **Portability:** containers move easily between host systems.

2.3.4 Disadvantages

- **OS dependency:** every container must use the same operating system as the host.
- **Security concerns:** a kernel vulnerability can affect all containers at once.
- **Limited isolation:** isolation is weaker than in hardware-based virtualization.

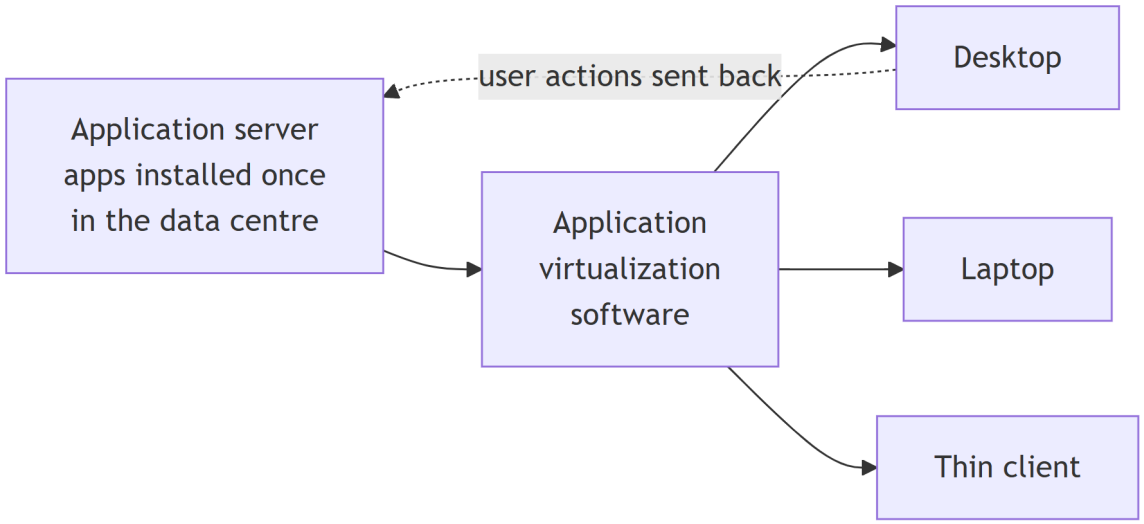
2.4. Application Virtualization

Short description: Application virtualization is the ability to run an application in an environment different from the one it was developed for.

Theory: It allows IT administrators to install applications on a server and then deliver them remotely to users' computers. The experience of using a virtualized application is the same as using an application installed on a physical machine.

How it works: The common approach is server based. An administrator installs the applications once, either in the organisation's data centre or through a hosting service, and then uses application virtualization software to deliver them to users' desktops and other connected devices. The user accesses the application as if it were installed locally, and the user's actions are conveyed back to the server to be executed. Application virtualization is an integral part of digital workspaces and desktop virtualization.

2.4.1 Architecture diagram



2.4.2 Benefits

- **Simplified management:** the applications to be delivered are installed on a single server rather than on every user device, which makes installing software, updates and patches far easier for the IT team.
- **Scalability:** administrators can deploy virtual applications to every connected device regardless of its operating system or storage capacity. The organisation then spends less on hardware, because employees only need a basic computer to reach the applications they work with.
- **Security:** access to applications is controlled centrally, so licences are easy to revoke when a user no longer needs them. The application never has to be uninstalled from a user's device, because it stays resident but reachable only through its container. This central control matters most when a device is lost or stolen, since sensitive data can no longer be reached remotely.

2.5. Service Virtualization

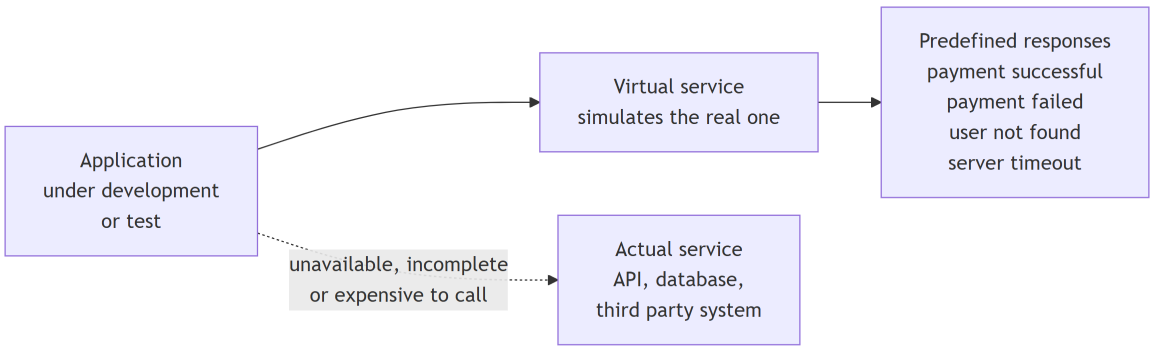
Short description: Service virtualization simulates the behaviour of software services, APIs, databases or third-party applications.

Theory: It lets developers and testers build, integrate and test an application even when the real service is unavailable, incomplete, expensive to access or difficult to configure. In simple terms, it creates a virtual version of a dependent service that behaves like the real one without requiring the actual system.

How it works: Instead of communicating with the actual service, the application talks to the virtual service, which returns predefined or dynamically generated responses.

Examples: Typical simulated responses are payment successful, payment failed, user not found, server timeout and invalid request.

2.5.1 Architecture diagram



2.6. Hardware Virtualization

Short description: Hardware virtualization allows multiple virtual machines to run on a single physical machine.

Theory: It creates a virtual version of a computer's hardware components: the CPU, memory, storage and network. The resulting virtual environment, the virtual machine, acts like a real computer and runs its own operating system and applications independently of the host system.

How it works: A software layer called the hypervisor creates and manages the virtual machines on a single physical server. The hypervisor allocates the server's resources, such as CPU, memory and storage, to each VM, so several operating systems and applications run independently on the same hardware.

2.6.1 Benefits

- **Improved resource utilization:** consolidating several workloads onto one physical server makes better use of the hardware, so fewer physical servers are needed and energy consumption and floor space both drop.
- **Cost savings:** fewer machines means savings on both capital expenditure, which is buying the servers, and operational expenditure, which is maintaining and powering them.
- **Flexibility and scalability:** virtual machines are easy to create, modify and move, which matters for a business that has to adapt quickly to changing workloads.
- **Isolation and security:** each VM operates in isolation, so malware or a system failure is contained within a single VM.
- **Simplified management:** centralised tools let administrators manage many virtual environments from one interface, which is especially useful when the infrastructure is spread across several locations.

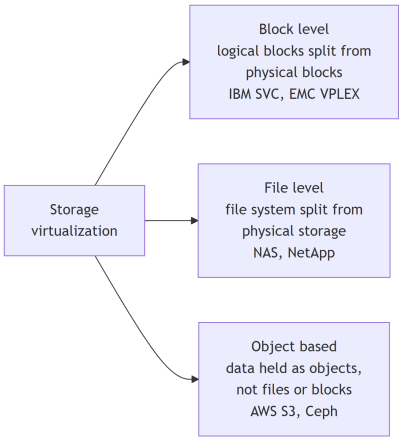
2.7. Storage Virtualization

Short description: Storage virtualization brings several storage devices together into a single entity, for better efficiency, security and management.

Theory: Storage virtualization is a newer concept under virtualization: storage systems use it to gain functionality and features. A storage system is also called a storage array, a disk array or a filer. It uses special hardware and software to provide fast and reliable storage for computing and processing data, and it is designed to accommodate large capacity together with data protection.

How it works: Multiple physical storage devices are combined into a single logical storage pool by a virtualization layer. That layer manages allocation, maps logical volumes to physical devices and handles every read and write transparently. Users and applications see a single logical storage system, while the software manages the physical storage underneath and improves utilization, scalability, performance and ease of management.

2.7.1 Types of storage virtualization



- **Block level virtualization:** separates logical storage from physical storage at the block level, as in IBM SVC and EMC VPLEX.
- **File level virtualization:** abstracts the file system from the physical storage, as in NAS solutions such as NetApp.
- **Object based storage:** holds data as objects rather than as files or blocks, as in AWS S3 and Ceph.

2.7.2 Benefits

- **Enhanced storage utilization:** aggregates different storage devices efficiently.
- **Scalability:** storage scales easily as requirements grow.
- **Savings:** hardware cost falls because existing resources are reused.
- **Very high availability:** redundancy and failover support are built in.
- **Improved performance:** workload is distributed dynamically across the storage.
- **Easier data migration:** data moves from one storage system to another seamlessly.
- **Energy efficiency:** optimised storage usage draws less power.

2.8. The Data Center

Short description: A data centre is a centralised facility that houses computing resources: servers, storage systems, networking equipment, power systems, cooling systems and security infrastructure.

Theory: It is designed to store, process, manage and distribute large amounts of data and applications continuously and securely. In simple terms, the data centre is the backbone of modern IT infrastructure, providing the environment needed to run business applications, websites, cloud services, databases and enterprise systems.

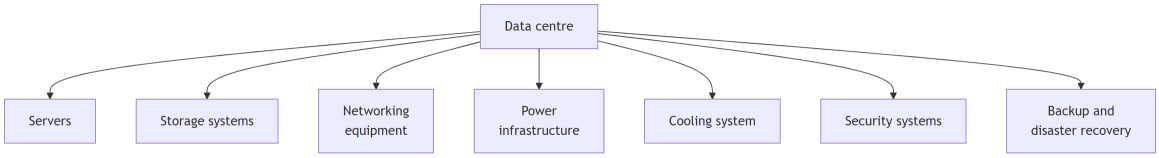
2.8.1 Need for a data centre

- Centralised storage and management of data
- High-speed data processing
- Secure storage of business information
- Continuous availability of services
- Disaster recovery and backup
- Support for cloud computing and virtualization
- Efficient resource utilization

2.8.2 Objectives of a data centre

- Provide reliable IT services
- Ensure high availability of applications
- Store and process large volumes of data
- Protect data against failures and cyber threats
- Improve business continuity
- Optimise resource utilization

2.8.3 Components

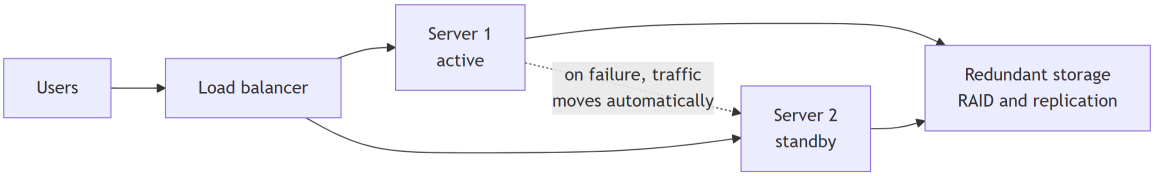


2.9. High Availability of a Data Center

Short description: High availability is the capability of a data centre to keep applications, servers, storage and network services available with minimal or no downtime, even when hardware, software or network failures occur.

Theory: The primary objective is continuous service to users, achieved by eliminating single points of failure and by enabling automatic recovery from failures.

2.9.1 Architecture diagram



2.9.2 Need for high availability

Organisations require uninterrupted service, for the following reasons.

- Continuous business operations
- Better customer satisfaction
- Prevention of financial loss
- Protection against hardware failures
- Data protection
- Disaster recovery
- Compliance with service level agreements
- Improved business reputation

2.9.3 Causes of data centre downtime

Category	Typical causes
Hardware failures	Server crash, hard disk failure, storage controller failure, memory failure, power supply unit failure
Software failures	Operating system crash, application bugs, database corruption, configuration errors
Network failures	Router failure, switch failure, ISP outage, cable damage
Human errors	Wrong configuration, accidental deletion, improper maintenance
Power failures	Power outage, UPS failure, generator malfunction

2.10. Data Center Maintenance

Short description: Data centre maintenance is the set of processes that keep a data centre operating as required, for example to meet an uptime objective such as 99.9 per cent or a service level agreement.

Theory: The tasks are monitoring and inspecting the equipment and systems, from hardware to electrical, cooling and cabling, and then carrying out the maintenance and repair work needed to meet the operational objectives.

2.10.1 Why it matters

Maintenance underpins most of the IT services a business runs. If the business runs its ERP software in its own data centre, maintenance is what keeps the ERP working. Maintenance is also cheaper than the alternative: system failures are stressful and costly to handle, and proactive work detects problems before they cause trouble.

The following problems arise from insufficient or unplanned maintenance.

- **Equipment failures:** neglected cooling leads to hot spots, which cause equipment to overheat and fail.
- **Power outages:** poor maintenance of the power system causes an outage, which disrupts service and can even lose data.
- **Cabling problems:** badly set up and managed cables lead to unintended disconnections.
- **Water damage:** ignoring water lines and temperature changes leads to flooding, which makes equipment fail.
- **Cleaning problems:** accumulated dust and debris interrupt air flow, damage sensitive equipment and can even start fires.

2.11. Planned Maintenance

Short description: Planned maintenance is maintenance scheduled in advance and carried out during predefined maintenance windows, so that disruption to users and business operations is minimal.

Theory: Because the work is planned, users are informed beforehand, backups are taken and contingency plans are prepared.

2.11.1 Objectives

- Improve system performance
- Prevent equipment failure
- Increase system reliability
- Enhance security
- Extend equipment lifespan
- Ensure business continuity

2.11.2 Characteristics

- Scheduled in advance
- Performed during maintenance windows
- Users are notified before the work starts
- Backup systems are prepared
- Low risk of unexpected failure
- Easy to manage and monitor

2.11.3 Activities

Area	Activities
Hardware	Cleaning servers and racks, replacing faulty hard disks, upgrading RAM, replacing power supplies and cooling fans, checking UPS batteries, generator maintenance. For example, replacing an aging hard disk before it fails.
Software	Operating system updates, security patches, application and database upgrades, firmware and driver updates. For example, applying the latest security patches to Linux servers.
Network	Router firmware updates, switch software upgrades, firewall configuration updates, cable inspection, network optimisation. For example, upgrading switch firmware for stability.
Storage	RAID health check, storage expansion, disk replacement, SAN firmware update, backup verification.
Security	Antivirus updates, firewall rule updates, access control review, password policy updates, security auditing.
Backup	Testing backups, verifying recovery procedures, restoring sample data, disaster recovery testing.
Cooling	Cleaning air filters, checking cooling units, temperature monitoring, airflow optimisation.
Power	UPS testing, generator testing, battery replacement, PDU inspection.

2.11.4 Advantages

- Reduces unexpected failures
- Improves system availability
- Enhances security
- Increases equipment life
- Reduces repair costs
- Better resource planning
- Minimises business disruption

2.11.5 Disadvantages

- Requires scheduled downtime, unless redundant systems are in place
- Needs careful planning
- May temporarily affect users
- Requires extra manpower and coordination

2.12. Unplanned Maintenance

Short description: Unplanned maintenance is unexpected maintenance caused by a sudden failure, fault or emergency. It is not scheduled in advance and needs immediate action to restore normal operation and keep downtime short.

2.12.1 Causes

- Hardware failures in servers, storage devices or network switches
- Power outages or UPS failure
- Cooling system malfunction
- Network connectivity issues
- Software crashes or operating system failures
- Cyberattacks such as malware, ransomware and DDoS
- Human errors such as accidental deletion or wrong configuration
- Fire, flood or other natural disasters

2.12.2 Common activities

1. **Emergency hardware replacement:** replace failed hard disks, SSDs, RAM, CPUs, power supplies or network cards, and faulty switches, routers or cables.
2. **Server recovery:** reboot crashed servers, restore failed virtual machines and recover operating systems from backup images.
3. **Storage recovery:** replace failed disks in RAID arrays, rebuild the RAID configuration and restore lost data from backups.
4. **Network troubleshooting:** find and repair broken links, replace damaged switches or routers and reconfigure network settings after a failure.
5. **Power system recovery:** repair or replace faulty UPS batteries, restore generator functionality and switch to backup power during a utility failure.
6. **Cooling system repair:** fix malfunctioning air conditioning units, restore temperature and humidity control and prevent servers from overheating.

2.12.3 Steps followed

1. Detect the problem through the monitoring systems.
2. Analyse and identify the root cause.
3. Isolate the affected equipment or service.
4. Repair or replace the faulty components.
5. Restore systems and services.
6. Test functionality to confirm normal operation.
7. Document the incident and the corrective actions.
8. Implement preventive measures so it does not recur.

2.12.4 Challenges

- Unexpected downtime and loss of productivity
- Risk of data loss
- Increased operational costs
- Service level agreement violations and customer dissatisfaction
- Higher workload for the IT staff

2.12.5 Best practices to reduce it

- Implement continuous monitoring and alerting.
- Perform regular preventive maintenance.
- Maintain redundant hardware and network paths.
- Keep reliable backups and disaster recovery plans.
- Use high availability and failover systems.
- Conduct periodic hardware health checks.
- Apply security updates and patches on time.

2.12.6 Planned versus unplanned maintenance

Planned maintenance	**Unplanned maintenance**
Scheduled in advance	Occurs unexpectedly
Preventive in nature	Corrective or emergency in nature
Minimal downtime	Downtime is often unavoidable
Lower maintenance cost	Higher maintenance cost
Better resource planning	Immediate resource allocation required
Reduces the risk of failures	Responds after failures occur

2.13. Server Migration

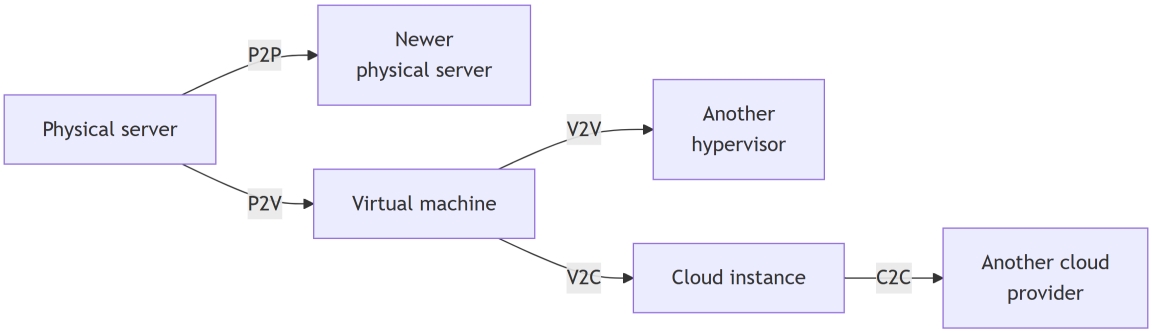
Short description: Server migration is the process of transferring a server's data, processes and configuration to a new target server or cloud instance.

Theory: Organisations migrate servers for resource optimisation, to reduce maintenance overhead, for better support and for deeper integration and modernisation. The process varies with the source and target machine architectures, and it needs careful planning, the right tools and testing to succeed.

2.13.1 Benefits

- **Update to modern services:** move from outdated and restrictive infrastructure to modern, scalable, maintainable platforms. In a cloud environment the servers are virtualized and are called instances.
- **Integration with other services:** modern server environments connect more easily to well-known tools and services, which enables faster automation.
- **Lower management overhead:** migrating to the cloud shifts maintenance from the organisation's own IT team to the hosting provider. On-premises hardware means the internal team handles updates and backups; on cloud infrastructure the provider does.
- **Cost reductions:** on-premises infrastructure is a capital expenditure, while cloud services are billed monthly, annually or pay as you go, which for many organisations is the more efficient budgeting model.
- **Security hardening:** modern platforms carry the latest encryption, identity controls and data security tools, although configuring the environment to suit the organisation's own requirements remains the customer's job.
- **Better performance:** cloud instances reach data held in the same cloud environment faster, and offer a wider range of storage and compute options.

2.13.2 Types of server migration



- **Physical to physical (P2P):** moving a workload from one physical server to another, for example replacing an old server with a newer, faster one.
- **Physical to virtual (P2V):** converting a physical server into a virtual machine. This gives better resource utilization, easier backup, lower hardware cost and simpler management.
- **Virtual to virtual (V2V):** migrating a VM from one virtualization platform to another, for example VMware to Hyper-V or VMware ESXi to KVM.
- **Virtual to cloud (V2C):** moving virtual machines from on-premises infrastructure to a cloud service.
- **Cloud to cloud (C2C):** migrating applications and data between cloud providers, for example AWS to Microsoft Azure or Azure to Google Cloud Platform.

2.13.3 Migration considerations

Business considerations

Migration should align with the business goals. The points to weigh are business continuity, downtime requirements, budget constraints, return on investment, project timeline and regulatory compliance. For example, a hospital migrating its patient databases must keep medical records reachable throughout the migration.

Hardware considerations

Verify that the destination environment meets the hardware requirements: processor architecture, number of CPU cores, RAM capacity, storage capacity, disk type (SSD or HDD) and network interface speed.

Software compatibility

Applications must still work after the move. Check operating system compatibility, database versions, middleware compatibility, drivers and application dependencies.

Data integrity

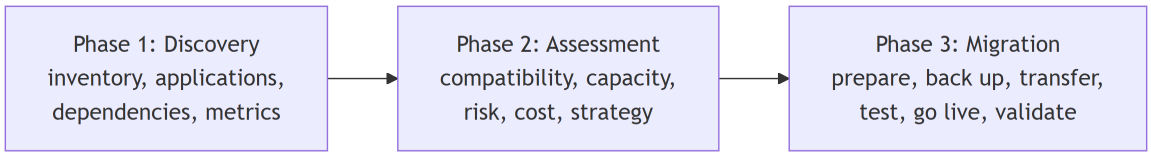
Data must stay accurate and complete throughout the migration. The activities are a full data backup, data validation, checksum verification and database consistency checks.

Self study

Network considerations, security considerations, performance considerations, downtime planning and risk management are left as self study.

2.14. Planning a Server Migration

A successful migration begins with careful planning. The pre-migration process has three phases: discovery, assessment and migration planning.



2.14.1 Phase 1: Discovery

Discovery is the process of collecting complete information about the existing IT infrastructure before the migration. Its purpose is to understand the current environment and identify every resource that has to move.

1. **Server inventory:** server name, IP address, operating system, CPU details, RAM, storage and installed software.
2. **Application discovery:** installed applications, running services, web servers, databases and middleware.
3. **Dependency mapping:** applications rarely work on their own, so identify the dependencies among applications, databases, file servers, authentication servers and external APIs.
4. **Performance monitoring:** CPU utilization, RAM usage, disk usage, network traffic, peak workload and storage growth. These metrics are what size the new environment correctly.
5. **Security discovery:** user accounts, access rights, certificates, firewall rules and encryption methods.

2.14.2 Phase 2: Assessment

Assessment analyses the information collected during discovery, to judge whether the migration is feasible and to prepare a strategy. Its objectives are to evaluate compatibility, estimate resources, identify risks, estimate costs and select the migration method.

1. **Compatibility assessment:** operating system support, application compatibility, database compatibility, driver support and hardware compatibility.
2. **Capacity planning:** estimate future CPU, memory, storage and network bandwidth requirements, so the new environment can carry both the current and the future workload.
3. **Risk assessment:** analyse downtime, performance degradation, security issues, data corruption and hardware failure, and assign a probability and an impact to each.
4. **Cost assessment:** hardware, cloud services, licensing, migration tools, staff training and maintenance.
5. **Migration strategy selection:** covered below.
6. **Backup planning:** take complete backups, verify their integrity, store them securely and prepare rollback procedures.

Migration strategies

Strategy	What it means	Notes
Rehosting	Move applications without changing their architecture, also called lift and shift.	Fast, low cost, minimal modifications.
Replatforming	Make minor optimisations while migrating.	For example, moving a database to a managed cloud database service.
Refactoring	Redesign the application for a cloud-native environment, also called re-architecting.	High scalability, better performance and long-term flexibility, but high cost and complexity.
Repurchasing	Replace the existing software with a SaaS product.	For example, replacing an on-premises email server with Microsoft 365.

2.14.3 Phase 3: Migration

Migration is the actual transfer of applications, operating systems, databases and data to the target environment.

1. **Prepare the target environment:** configure servers, virtual machines, storage, networks and security policies.
2. **Back up the existing systems:** operating system, applications, databases, configuration files and user data.
3. **Transfer the data:** by network transfer, storage replication, backup restoration, migration tools or cloud migration services.
4. **Migrate the applications:** install the applications, restore the databases, configure the services and update the configuration files.
5. **Test:** functional testing, performance testing, security testing and user acceptance testing, to confirm the applications work after the move.
6. **Go live:** redirect user traffic, update DNS records, activate production services and monitor system health.
7. **Post-migration validation:** verify data integrity, application functionality, user authentication, network connectivity, backup operations and performance.

2.14.4 Best practices

- Maintain a complete inventory of servers and applications.
- Perform full backups and verify their integrity.
- Test the migration in a staging environment before production.
- Schedule the migration during off-peak hours to minimise downtime.
- Monitor system performance throughout the migration.
- Prepare and test a rollback plan.
- Validate data integrity and application functionality afterwards.
- Document every step for future reference and audits.

2.14.5 Advantages of proper planning

Reduced downtime, improved application performance, better scalability, efficient resource utilization, lower operational costs, enhanced security, improved disaster recovery and simpler infrastructure management.

2.15. Networking Concepts: Masks and CIDR

Short description: A mask is what separates the network identifier from the host identifier in an IP address.

Theory: A mask is a 32-bit number made of a run of 1s followed by a run of 0s. It does not apply to classes D and E. Classless interdomain routing, or CIDR, writes the mask in the short form `/n`, where `n` is the number of 1 bits.

CIDR	Dotted decimal	Binary	Class
`/8`	`255.0.0.0`	`11111111 00000000 00000000 00000000`	A
`/16`	`255.255.0.0`	`11111111 11111111 00000000 00000000`	B
`/24`	`255.255.255.0`	`11111111 11111111 11111111 00000000`	C

Notes

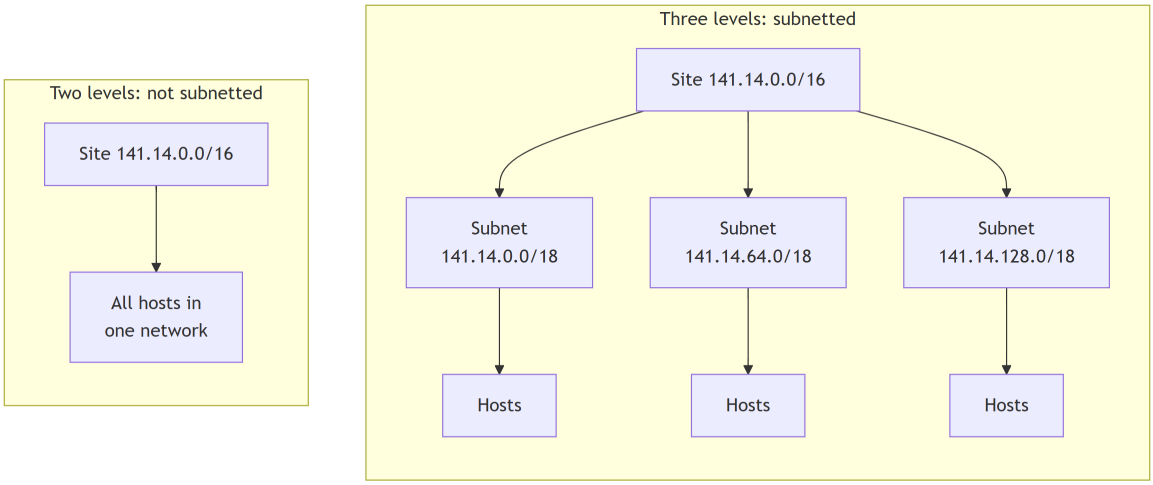
- **Subnetting:** an organisation granted a large block in class A or B divides the addresses into several contiguous groups and assigns each group to a smaller network, called a subnet.
- **Supernetting:** an organisation combines several class C blocks to create a larger range of addresses. Several networks are combined into one supernetwork, or supernet.

2.16. Subnetting

Short description: Subnetting divides a large granted block into several smaller networks, by borrowing bits from the host part and adding them to the network part.

Theory: Subnetting increases the number of 1s in the mask: a subnet mask has more 1s than the corresponding default mask. To build a subnet mask, change some of the leftmost 0s in the default mask to 1s. The number of subnets follows from the number of extra 1s: if there are n extra 1s there are 2^n subnets, and if N subnets are needed the number of extra 1s is $\log_2 N$. The number of subnets must therefore be a power of 2.

2.16.1 Levels of hierarchy



Without subnetting the address has two levels of hierarchy, network and host. With subnetting it has three: network, subnet and host.

2.16.2 Finding the subnetwork address

The subnet address is found the same way as the network address, by applying the mask to the address. There are two ways to do it.

- **Straight method:** write both the address and the mask in binary and apply the AND operation.
- **Short-cut method:** if the byte in the mask is 255, copy the byte of the address. If the byte in the mask is 0, replace the byte of the address with 0. If the byte is neither 255 nor 0, write that byte of the mask and of the address in binary and AND them.

Example: straight method

What is the subnetwork address if the destination address is `200.45.34.56` and the subnet mask is `255.255.240.0`?

Line	Binary
Address	`11001000 00101101 00100010 00111000`
Mask	`11111111 11111111 11110000 00000000`
Result	`11001000 00101101 00100000 00000000`

The subnetwork address is `200.45.32.0`.

Example: short-cut method

What is the subnetwork address if the destination address is `19.30.84.5` and the mask is `255.255.192.0`? The first two bytes of the mask are 255, so `19` and `30` are copied. The last byte of the mask is 0, so `5` becomes `0`. The third byte is neither, so AND it: `01010100` AND `11000000` is `01000000`, which is 64. The subnetwork address is `19.30.64.0`.

Example: designing subnets in class C

A company is granted the site address `201.70.64.0`, a class C address, and needs six subnets.

The default mask of class C has 24 ones. Six is not a power of 2, and the next power of 2 is $8 = 2^3$, so three more 1s are needed. The subnet mask has $24 + 3 = 27$ ones and $32 - 27 = 5$ zeros, which is `11111111 11111111 11111111 11100000`, or `255.255.255.224`. There are 8 subnets and each holds $2^5 = 32$ addresses.

Example: designing subnets in class B

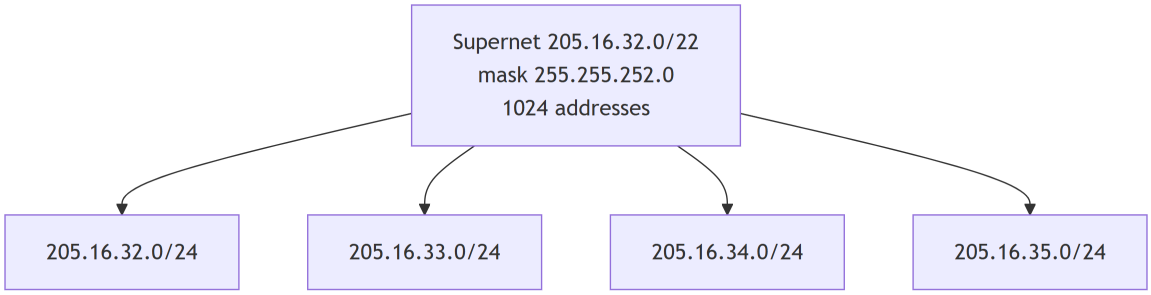
A company is granted the site address `181.56.0.0`, a class B address, and needs 1000 subnets.

The default mask of class B has 16 ones. One thousand is not a power of 2, and the next power of 2 is $1024 = 2^{10}$, so ten more 1s are needed. The subnet mask has $16 + 10 = 26$ ones and $32 - 26 = 6$ zeros, which is `11111111 11111111 11111111 11000000`, or `255.255.255.192`. There are 1024 subnets and each holds $2^6 = 64$ addresses.

2.17. Supernetting

Short description: Supernetting is the opposite of subnetting. Bits are borrowed from the network side to combine a group of networks into one large supernetwork.

Theory: Demand for midsize blocks is high. Class A and class B addresses are almost depleted, while class C addresses are still available but a block of 256 addresses is too small for many organisations. In supernetting, several class C blocks are combined into a larger range. This decreases the number of 1s in the mask: for four class C blocks the mask changes from `/24` to `/22`. An organisation needing 1000 addresses can be granted four contiguous class C blocks forming one supernetwork.



In subnetting, the first address of the subnet and the subnet mask define the range of addresses. In supernetting, the first address of the supernet and the supernet mask define the range.

Example: which packet belongs to the supernet

A supernet has the first address `205.16.32.0` and the supernet mask `255.255.248.0`. A router receives three packets, for `205.16.37.44`, `205.16.42.56` and `205.17.33.76`.

Applying the supernet mask gives `205.16.32.0`, `205.16.40.0` and `205.17.32.0` respectively. Only the first address belongs to this supernet.

Example: valid sets of blocks

A company needs 600 addresses. Which of these sets of class C blocks can form a supernet?

1. `198.47.32.0`, `198.47.33.0`, `198.47.34.0`. No, there are only three blocks.
2. `198.47.32.0`, `198.47.42.0`, `198.47.52.0`, `198.47.62.0`. No, the blocks are not contiguous.
3. `198.47.31.0`, `198.47.32.0`, `198.47.33.0`, `198.47.52.0`. No, 31 in the first block is not divisible by 4.
4. `198.47.32.0`, `198.47.33.0`, `198.47.34.0`, `198.47.35.0`. Yes, all three requirements are fulfilled.

Example: the supernet mask for 16 blocks

Sixteen blocks need four 1s of the default mask changed to 0s, so the mask is `11111111 11111111 11110000 00000000`, or `255.255.240.0`.

Example: size and range of a supernet

A supernet has the first address `205.16.32.0` and the supernet mask `255.255.248.0`. The mask has 21 ones and the default mask has 24, so the difference of 3 gives $2^3 = 8$ blocks. They run from `205.16.32.0` to `205.16.39.0`, so the first address is `205.16.32.0` and the last is `205.16.39.255`.

2.18. Classless Addressing

Short description: Classless addressing removes the classes altogether. Addresses are still granted in blocks, but the size of a block varies with the nature and size of the organisation.

Theory: Classless addressing was designed to overcome address depletion and give more organisations access to the Internet. A household may be given only two addresses, while a large organisation may be given thousands. It is referred to as classless interdomain routing, or CIDR, and the block granted is called a CIDR block.

2.18.1 Rules for allocating a block

1. The IP addresses in the CIDR block must all be contiguous.
2. The block size must be a power of 2, and the size of the block equals the number of IP addresses in it.
3. The first IP address of the block must be divisible by the block size.

For the beginning address, if the block has fewer than 256 addresses only the rightmost byte has to be checked; if it has fewer than 65,536 addresses only the two rightmost bytes have to be checked, and so on.

2.18.2 Slash notation

Slash notation, also called CIDR notation, is a shorter way of writing a subnet mask: instead of the full mask, only the number of 1 bits is written after a slash. A block in classes A, B and C is written `A.B.C.D/n`, where n is 8, 16 or 24 respectively.

2.18.3 Extracting information from an address

Given any address in the block and the prefix length n, three facts follow.

1. The number of addresses in the block is $N = 2^{32-n}$.
2. The first address is found by keeping the n leftmost bits and setting the $32 - n$ rightmost bits to 0.
3. The last address is found by keeping the n leftmost bits and setting the $32 - n$ rightmost bits to 1.

2.18.4 Address mask

The address mask is a 32-bit number whose n leftmost bits are 1 and whose remaining $32 - n$ bits are 0. A computer finds it easily because it is the complement of $2^{32-n} - 1$. Defined this way, the mask lets a program extract the block information with the bitwise NOT, AND and OR operations.

1. Number of addresses: N = NOT(mask) +1.
2. First address: (any address in the block) AND (mask).
3. Last address: (any address in the block) OR NOT(mask).

Example: range of a block

A small organisation is given the block `205.16.37.24/29`. To find the last address, keep the first 29 bits and change the last 3 bits to 1s.

Line	Binary
Beginning	`11001111 00010000 00100101 00011000`
Ending	`11001111 00010000 00100101 00011111`

There are 8 addresses in the block. The same answer follows from the suffix length: it is $32 - 29 = 3$, so there are $2^3 = 8$ addresses, and if the first address is `205.16.37.24` the last is `205.16.37.31`, because $24 + 7 = 31$.

Example: finding the network address

What is the network address if one of the addresses is `167.199.170.82/27`?

The prefix length is 27, so keep the first 27 bits and change the remaining 5 to 0s. Those 5 bits fall in the last byte, which is `01010010`. Changing the last 5 bits to 0 gives `01000000`, or 64. The network address is `167.199.170.64/27`.

Example: subnetting a classless block

An organisation is granted `130.34.12.64/26` and needs four subnets.

The suffix length is 6, so the block holds $2^6 = 64$ addresses. Split four ways, each subnet has 16 addresses. Four subnets need two more 1s on the site prefix, so the subnet prefix is `/28`.

Subnet	Range
1	`130.34.12.64/28` to `130.34.12.79/28`
2	`130.34.12.80/28` to `130.34.12.95/28`
3	`130.34.12.96/28` to `130.34.12.111/28`
4	`130.34.12.112/28` to `130.34.12.127/28`

Example: first and last address of a block

One address of a block is `205.16.37.39/28`. In binary it is `11001101 00010000 00100101 00100111`.

Setting the $32 - 28 = 4$ rightmost bits to 0 gives `11001101 00010000 00100101 00100000`, which is `205.16.37.32`. Setting the same 4 bits to 1 gives `11001101 00010000 00100101 00101111`, which is `205.16.37.47`.

The same result comes from the mask, which for `/28` is `11111111 11111111 11111111 11110000`. ANDing the address with the mask gives the first address. ORing it with the complement of the mask gives the last address. Complementing the mask, reading it as a decimal number and adding 1 gives the number of addresses.

Example: an ISP distributing a block

An ISP is granted `190.100.0.0/16` and has to serve three groups of customers: 64 customers needing 256 addresses each, 128 customers needing 128 addresses each, and 128 customers needing 64 addresses each.

Group	Suffix	Prefix	Sub-blocks
1	8	`/24`	`190.100.0.0/24` to `190.100.63.255/24`, total $64 \times 256 = 16{,}384$
2	7	`/25`	`190.100.64.0/25` to `190.100.127.255/25`, total $128 \times 128 = 16{,}384$
3	6	`/26`	`190.100.128.0/26` to `190.100.159.255/26`, total $128 \times 64 = 8{,}192$

The block granted holds 65,536 addresses and 40,960 of them are allocated, so 24,576 addresses are still available.

2.19. Practice Problems

Short description: Twenty-two worked problems covering classful addressing, subnetting, classless (CIDR) blocks, VLSM and supernetting, in the style of the examples above. Each question is followed immediately by its full solution.

Question 1

Find the network address of the host with IP address `168.14.32.12`, assuming the default classful mask.

Given

```
IP Address = 168.14.32.12
```

Step 1: Identify the Class

The first octet is **168**.

Class	Range
A	0–127
B	128–191
C	192–223

Since $128 \leq 168 \leq 191$, this is a **class B** address.

Step 2: Default Classful Mask

The class B default mask is:

`255.255.0.0` → `/16`

This means the **first two octets** identify the network, and the **last two octets** identify the host.

Step 3: Apply the Mask (Binary AND)

Line	Binary
IP (`168.14.32.12`)	`10101000 00001110 00100000 00001100`
Mask (`255.255.0.0`)	`11111111 11111111 00000000 00000000`
AND (`168.14.0.0`)	`10101000 00001110 00000000 00000000`

Final Answer

Term	Value
Class	B
Default mask	`255.255.0.0` (`/16`)
Network address	`168.14.0.0`

Question 2

Find the subnetwork address for IP address `200.45.34.56` and subnet mask `255.255.240.0`, using the straight binary method.

Given

```
IP Address = 200.45.34.56
Subnet Mask = 255.255.240.0
```

Step 1: Convert Both to Binary

Write each octet in 8-bit binary.

Octet	Decimal	Binary
IP 1	200	`11001000`
IP 2	45	`00101101`
IP 3	34	`00100010`
IP 4	56	`00111000`
Mask 1	255	`11111111`
Mask 2	255	`11111111`
Mask 3	240	`11110000`
Mask 4	0	`00000000`

So in full:

Line	Binary
IP (`200.45.34.56`)	`11001000 00101101 00100010 00111000`
Mask (`255.255.240.0`)	`11111111 11111111 11110000 00000000`

The mask has $8 + 8 + 4 = 20$ ones, so the prefix length is `/20`. The first two octets are fully masked (255), the fourth is fully unmasked (0), and the third octet is the only partial byte.

Step 2: Perform the Bitwise AND

AND each bit position: 1 AND 1 = 1, anything else = 0.

Line	Binary
IP	`11001000 00101101 00100010 00111000`
Mask	`11111111 11111111 11110000 00000000`
AND	`11001000 00101101 00100000 00000000`

Octet by octet: the first two octets pass through unchanged because the mask byte is 255; the fourth octet becomes 0 because the mask byte is 0; only the third octet needs real work, where `00100010` AND `11110000` gives `00100000`.

Step 3: Convert the Result Back to Decimal

Binary	Decimal
`11001000`	200
`00101101`	45
`00100000`	32
`00000000`	0

Final Answer

Term	Value
Prefix length	`/20`
Subnetwork address	`200.45.32.0`

Question 3

Find the subnetwork address for IP address `19.30.84.5` and subnet mask `255.255.192.0`, using the shortcut method.

Given

```
IP Address = 19.30.84.5
Subnet Mask = 255.255.192.0
```

Step 1: Convert the Mask to a Prefix

Line	Binary
Mask (`255.255.192.0`)	`11111111 11111111 11000000 00000000`

Counting the ones gives $8 + 8 + 2 = 18$, so the prefix is `/18`.

Step 2: Identify the Interesting Octet

The mask stops being all-ones in the **third octet** (value 192). By the shortcut rule: mask byte 255 means copy the address byte, mask byte 0 means write 0, and only a byte that is neither needs binary work. Hence the first two octets (`19.30`) carry over unchanged, the fourth octet becomes 0, and only the third octet must be calculated.

Step 3: Find the Block Size

```
Block Size = 256 − Mask value (interesting octet)
Block Size = 256 − 192 = 64
```

Step 4: Find the Subnet Containing the Third Octet

The third octet of the IP is **84**. List the multiples of 64:

Block	Range
0	0–63
64	64–127 (← 84 falls here)
128	128–191
192	192–255

So the subnet starts at **64**.

Step 5: Assemble the Subnetwork Address

```
First two octets unchanged:  19.30
Third octet:  64
Fourth octet:  always 0
```

Final Answer

Term	Value
Prefix length	`/18`
Block size	64
Subnetwork address	`19.30.64.0`

Question 4

A router receives a packet with destination address `190.240.224.33`. Find the subnetwork address if the subnet mask is `/19`.

Given

```
Destination Address = 190.240.224.33
Prefix = /19
```

Step 1: Convert the Prefix to a Mask

`/19` means 19 ones followed by 13 zeros:

Line	Binary
Mask (`/19`)	`11111111 11111111 11100000 00000000`
Decimal	`255.255.224.0`

Step 2: Identify the Interesting Octet

The mask breaks in the **third octet** (value 224). The first two octets are fully masked and carry over unchanged; the fourth octet is fully unmasked and becomes 0.

Step 3: Find the Block Size

```
Block Size = 256 − 224 = 32
```

Step 4: Find the Subnet Containing the Third Octet

The third octet of the IP is **224**. Multiples of 32 are 0, 32, 64, 96, 128, 160, 192, 224. Since 224 is itself an exact multiple of 32, it is the **start** of its own block:

```
224--255
```

Step 5: Assemble the Subnetwork Address

```
First two octets unchanged:  190.240
Third octet:  224
Fourth octet:  0
```

Binary Verification

Line	Binary
Address (`190.240.224.33`)	`10111110 11110000 11100000 00100001`
Mask (`255.255.224.0`)	`11111111 11111111 11100000 00000000`
Result (`190.240.224.0`)	`10111110 11110000 11100000 00000000`

Final Answer

Term	Value
Mask	`255.255.224.0` (`/19`)
Block size	32
Subnetwork address	`190.240.224.0`

Question 5

A router receives a packet with destination address `190.240.33.91`. Show how the router finds the network address and the subnetwork address, assuming the subnet mask is `/19`.

Given

```
Destination Address = 190.240.33.91
Prefix (subnet) = /19
```

Two different things are asked, using two different masks: the **network address** uses the default classful mask of the address class, while the **subnetwork address** uses the given `/19` mask.

Part A: Network Address (Classful)

Step A1: Identify the class. The first octet is 190. Since $128 \leq 190 \leq 191$, this is a **class B** address.

Step A2: Default mask. The class B default mask is `255.255.0.0` (`/16`): the first two octets are network, the last two are host.

Step A3: Apply the AND.

Line	Binary
IP (`190.240.33.91`)	`10111110 11110000 00100001 01011011`
Mask (`255.255.0.0`)	`11111111 11111111 00000000 00000000`
AND (`190.240.0.0`)	`10111110 11110000 00000000 00000000`

So the **network address is `190.240.0.0`**.

Part B: Subnetwork Address (/19)

Step B1: Convert the mask. `/19` is `11111111 11111111 11100000 00000000`, or `255.255.224.0`.

Step B2: Block size in the third octet. `Block Size = 256 − 224 = 32`.

Step B3: Locate the block. The third octet of the IP is **33**. Multiples of 32 give the blocks 0–31, 32–63, and so on; 33 falls in **32–63**, so the subnet starts at 32.

Step B4: Assemble. Keeping the first two octets and zeroing the last gives `190.240.32.0`.

Binary verification:

Line	Binary
IP	`10111110 11110000 00100001 01011011`
Mask (`/19`)	`11111111 11111111 11100000 00000000`
AND	`10111110 11110000 00100000 00000000`

Final Answer

Term	Value
Network address (classful, `/16`)	`190.240.0.0`
Subnetwork address (`/19`)	`190.240.32.0`

Question 6

A company is granted the class C address `201.70.64.0` and needs 6 subnets. Design the subnet mask and determine the borrowed bits, number of subnets, hosts per subnet, network addresses, broadcast addresses and valid host ranges.

Given

```
Network = 201.70.64.0/24 (Class C, default 8 host bits)
Subnets needed = 6
```

Step 1: Determine the Borrowed Bits

The number of subnets must be a power of 2, so find the smallest n with $2^n \geq 6$:

2^1 = 2 (not enough)
2^2 = 4 (not enough)
2^3 = 8 (enough)

Borrowed bits $n = 3$.

Step 2: New Prefix and Mask

```
New prefix = /24 + 3 = /27
New mask = 255.255.255.224
Last octet 11100000 = 224
```

Step 3: Number of Subnets Created

2^3 = 8 subnets (6 required, 2 remain spare for growth)

Step 4: Hosts per Subnet

```
Host bits = 32 − 27 = 5
Total addresses per subnet = 2^5 = 32
Usable hosts = 32 − 2 = 30 (subtract network + broadcast)
```

Step 5: Block Size

```
Block Size = 256 − 224 = 32
```

Step 6: List All Subnets

Subnet	Network	Broadcast	Usable host range
1	201.70.64.0	201.70.64.31	201.70.64.1–201.70.64.30
2	201.70.64.32	201.70.64.63	201.70.64.33–201.70.64.62
3	201.70.64.64	201.70.64.95	201.70.64.65–201.70.64.94
4	201.70.64.96	201.70.64.127	201.70.64.97–201.70.64.126
5	201.70.64.128	201.70.64.159	201.70.64.129–201.70.64.158
6	201.70.64.160	201.70.64.191	201.70.64.161–201.70.64.190
7*	201.70.64.192	201.70.64.223	201.70.64.193–201.70.64.222
8*	201.70.64.224	201.70.64.255	201.70.64.225–201.70.64.254

* Subnets 7 and 8 are spare capacity, since only 6 subnets were requested.

Final Answer

Term	Value
Borrowed bits	3
New mask	`255.255.255.224` (`/27`)
Number of subnets	8
Hosts per subnet	32 total, 30 usable

Question 7

A company is granted the class B address `181.56.0.0` and needs 1000 subnets. Determine the borrowed bits, new subnet mask, prefix length, number of subnets and hosts per subnet.

Given

```
Network = 181.56.0.0/16 (Class B, default 16 host bits)
Subnets needed = 1000
```

Step 1: Determine the Borrowed Bits

Find the smallest n with $2^n \geq 1000$:

2^9 = 512 (not enough)
2^{10} = 1024 (enough)

Borrowed bits $n = 10$.

Step 2: New Prefix and Mask

```
New prefix = /16 + 10 = /26
Third octet:  11111111 = 255
Fourth octet:  11000000 = 192
New mask = 255.255.255.192
```

Step 3: Number of Subnets Created

2^{10} = 1024 subnets

Step 4: Hosts per Subnet

```
Host bits = 32 − 26 = 6
Total addresses per subnet = 2^6 = 64
Usable hosts = 64 − 2 = 62
```

Final Answer

Term	Value
Borrowed bits	10
New subnet mask	`255.255.255.192` (`/26`)
Number of subnets	1024
Hosts per subnet	64 total, 62 usable

Question 8

An organisation is granted the block `130.56.0.0/16` and the administrator requires 1024 subnets. Find the number of addresses in each subnet, the subnet prefix, and the first and last address of the first and last subnets.

Given

```
Network = 130.56.0.0/16
Subnets needed = 1024
```

Step 1: Borrowed Bits

2^{10} = 1024 subnets exactly, so borrowed bits = 10

Step 2: New Prefix

```
New prefix = /16 + 10 = /26
```

Step 3: Addresses per Subnet

```
Host bits = 32 − 26 = 6
Addresses per subnet = 2^6 = 64
```

Step 4: Subnet Prefix and Mask

```
/26 = 255.255.255.192
```

Borrowing 10 bits splits the original 16-bit host space (third and fourth octets, values 0–65535) into 1024 blocks of 64 addresses each.

Step 5: First Subnet (Block Index 0)

The first block starts at offset 0 in the 16-bit host space:

```
First address = 130.56.0.0
Last address = 130.56.0.63
```

Step 6: Last Subnet (Block Index 1023)

```
Offset of last block = (1024 − 1) × 64 = 65472
65472 ÷ 256 = 255 remainder 192
Third octet = 255, fourth octet (start) = 192
First address of last subnet = 130.56.255.192
Last address of last subnet = 130.56.255.192 + 63 = 130.56.255.255
```

Final Answer

Term	Value
Addresses per subnet	64
Subnet prefix	`/26` (`255.255.255.192`)
First subnet	`130.56.0.0` to `130.56.0.63`
Last subnet	`130.56.255.192` to `130.56.255.255`

Question 9

A small organisation is given the block `205.16.37.24/29`. Find the first address, the last address and the number of addresses in the block, using both the binary method and the shortcut method.

Given

```
Address = 205.16.37.24/29
```

Method 1: Binary Method

Step 1: Convert to binary.

Line	Binary
Address (`205.16.37.24`)	`11001101 00010000 00100101 00011000`
Mask (`/29`)	`11111111 11111111 11111111 11111000`
Decimal mask	`255.255.255.248`

Step 2: AND to find the first address.

Line	Binary
Address	`11001101 00010000 00100101 00011000`
Mask	`11111111 11111111 11111111 11111000`
First (AND)	`11001101 00010000 00100101 00011000`

The AND leaves `205.16.37.24` unchanged, confirming it is already the first (network) address of its block.

Step 3: OR with the inverse mask to find the last address. The inverse mask is `00000000 00000000 00000000 00000111`:

Line	Binary
First address	`11001101 00010000 00100101 00011000`
Inverse mask	`00000000 00000000 00000000 00000111`
Last (OR)	`11001101 00010000 00100101 00011111`

`11001101 00010000 00100101 00011111` is `205.16.37.31`.

Method 2: Shortcut Method

Step 1: Block size. `/29` leaves $32 - 29 = 3$ host bits, so `Block Size =` 2^3 `= 8`, equivalently $256 - 248 = 8$.

Step 2: Locate the block. The fourth octet is 24. Multiples of 8 run 0, 8, 16, 24, 32, and so on; 24 is itself a multiple of 8, so it starts its own block, **24–31**.

Step 3: First and last address. The first address is `205.16.37.24` and the last is `205.16.37.31`.

Final Answer

Term	Value
First address	`205.16.37.24`
Last address	`205.16.37.31`
Total addresses	$2^3 = 8$

Both methods agree.

Question 10

One address in a block is `167.199.170.82/27`. Find the network address.

Given

```
Address = 167.199.170.82/27
```

Step 1: Convert the Mask

```
/27 = 255.255.255.224
Binary:  11111111 11111111 11111111 11100000
```

Step 2: Block Size

```
Block Size = 256 − 224 = 32
```

Step 3: Locate the Block

The fourth octet is 82. Multiples of 32 run 0, 32, 64, 96, 128, and so on:

```
64--95 (← 82 falls here)
```

So the network (first) address of the block uses the fourth octet 64.

Step 4: Network Address

```
167.199.170.64
```

Binary Verification

Line	Binary
Address (`167.199.170.82`)	`10100111 11000111 10101010 01010010`
Mask (`255.255.255.224`)	`11111111 11111111 11111111 11100000`
Result (`167.199.170.64`)	`10100111 11000111 10101010 01000000`

The last 5 bits of the final octet (`10010`) are cleared to `00000`, changing 82 to 64.

Final Answer

Term	Value
Network address	`167.199.170.64`

Question 11

One address in a block is `205.16.37.39/28`. Find the first address, last address, total addresses and usable host range, using the binary method and the mask method.

Given

```
Address = 205.16.37.39/28
```

Method 1: Binary Method

`/28` is `255.255.255.240`, or `11111111 11111111 11111111 11110000` in binary.

Line	Binary
IP (`205.16.37.39`)	`11001101 00010000 00100101 00100111`
Mask (`/28`)	`11111111 11111111 11111111 11110000`
AND (first address)	`11001101 00010000 00100101 00100000`

`11001101 00010000 00100101 00100000` is `205.16.37.32`, the network (first) address.
The last address is the network OR the inverse mask (`00000000 00000000 00000000 00001111`):

Line	Binary
First address	`11001101 00010000 00100101 00100000`
Inverse mask	`00000000 00000000 00000000 00001111`
Last (OR)	`11001101 00010000 00100101 00101111`

`11001101 00010000 00100101 00101111` is `205.16.37.47`.

Method 2: Mask (Shortcut) Method

```
Block Size = 256 − 240 = 16
```

The fourth octet is 39. Multiples of 16 run 0, 16, 32, 48, and so on; 39 falls in **32–47**, giving the same first address (`205.16.37.32`) and last address (`205.16.37.47`). Both methods match.

Total Addresses and Usable Range

```
Host bits = 32 − 28 = 4
Total addresses = 2^4 = 16
Usable hosts = 16 − 2 = 14
```

Final Answer

Term	Value
First address	`205.16.37.32`
Last address	`205.16.37.47`
Total addresses	16
Usable host range	`205.16.37.33`–`205.16.37.46`

Question 12

Each of the following addresses belongs to a block. Find the first address, last address and usable host range for each.

Given

```
(a) 14.12.72.8/24, (b) 200.107.16.17/18, (c) 70.110.19.17/16
```

(a) 14.12.72.8/24

Step 1: Block size. `/24` means the entire fourth octet is host space, so the block size is 256 addresses (the whole final octet); the network is fixed at the third octet.

Step 2: Network address. Since the mask covers exactly the first three octets, the third octet (72) does not change and only the fourth octet resets to 0.

```
First address = 14.12.72.0
Last address = 14.12.72.255
Usable host range = 14.12.72.1--14.12.72.254
```

(b) 200.107.16.17/18

Step 1: Mask. `/18` is `11111111 11111111 11000000 00000000`, or `255.255.192.0`.

Step 2: Block size in the third octet. `Block Size = 256 − 192 = 64`.

Step 3: Locate the block. The third octet is 16. Multiples of 64 are 0, 64, 128, 192; 16 falls in **0–63**.

Step 4: Addresses. Keeping the first two octets (200.107), setting the third octet to the block start (0), and spanning the full fourth octet:

```
First address = 200.107.0.0
Last address = 200.107.63.255
Usable host range = 200.107.0.1--200.107.63.254
```

(c) 70.110.19.17/16

Step 1: Mask. /16 is 255.255.0.0.

Step 2: Network address. The first two octets (70.110) are fixed; the last two reset to 0 for the first address and to 255 for the last.

```
First address = 70.110.0.0
Last address = 70.110.255.255
Usable host range = 70.110.0.1--70.110.255.254
```

Final Answer

Address	First	Last	Usable host range
(a) 14.12.72.8/24	14.12.72.0	14.12.72.255	14.12.72.1–14.12.72.254
(b) 200.107.16.17/18	200.107.0.0	200.107.63.255	200.107.0.1–200.107.63.254
(c) 70.110.19.17/16	70.110.0.0	70.110.255.255	70.110.0.1–70.110.255.254

Question 13

An organisation is granted the block 130.34.12.64/26 and requires 4 subnets. Design the subnets and show the address range of each.

Given

```
Block = 130.34.12.64/26
Subnets needed = 4
```

Step 1: Verify the Base Block

/26 gives host bits = 32 − 26 = 6, so block size = 2^6 = 64

Since the fourth octet is 64, and 64 is a multiple of 64, the given block spans:

```
130.34.12.64--130.34.12.127
```

Step 2: Borrowed Bits for 4 Subnets

2^2 = 4, so borrowed bits = 2

Step 3: New Prefix

```
New prefix = /26 + 2 = /28
New mask = 255.255.255.240
```

Step 4: New Block Size

Block Size = 256 − 240 = 16

Step 5: Divide the Range 64–127 into 4 Blocks of 16

Each new subnet holds 16 addresses ($16 - 2 = 14$ usable hosts). Starting at 64 and stepping by 16:

Subnet	Network address	Broadcast	Usable host range
1	130.34.12.64/28	130.34.12.79	130.34.12.65–130.34.12.78
2	130.34.12.80/28	130.34.12.95	130.34.12.81–130.34.12.94
3	130.34.12.96/28	130.34.12.111	130.34.12.97–130.34.12.110
4	130.34.12.112/28	130.34.12.127	130.34.12.113–130.34.12.126

Final Answer

The /26 block is split with 2 borrowed bits into four /28 subnets: 130.34.12.64/28, 130.34.12.80/28, 130.34.12.96/28 and 130.34.12.112/28, as tabulated above.

Question 14

An organisation is granted the block 14.24.74.0/24 and needs three sub-blocks: one of 120 addresses, one of 60 and one of 10. Design the VLSM allocation.

Given

```
Block = 14.24.74.0/24 (256 addresses total)
Sub-blocks required:  120, 60, 10 addresses
```

VLSM Principle

Always allocate the **largest** requirement first, using the smallest power-of-two block that is at least as big as the requirement, then continue allocating from where the previous block ended.

Step 1: Sort the Requirements (Descending)

```
120, then 60, then 10
```

Step 2: Sub-Block 1 for 120 Addresses

```
Smallest power of 2 ≥ 120 = 128 = 2^7
Host bits = 7, so prefix = 32 − 7 = /25
Range:  14.24.74.0--14.24.74.127 (128 addresses)
Sub-block 1 = 14.24.74.0/25
```

Step 3: Sub-Block 2 for 60 Addresses

```
Smallest power of 2 ≥ 60 = 64 = 2^6
Host bits = 6, so prefix = 32 − 6 = /26
```

It starts immediately after block 1 ends (127), that is at **128**:

```
Range:  14.24.74.128--14.24.74.191 (64 addresses)
Sub-block 2 = 14.24.74.128/26
```

Step 4: Sub-Block 3 for 10 Addresses

```
Smallest power of 2 ≥ 10 = 16 = 2^4
Host bits = 4, so prefix = 32 − 4 = /28
```

It starts after block 2 ends (191), that is at **192**:

```
Range:  14.24.74.192--14.24.74.207 (16 addresses)
Sub-block 3 = 14.24.74.192/28
```

Step 5: Remaining Address Space

```
Used:  128 + 64 + 16 = 208 addresses
Remaining:  256 − 208 = 48 addresses
Range:  14.24.74.208--14.24.74.255
```

Final Answer

Sub-block	Hosts needed	Prefix	Network address	Range
1	120	/25	14.24.74.0	14.24.74.0–14.24.74.127
2	60	/26	14.24.74.128	14.24.74.128–14.24.74.191
3	10	/28	14.24.74.192	14.24.74.192–14.24.74.207
Unused	—	—	14.24.74.208	14.24.74.208–14.24.74.255

Question 15

An organisation is granted the block 14.24.74.0/24 and needs three sub-blocks: one of 10 addresses, one of 60 and one of 120. Design the VLSM allocation.

Given

```
Block = 14.24.74.0/24 (256 addresses total)
Sub-blocks required (as listed):  10, 60, 120 addresses
```

Important VLSM Rule

The **order in which the requirements are listed does not determine the allocation order**. VLSM always allocates from **largest to smallest**, regardless of the order the sizes are mentioned in, because larger blocks need stricter address alignment and must be placed first to avoid fragmentation and wasted space. Sorting descending gives 120 → 60 → 10, exactly as in Question 14, so this question is mathematically identical and only the wording order differs.

Step 1: Sub-Block for 120 Addresses

2^7 = 128 ≥ 120, so prefix = /25
Range: 14.24.74.0--14.24.74.127

Step 2: Sub-Block for 60 Addresses

2^6 = 64 ≥ 60, so prefix = /26
Range: 14.24.74.128--14.24.74.191

Step 3: Sub-Block for 10 Addresses

2^4 = 16 $\geq$ 10, so prefix = /28
Range: 14.24.74.192--14.24.74.207

Step 4: Remaining Space

Used = 128 + 64 + 16 = 208
Remaining = 256 − 208 = 48
Range: 14.24.74.208--14.24.74.255

Final Answer

Requested size	Order	Prefix	Network address	Range
120	1st (largest)	/25	14.24.74.0	14.24.74.0–14.24.74.127
60	2nd	/26	14.24.74.128	14.24.74.128–14.24.74.191
10	3rd (smallest)	/28	14.24.74.192	14.24.74.192–14.24.74.207

The final allocation is identical to Question 14, because VLSM design depends on block sizes, not on the order they are requested in.

Question 16

An ISP is granted the block 190.100.0.0/16 and must serve 64 customers requiring 256 addresses each, 128 customers requiring 128 addresses each, and 128 customers requiring 64 addresses each. Determine the prefix for each group, the address ranges allocated, the total addresses used and the addresses remaining.

Given

ISP Block = 190.100.0.0/16 (65,536 addresses total)
Group A: 64 customers × 256 addresses each
Group B: 128 customers × 128 addresses each
Group C: 128 customers × 64 addresses each

Step 1: Prefix for Each Group

Group A: 256 = 2^8, host bits = 8, prefix = /24
Group B: 128 = 2^7, host bits = 7, prefix = /25
Group C: 64 = 2^6, host bits = 6, prefix = /26

Step 2: Total Address Demand per Group

```
Group A: 64 × 256 = 16,384 addresses
Group B: 128 × 128 = 16,384 addresses
Group C: 128 × 64 = 8,192 addresses
```

Step 3: Allocate in VLSM Order (Largest Block-Size First)

Group A (16,384 addresses, /24 each): 16,384 addresses is 64 × 256, spanning third-octet values 0–63:

```
Range:  190.100.0.0--190.100.63.255
Example:  190.100.0.0/24, 190.100.1.0/24, ...  190.100.63.0/24 (64 subnets)
```

Group B (16,384 addresses, /25 each), starting where Group A ended (third octet 64): 16,384 addresses again spans 64 third-octet values, 64–127:

```
Range:  190.100.64.0--190.100.127.255
Example:  190.100.64.0/25, 190.100.64.128/25, ...  (128 subnets, 2 per
third-octet value)
```

Group C (8,192 addresses, /26 each), starting where Group B ended (third octet 128): 8,192 addresses is 32 × 256, spanning third-octet values 128–159:

```
Range:  190.100.128.0--190.100.159.255
Example:  190.100.128.0/26, 190.100.128.64/26, 190.100.128.128/26, ...  (128
subnets, 4 per third-octet value)
```

Step 4: Total Used and Remaining

```
Total used = 16,384 + 16,384 + 8,192 = 40,960 addresses
Total available (/16) = 65,536 addresses
Remaining = 65,536 − 40,960 = 24,576 addresses
Remaining range:  190.100.160.0--190.100.255.255
```

Final Answer

Group	Customers	Addr./customer	Prefix	Range
A	64	256	/24	190.100.0.0–190.100.63.255 (16,384 addr.)
B	128	128	/25	190.100.64.0–190.100.127.255 (16,384 addr.)
C	128	64	/26	190.100.128.0–190.100.159.255 (8,192 addr.)

The block holds 65,536 addresses; 40,960 are allocated, leaving 24,576 addresses free in the range 190.100.160.0–190.100.255.255.

Question 17

An ISP is granted the block 16.12.64.0/20 and must allocate addresses to 8 organisations, each requiring 256 addresses. Find the number and range of addresses in the ISP block, the range allocated to each organisation, the unallocated range and a distribution outline.

Given

```
ISP Block = 16.12.64.0/20
Organisations = 8, each needing 256 addresses
```

Step 1: Total Addresses in the ISP Block

```
Host bits = 32 − 20 = 12
Total addresses = 2^12 = 4096
```

Step 2: Range of the ISP Block

/20 is 255.255.240.0, so the third-octet block size is $256 - 240 = 16$. The third octet starts at 64, giving:

```
64 + 16 − 1 = 79
Range:  16.12.64.0--16.12.79.255
```

Step 3: Prefix per Organisation

```
256 = 2^8, host bits = 8, prefix = /24
```

Step 4: Allocate the 8 Organisations Sequentially

Each organisation takes one full third-octet value, starting at 64:

Organisation	Range
Org 1	`16.12.64.0/24` (`16.12.64.0`–`16.12.64.255`)
Org 2	`16.12.65.0/24` (`16.12.65.0`–`16.12.65.255`)
Org 3	`16.12.66.0/24` (`16.12.66.0`–`16.12.66.255`)
Org 4	`16.12.67.0/24` (`16.12.67.0`–`16.12.67.255`)
Org 5	`16.12.68.0/24` (`16.12.68.0`–`16.12.68.255`)
Org 6	`16.12.69.0/24` (`16.12.69.0`–`16.12.69.255`)
Org 7	`16.12.70.0/24` (`16.12.70.0`–`16.12.70.255`)
Org 8	`16.12.71.0/24` (`16.12.71.0`–`16.12.71.255`)

Step 5: Total Used versus Unallocated

```
Used = 8 × 256 = 2048 addresses (16.12.64.0--16.12.71.255)
Total = 4096 addresses
Unallocated = 4096 − 2048 = 2048 addresses
Unallocated range:  16.12.72.0--16.12.79.255
```

Final Answer

Term	Value
ISP block range	`16.12.64.0`–`16.12.79.255`
Total addresses	4096
Prefix per organisation	`/24`
Allocated range	`16.12.64.0`–`16.12.71.255`
Unallocated range	`16.12.72.0`–`16.12.79.255`

Question 18

An ISP is granted the block `80.70.56.0/21` and must allocate addresses to two organisations requiring 500 addresses each, two requiring 250 each and three requiring 50 each. Find the number and range of addresses in the ISP block, the allocation for each organisation, the unallocated range and a distribution outline.

Given

```
ISP Block = 80.70.56.0/21
Group A: 2 organisations × 500 addresses
Group B: 2 organisations × 250 addresses
Group C: 3 organisations × 50 addresses
```

Step 1: Total Addresses in the ISP Block

Host bits = 32 − 21 = 11
Total addresses = 2^{11} = 2048

Step 2: Range of the ISP Block

/21 is 255.255.248.0, so the third-octet block size is $256 - 248 = 8$. The third octet starts at 56, and 56 is already an exact multiple of 8:

56 + 8 − 1 = 63
Range: 80.70.56.0--80.70.63.255

Step 3: Prefix for Each Group

For each group take the smallest power of 2 at least as big as the need:

Group A (500): $512 = 2^9$, prefix = /23
Group B (250): $256 = 2^8$, prefix = /24
Group C (50): $64 = 2^6$, prefix = /26

Step 4: VLSM Allocation (Largest First)

Group A: two /23 blocks (512 addresses each), starting at 80.70.56.0. A /23 spans two third-octet values:

Org A1: 80.70.56.0/23, range 80.70.56.0--80.70.57.255
Org A2: 80.70.58.0/23, range 80.70.58.0--80.70.59.255

Group B: two /24 blocks (256 addresses each), starting at 80.70.60.0:

Org B1: 80.70.60.0/24, range 80.70.60.0--80.70.60.255
Org B2: 80.70.61.0/24, range 80.70.61.0--80.70.61.255

Group C: three /26 blocks (64 addresses each), starting at 80.70.62.0. A /26 steps by 64 in the fourth octet:

Org C1: 80.70.62.0/26, range 80.70.62.0--80.70.62.63
Org C2: 80.70.62.64/26, range 80.70.62.64--80.70.62.127
Org C3: 80.70.62.128/26, range 80.70.62.128--80.70.62.191

Step 5: Unallocated Range

```
Used = (2 × 512) + (2 × 256) + (3 × 64) = 1728
Total = 2048
Unallocated = 2048 − 1728 = 320 addresses
Range:  80.70.62.192--80.70.63.255
```

Final Answer

Organisation	Addresses needed	Prefix	Range
A1	500	/23	80.70.56.0–80.70.57.255
A2	500	/23	80.70.58.0–80.70.59.255
B1	250	/24	80.70.60.0–80.70.60.255
B2	250	/24	80.70.61.0–80.70.61.255
C1	50	/26	80.70.62.0–80.70.62.63
C2	50	/26	80.70.62.64–80.70.62.127
C3	50	/26	80.70.62.128–80.70.62.191
Unallocated	—	—	80.70.62.192–80.70.63.255

Question 19

A supernet has first address 205.16.32.0 and mask 255.255.248.0. A router receives packets for 205.16.37.44, 205.16.42.56 and 205.17.33.76. Determine which packets belong to the supernet.

Given

```
Supernet first address = 205.16.32.0
Supernet mask = 255.255.248.0
```

Step 1: Convert the Mask to a Prefix

Line	Binary
Mask	11111111 11111111 11111000 00000000
Count	$8+8+5=21$ bits, so /21

Step 2: Find the Supernet Address Range

```
Block size (third octet) = 256 − 248 = 8
Third-octet range:  32 to (32 + 8 − 1) = 39
Supernet range:  205.16.32.0--205.16.39.255
```

Step 3: Test Each Destination Address

Address 1: `205.16.37.44`**.** First two octets match (`205.16`); third octet 37 satisfies $32 \leq 37 \leq 39$. **Belongs to the supernet.**

Address 2: `205.16.42.56`**.** First two octets match, but third octet 42 gives $42 > 39$. **Does not belong.**

Address 3: `205.17.33.76`**.** Second octet 17 does not equal 16, so it fails already at the second octet. **Does not belong.**

Verification by Binary AND (for Address 1)

Line	Binary
IP (`205.16.37.44`)	`11001101 00010000 00100101 00101100`
Mask (`255.255.248.0`)	`11111111 11111111 11111000 00000000`
AND (`205.16.32.0`)	`11001101 00010000 00100000 00000000`

The AND equals the supernet address, confirming membership.

Final Answer

Destination	Belongs to supernet?
`205.16.37.44`	Yes – third octet 37 lies in 32–39
`205.16.42.56`	No – third octet 42 is outside 32–39
`205.17.33.76`	No – second octet 17 does not match 16

Question 20

A supernet has first address `205.16.32.0` and mask `255.255.248.0`. Find the prefix length, the number of class C blocks aggregated, the first address, the last address and the total number of addresses.

Given

```
First address = 205.16.32.0
Mask = 255.255.248.0
```

Step 1: Prefix Length

Line	Binary
Mask	`11111111 11111111 11111000 00000000`
Count	$8 + 8 + 5 = 21$ bits, so prefix = `/21`

Step 2: Number of Class C Blocks Aggregated

Each class C block is a `/24` (256 addresses); this supernet is a `/21`:

```
Bits reclaimed = 24 − 21 = 3
Blocks aggregated = 2^3 = 8 class C networks
```

Step 3: First Address

Given directly: `205.16.32.0`.

Step 4: Last Address

```
Block size (third octet) = 256 − 248 = 8
Third-octet range:  32 to 32 + 8 − 1 = 39
Last address = 205.16.39.255
```

Step 5: Total Addresses

```
Host bits = 32 − 21 = 11
Total addresses = 2^11 = 2048
Cross-check:  8 class C blocks × 256 = 2048
```

Final Answer

Term	Value
Prefix length	`/21`
Class C blocks aggregated	8
First address	`205.16.32.0`
Last address	`205.16.39.255`
Total addresses	2048

Question 21

A company needs approximately 600 addresses. For each of the following sets of class C blocks, determine whether a valid supernet can be formed and explain why.

Given

```
Requirement:  about 600 addresses
Four candidate sets of /24 blocks (see below)
```

Rules for a Valid Supernet

A set of class C (`/24`) blocks can be combined into one valid supernet only if **all three** conditions hold: (1) the **number of blocks** is a power of 2 (1, 2, 4, 8, 16, ...), so a single contiguous mask can represent them; (2) the blocks are **contiguous** (consecutive network numbers with no gaps); (3) the **first block number is an exact multiple of the block count** (correct address alignment and boundary).

(a) 198.47.32.0, 198.47.33.0, 198.47.34.0

```
Number of blocks = 3
```

Three is not a power of 2 ($2^1 = 2$, $2^2 = 4$, with no exact match for 3). **Result: invalid.** A single supernet mask cannot represent exactly 3 class C networks; either 2 or 4 blocks would be needed.

(b) 198.47.32.0, 198.47.42.0, 198.47.52.0, 198.47.62.0

```
Number of blocks = 4 = 2^2 (power of 2)
```

Check contiguity: 32, 42, 52, 62 differ by 10 each, not by 1. **Result: invalid.** The blocks are not contiguous (ranges 33–41, 43–51 and 53–61 are missing), so they cannot be aggregated under one supernet mask.

(c) 198.47.31.0, 198.47.32.0, 198.47.33.0, 198.47.52.0

```
Number of blocks = 4 = 2^2 (power of 2)
```

Check contiguity: 31, 32, 33 are consecutive, but then the set jumps to 52, a large gap. **Result: invalid.** The fourth block is not contiguous with the first three, so no single supernet range can cover all four.

(d) 198.47.32.0, 198.47.33.0, 198.47.34.0, 198.47.35.0

```
Number of blocks = 4 = 2^2 (power of 2)
Contiguity:  32, 33, 34, 35 are consecutive
Alignment:  for 4 blocks the start must be a multiple of 4; 32 ÷ 4 = 8 exactly
```

Result: valid. The combined range is `198.47.32.0`–`198.47.35.255`, giving $4 \times 256 = 1024$ addresses under the mask `/22` (`255.255.252.0`), which covers the required 600 addresses.

Final Answer

Set	Blocks	Power of 2?	Contiguous?	Result
(a) `198.47.32/33/34.0`	3	No	Yes	**Invalid** – 3 is not a power of 2
(b) `198.47.32/42/52/62.0`	4	Yes	No	**Invalid** – gaps of 10 between blocks
(c) `198.47.31/32/33/52.0`	4	Yes	No	**Invalid** – block 52 breaks contiguity
(d) `198.47.32/33/34/35.0`	4	Yes	Yes	**Valid** – aligned, `198.47.32.0`–`198.47.35.255`

Question 22

Determine the supernet mask required to aggregate 16 contiguous class C networks. Show the binary calculation, the prefix length and the decimal mask.

Given

```
Number of class C networks to aggregate = 16
```

Step 1: Bits to Borrow (Reclaim) from the Network Portion

Each class C network is a `/24`. To combine 16 of them into one block:

2^n = 16, so n = 4
Reclaim 4 bits from the network portion

Step 2: New Prefix Length

```
New prefix = /24 − 4 = /20
```

Step 3: Binary Calculation of the Mask

`/20` in binary is 20 ones followed by 12 zeros:

Line	Binary
Mask (`/20`)	`11111111 11111111 11110000 00000000`
Breakdown	`(8 ones) (8 ones) (4 ones + 4 zeros) (all zeros)`

Convert each octet:

Binary	Decimal
`11111111`	255
`11111111`	255
`11110000`	240
`00000000`	0

Step 4: Decimal Mask

```
Supernet mask = 255.255.240.0
```

Step 5: Verify the Capacity

```
Host bits = 32 − 20 = 12
Total addresses = 2^12 = 4096
Cross-check:  16 class C blocks × 256 = 4096
```

Final Answer

Term	Value
Class C networks aggregated	16
Bits reclaimed	4
Prefix length	`/20`
Supernet mask	`255.255.240.0`
Total addresses	4096

Unit 3: Cloud Computing Fundamentals

3.1. Importance of Cloud Computing

Short description: Cloud computing lets an organisation obtain computing resources as a service instead of buying and running them itself.

Motivating example: Suppose tomorrow you want to start an online shopping website. What things do you need?

- Computer / server
- Storage
- Internet
- Database
- Operating system
- Software
- Security
- Application

The point: Every one of these can be rented from a cloud provider instead of bought and maintained, which is why cloud computing matters to a business getting started quickly.

3.2. Cloud Service Models

Short description: A cloud service model defines how much of the computing stack the provider manages on the consumer's behalf.

3.2.1 Overview

- **IaaS** – Infrastructure as a Service
- **PaaS** – Platform as a Service
- **SaaS** – Software as a Service

3.3. Infrastructure as a Service (IaaS)

Short description: IaaS gives the provider's access to fundamental resources such as physical machines, virtual machines and virtual storage.

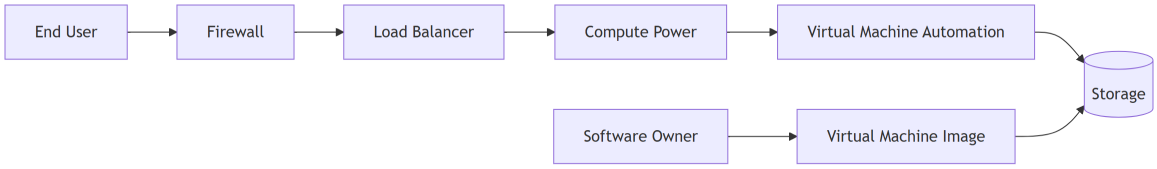
Also offered:

- Virtual machine disk storage
- Virtual local area networks (VLANs)
- Load balancers
- IP addresses

- Software bundles

How it works: All of the above resources are made available to the end user via server virtualization, and are accessed by the customer as if they own them.

3.3.1 Architecture diagram



3.3.2 Benefits

IaaS allows the cloud provider to freely locate the infrastructure over the Internet in a cost-effective manner. Key benefits:

- Full control of the computing resources through administrative access to VMs.
- Flexible and efficient renting of computer hardware.
- Portability and interoperability with legacy applications.

3.3.3 Issues

IaaS shares issues with PaaS and SaaS, such as network dependence and browser-based risks. It also has issues specific to itself:

Compatibility with legacy security vulnerabilities: Because IaaS lets the consumer run legacy software on the provider's infrastructure, it exposes the consumer to all of the security vulnerabilities of that legacy software.

Virtual machine sprawl: A VM can fall out of date with respect to security updates because IaaS allows the consumer to run, suspend or power off virtual machines at will. The provider can automatically update such VMs, but the mechanism is hard and complex.

Robustness of VM-level isolation: IaaS offers each consumer an isolated environment through the hypervisor, a software layer with hardware support for virtualization that splits one physical computer into multiple virtual machines.

Data erase practices: Consumers use virtual machines that share the provider's common disk resources. When a consumer releases a resource, the provider must ensure the next consumer to rent it does not see data residue from the previous one.

3.3.4 Characteristics

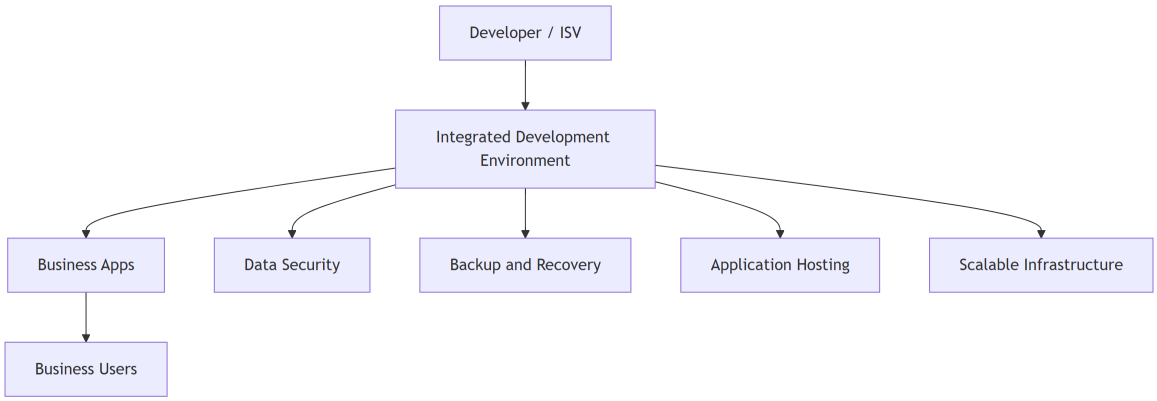
- Virtual machines with pre-installed software.
- Virtual machines with pre-installed operating systems, such as Windows or Linux.
- On-demand availability of resources.
- Ability to store copies of the same data in different locations.
- Computing resources that can be easily scaled up and down.

3.4. Platform as a Service (PaaS)

Short description: PaaS offers the development and deployment tools required to build applications, including point-and-click tools that let non-developers create web applications. Google App Engine and Force.com are example PaaS vendors – a developer logs into these platforms and uses their built-in API to build web applications.

Main drawback: Developer lock-in with a particular vendor. An application written in Python against Google's App Engine API is likely to work only in that environment, so vendor lock-in is the biggest problem in PaaS.

3.4.1 Architecture diagram



3.4.2 Benefits

Lower administrative overhead: The cloud provider is responsible for administration, so the consumer need not worry about it.

Lower total cost of ownership: The consumer need not purchase expensive hardware, servers, power or data storage.

Scalable solutions: Very easy to scale up or down automatically based on application resource demand.

More current system software: The provider is responsible for maintaining software versions and patch installations.

3.4.3 Issues

Lack of portability between PaaS clouds: Although standard languages are used, the implementation of platform services may vary. File, queue or hash-table interfaces of one platform may differ from another's, making it hard to move workloads between platforms.

Event based processor scheduling: PaaS applications are event oriented, which places resource constraints on them – they must answer a request within a given interval of time.

Security engineering of PaaS applications: Because PaaS applications depend on the network, they must explicitly use cryptography and manage security exposures.

3.4.4 Characteristics

- Browser-based development environment; developers create databases and edit application code via an API or point-and-click tools.

- Built-in security, scalability and web service interfaces.
- Built-in tools for defining workflow, approval processes and business rules.
- Easy to integrate with other applications on the same platform.
- Web service interfaces that connect applications outside the platform.

3.5. Software as a Service (SaaS)

Short description: SaaS provides a software application as a service to end users – software deployed on a hosted service and accessed via the Internet.

Example SaaS applications:

- Billing and invoicing systems
- Customer Relationship Management (CRM) applications
- Help desk applications
- Human Resource (HR) solutions

Customisation: Some SaaS applications are not customisable, such as an office suite. But SaaS also provides an API that lets a developer build a customised application on top.

3.5.1 Characteristics

- Software is made available over the Internet.
- The software is maintained by the vendor, not by where it runs.
- Licensing may be subscription based or usage based, and is billed on a recurring basis.
- Cost effective, since there is no maintenance burden on the end user.
- Available on demand, and can be scaled up or down on demand.
- Automatically upgraded and updated.
- Offers a shared data model, so multiple users share a single instance of the infrastructure without hard-coding functionality per user.
- All users run the same version of the software.

3.5.2 Benefits

- Modest software tools
- Efficient use of software licenses
- Centralised management and data
- Platform responsibilities managed by the provider
- Multitenant solutions

3.5.3 Issues

Browser based risks: If the consumer visits a malicious website and the browser is infected, subsequent access to a SaaS application can compromise the consumer's data. Using multiple browsers, dedicating one to SaaS access, or using a virtual desktop reduces the risk.

Network dependence: A SaaS application can only be delivered while the network is continuously available, and network reliability cannot be guaranteed by either the provider or the consumer.

Lack of portability between SaaS clouds: Transferring workloads from one SaaS cloud to another is not easy, because workflow, business logic, user interfaces and support scripts can all be provider specific.

Service model examples

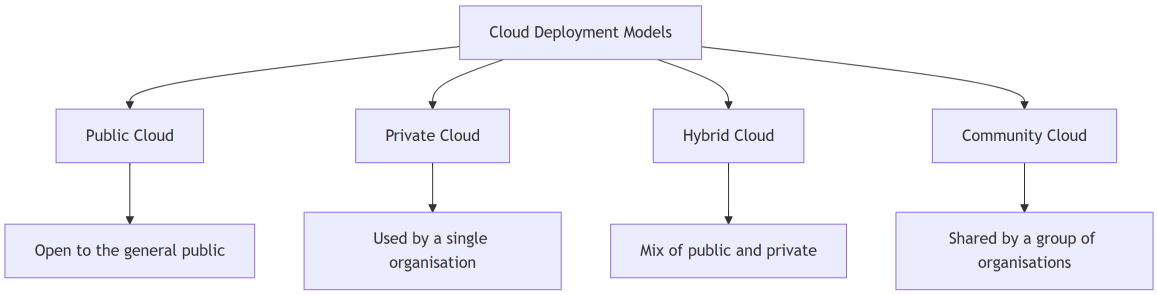
1. A company wants to host its website but does not want to purchase physical servers. It wants control over the operating system, storage and virtual machines. Identify the suitable cloud service model, justify the answer, and give one example.

2. A software development team wants to develop and deploy a web application without managing servers, operating systems or hardware. Identify the suitable cloud service model, explain why it is suitable, and give one example.

3. A college wants to provide students with online email and document-editing applications, accessed through a web browser without installing or maintaining software. Identify the suitable service model and give two reasons.

3.6. Cloud Deployment Models

3.6.1 Overview diagram



3.6.2 Public Cloud

Short description: The public cloud allows systems and services to be easily accessible to the general public – for example, Google, Amazon and Microsoft offer cloud services over the Internet.

Benefits:

- **Cost effective** – shares the same resources across a large number of consumers, so it has low cost.
- **Reliability** – draws on large numbers of resources from different locations, so a failed resource can be replaced by another one.
- **Flexibility** – easy to integrate with a private cloud, giving consumers a flexible approach.
- **Location independence** – services are delivered through the Internet.

- **Utility style costing** – pay-per-use, with resources accessible whenever the consumer needs them.
- **High scalability** – resources are made available on demand from a pool, and can be scaled up or down as required.

Disadvantages:

- **Low security** – data is hosted off-site on publicly shared resources, so it does not ensure a high level of security.
- **Less customisable** – comparatively less customisable than a private cloud.

3.6.3 Private Cloud

Short description: The private cloud allows systems and services to be accessible within a single organisation, which may manage it internally or through a third party.

Benefits:

- **Higher security and privacy** – not available to the general public, and resources are shared from a distinct pool, ensuring higher security and privacy.
- **More control** – more control over resources and hardware than a public cloud, because it is accessed only within the organisation.
- **Cost and energy efficiency** – not as cost effective as public clouds, but offers more efficiency than a public cloud.

Disadvantages:

- **Restricted area** – accessible only locally, and difficult to deploy globally.
- **Inflexible pricing** – purchasing new hardware to meet demand is very costly.
- **Limited scalability** – can be scaled only within the capacity of the internally hosted resources.

3.6.4 Hybrid Cloud

Short description: The hybrid cloud is a mixture of public and private cloud. Non-critical activities are performed using the public cloud, while critical activities are performed using the private cloud.

Benefits:

- **Scalability** – offers both public cloud scalability and private cloud scalability.
- **Flexibility** – offers both secure resources and scalable public resources.
- **Cost efficiencies** – public cloud is more cost effective than private, so the hybrid cloud gains this saving.
- **Security** – the private cloud portion ensures a higher degree of security.

Disadvantages:

- **Networking issues** – networking becomes complex due to the presence of both private and public cloud.
- **Security compliance** – it is necessary to ensure cloud services comply with the organisation's security policies.

3.6.5 Community Cloud

Short description: The community cloud allows systems and services to be accessible by a group of organisations, sharing infrastructure among several organisations from a specific community. It may be managed internally or by a third party.

Benefits:

- **Cost effective** – offers the same advantage as a private cloud at lower cost.
- **Sharing between organisations** – provides infrastructure to share cloud resources and capabilities among several organisations.
- **Security** – comparatively more secure than the public cloud.

Issues: Since all data is housed at one location, care is needed in storing data in a community cloud, because it might be accessible by others. Allocating responsibility for governance, security and cost is also challenging.

Deployment model examples

1. ABC E-Commerce has customers across India. During normal days it has 500 users, but during festival sales, users increase to 50,000. The company wants to quickly increase computing resources when demand increases and reduce them afterward, without purchasing and maintaining its own servers. Which cloud deployment model should ABC E-Commerce choose? Give a reason.

2. A government department stores highly confidential citizen information such as identity details, tax records and official documents. It requires complete control over its servers, network and security policies, and has its own IT team and budget to maintain infrastructure. Which cloud deployment model is most suitable? Why?

3. A bank wants to run its core banking system on infrastructure dedicated only to the bank, because of strict security and compliance requirements. However, it also wants to use cloud services for its public website and mobile application during periods of high traffic. Which cloud deployment model(s) should the bank use?

4. A company has two applications: Application A requires dedicated infrastructure and handles confidential employee data; Application B is an online shopping application whose users may increase from 1,000 to 100,000 during sales. Identify the suitable cloud deployment model for each application and justify the answer.

3.7. Regions, Availability Zones and Edge Locations

Short description: Cloud providers such as AWS, Microsoft Azure and Google Cloud have data centres distributed across the world. To organise this global infrastructure and provide reliable, fast and highly available services, they use the concepts of Regions, Availability Zones and Edge Locations.

3.7.1 Region

Definition: A Region is a geographical area of the world in which a cloud provider operates one or more data centres or Availability Zones. Providers divide their global infrastructure into regions so customers can choose where their cloud resources are deployed.

Example AWS regions: Mumbai, Singapore, Tokyo, London, Frankfurt, Northern Virginia, Sydney. Azure and Google Cloud have their own geographical regions.

Why regions are required: If a company operates in India with customers primarily located in India, but hosts its application in a distant location such as the United States, communication may take longer because data has to travel a greater distance. Deploying the application in an Indian region instead helps reduce network latency.

Characteristics:

- **Geographical location** – a region represents a specific geographical area.
- **Contains multiple Availability Zones** – a region is generally divided into multiple isolated Availability Zones.
- **Data residency** – organisations can choose a region based on where they want their data and applications hosted, subject to the provider's services and applicable regulations.
- **Disaster recovery** – organisations can use multiple regions for disaster recovery.
- **Reduced latency** – selecting a region closer to users reduces network latency.
- **Service availability** – not every cloud service is necessarily available in every region.

3.7.2 Availability Zone

Definition: An Availability Zone (AZ) is an isolated infrastructure location within a cloud Region. A Region contains multiple AZs, designed to operate independently enough that a failure in one AZ does not necessarily affect the others.

Why AZs are required: The main purpose of Availability Zones is high availability and fault tolerance. If a company runs its entire application in only one Availability Zone, and that AZ experiences a power failure, network failure, hardware failure, cooling failure or other infrastructure problem, the application may become unavailable. Distributing resources across multiple AZs avoids this.

Characteristics:

- **Isolation** – AZs are designed to be isolated from failures in other AZs.
- **Independent infrastructure** – each AZ has independent power, cooling, networking and physical facilities, reducing the chance a single failure affects all AZs.
- **High availability** – applications can be deployed across multiple AZs.
- **Fault tolerance** – using multiple AZs lets applications tolerate certain infrastructure failures.
- **Low-latency communication** – AZs within a region are connected by the provider's high-speed network, so applications distributed across AZs communicate efficiently.

3.7.3 Edge Location

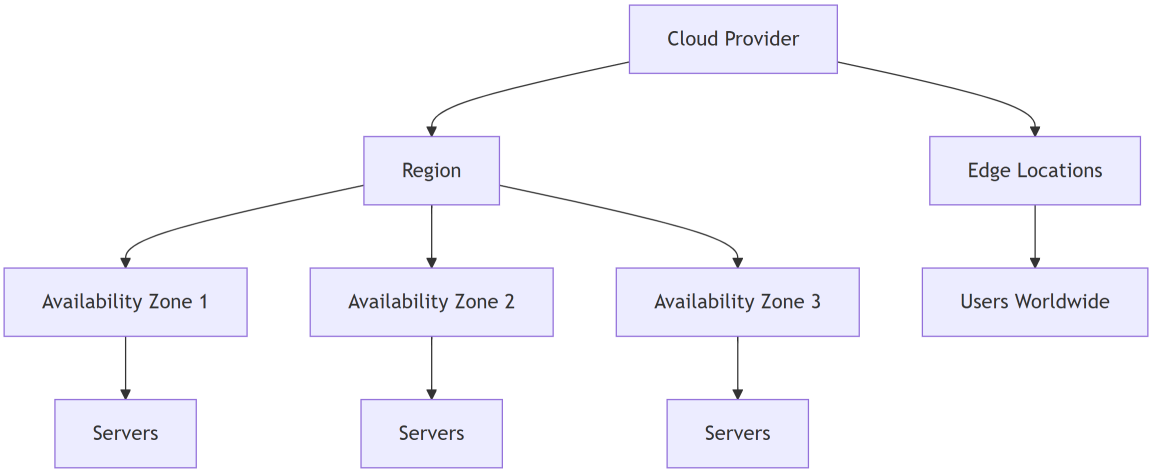
Definition: An Edge Location is a distributed location where a cloud provider's services can cache or deliver content closer to end users. Edge locations are particularly important for Content Delivery Networks (CDNs) – for example, AWS uses Amazon CloudFront to deliver content through a global network of edge locations.

Why edge locations are required: Consider a user in Nagpur accessing a website whose main server is in the United States. Without caching, the request must travel that long distance,

resulting in higher latency. With an edge location, the user instead receives cached content from a location closer to them, giving faster delivery and lower latency.

How they work: A website's images, videos, CSS files, JavaScript files, web pages and other static content are cached across edge locations. When a user requests this content, the CDN serves it from the nearest appropriate edge location.

3.7.4 Relationship between Region, AZ and Edge Location



- A **Region** is where cloud resources are deployed.
- An **Availability Zone** provides isolation and redundancy within a Region.
- An **Edge Location** brings frequently accessed content and services closer to users.
- If one AZ fails, traffic is served by the other AZs, which ensures high availability and fault tolerance.

3.7.5 Difference between Region, AZ and Edge Location

Parameter	Region	Availability Zone	Edge Location
Meaning	Geographical area	Isolated infrastructure location within a Region	Location closer to users for content delivery / caching
Main purpose	Resource deployment	High availability	Reduce latency
Contains	Multiple AZs	Infrastructure / data centres	CDN / cache infrastructure
Geographical size	Large	Smaller	Distributed near users
Main benefit	Location and regional resilience	Fault tolerance	Faster content delivery
Example	Mumbai Region	AZ within Mumbai Region	CloudFront edge location
Mainly used for	Compute, storage, databases, etc.	Redundant application deployment	CDN / content delivery
Failure impact	Regional failure can affect resources in that region	Failure can affect resources in that AZ	Cached content may be served from another location, depending on the service or design

Practice problems

1. An AWS Region has 3 Availability Zones. Each AZ contains 20 servers. How many servers are available in the entire Region?

2. An e-commerce company uses 2 Regions. Each Region has 3 AZs, and each AZ has 50 servers. Find the total number of servers deployed.

3. A cloud provider has 3 Regions. Each Region contains 2 AZs. Each AZ has 25 servers. Additionally, the provider has 4 Edge Locations per Region. Find: (i) total AZs, (ii) total servers, (iii) total Edge Locations.

3.8. Cloud Computing: Benefits and Challenges

Benefits	Challenges
Cost saving – reduces hardware and maintenance costs.	Security and privacy – risk of data breaches and unauthorized access.
Scalability – resources can be increased or decreased as needed.	Internet dependency – requires a reliable Internet connection.
Accessibility – access data and applications from anywhere.	Downtime – cloud service failures can interrupt operations.
Flexibility – easily adapt resources according to requirements.	Vendor lock-in – difficult to switch between cloud providers.
High availability – provides reliable access using redundancy.	Limited control – less control over the underlying infrastructure.
Automatic updates – provider manages updates and maintenance.	Data transfer issues – large data transfers can be slow or costly.
Disaster recovery – enables backup and recovery of data.	Compliance – must meet legal and regulatory requirements.
Faster deployment – resources can be deployed quickly.	Performance variations – performance may vary due to network or shared resources.

3.8.1 Benefits, in detail

Reduced total cost of ownership:

- Cost-effective – reduces hardware, storage, bandwidth and software costs.
- Scalable – suitable for users and organisations of all sizes.
- Pay-as-you-use – pay only for the resources consumed.
- Easy access – resources are managed through web services and APIs.
- Additional services – supports websites, billing, shopping and shipping.
- Business enablement – helps small businesses operate like established enterprises.

Increased scalability and reliability:

- Leverages the cloud provider's global infrastructure.
- Provides backup and fault tolerance.
- Reduces latency and improves performance.
- Easily handles peak demand – especially beneficial for small and medium businesses.

Increased test automation:

- Enables continuous build and automated testing.
- Provides on-demand test servers at scale.
- Virtualization supports testing across different hardware and software configurations.

- Easily performs large-scale stress and performance testing.
- Pay-as-you-use makes extensive testing cost-effective.

Enabling collaborative applications:

- Enables quick and flexible business partnerships.
- Supports easy sharing of information among partners.
- Facilitates collaboration in emerging markets and niche opportunities.
- Cloud platforms enable access to shared CRM and business applications.
- Helps businesses collaborate without complex infrastructure.

Reduced middleware tax:

- Reduces the cost and complexity of middleware.
- Simplifies application development and deployment.
- Minimises dependency on complex frameworks and tools.
- Reduces the burden of legacy middleware infrastructure.

Increased application development agility:

- Reduces dependency on middleware, databases and IT operations.
- Enables developers to work end-to-end independently.
- Cloud services provide simple, easy-to-use APIs.
- Speeds up application development and deployment, and improves team flexibility.

Increased end-user computing:

- Enables users to create customised solutions for their needs.
- Uses simple technologies like JavaScript and REST APIs.
- Mashup tools help users build personalised applications.
- Makes legacy data easily accessible through standard protocols, reducing the need for special-purpose applications.

3.8.2 The Challenges

Service Level Agreements: Define the service quality, availability and performance expected, and clarify provider and customer responsibilities. They help manage business risks and reliability; SLA monitoring tools ensure compliance, which matters for enterprise cloud adoption.

National data protectionism: Governments are concerned about data stored outside national borders. Data centres may be located far from the data owner's country, and laws may restrict data storage and transfer across borders, creating challenges for global cloud adoption and compliance.

Application security: Application developers are responsible for security beyond authorization. Critical security should be handled outside the browser, using strong encryption for data in transit and at rest and following SSL and other security best practices. Cloud platforms can provide enterprise-level security.

Loosely coupled service design: Enables applications to work with minimal dependency on each other, requiring standard protocols and consistent service design. ATOM publishing provides a simple service model, though developers must still manage different protocols and service interactions.

Query oriented versus API oriented programming: Cloud computing increasingly uses query-oriented programming. Technologies like MapReduce, LINQ and stream processing process large datasets using SQL/XQuery-style operations instead of complex APIs, so developers need skills in complex queries and data processing.

Occasionally disconnected operation:

- **Offline capability** – mobile applications must work offline and auto-sync with the cloud upon reconnection.
- **Client-side execution** – Google Gears runs web models locally on the client with server sync.
- **Sync frameworks** – Microsoft Azure offers flexible frameworks supporting varied data and file sync patterns.

3.9. Fundamentals of Pricing: Total Cost of Ownership

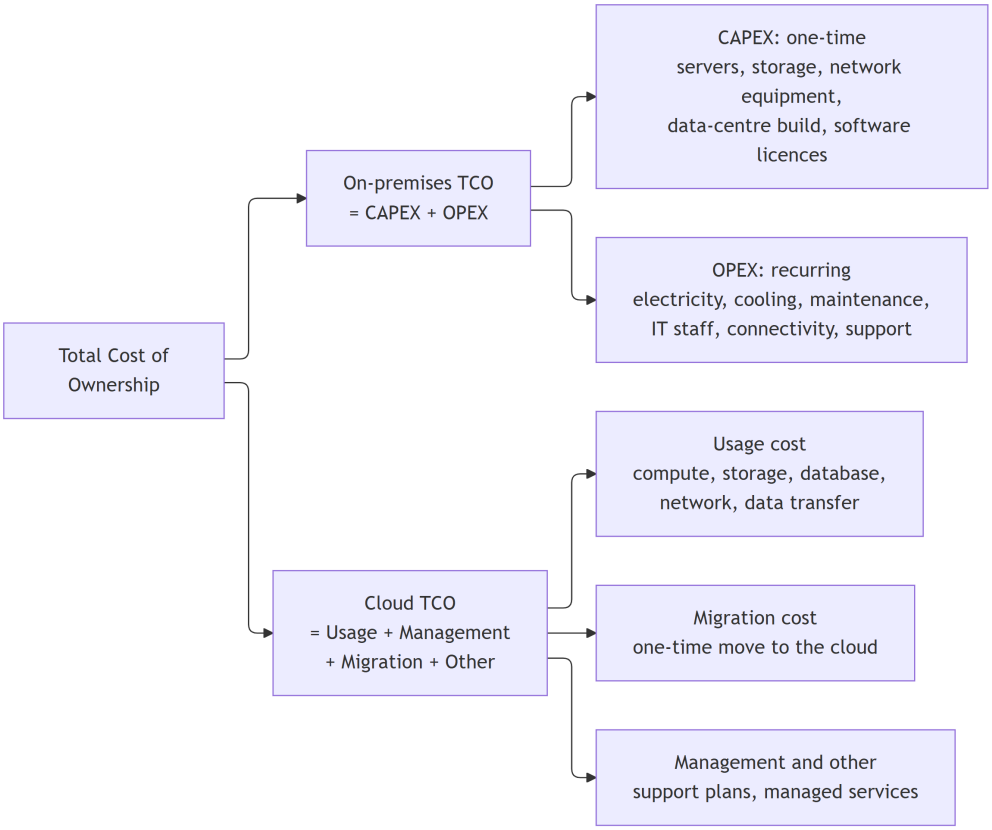
Short description: Total Cost of Ownership (TCO) is the total cost incurred to acquire, operate, maintain and eventually replace a technology or IT infrastructure over a specific period.

The question it answers: "How much will it *really* cost to run this workload on-premises, compared with running it in the cloud?"

Simple idea:

$$\text{TCO} = \text{Initial cost} + \text{Operating cost} + \text{Maintenance cost} + \text{Other associated costs}$$

3.9.1 Overview diagram



3.9.2 Need for TCO

Example: A company wants to run an application for 3 years. It can either **purchase and maintain its own servers** (on-premises) or **rent computing resources from a cloud provider** (cloud).

A common myth: "Cloud is always cheaper than on-premises." This is not necessarily true. The actual cost depends on:

- Number of users
- Workload
- CPU and memory requirements
- Storage requirements
- Network traffic
- Availability requirements
- Security requirements
- Number of years
- Cloud pricing model
- Existing infrastructure

Why TCO: TCO gives a complete cost comparison, rather than comparing only server prices. When calculating TCO, consider *all* costs associated with the infrastructure.

Major cost categories: Hardware, software, data centre, networking, power and cooling, IT staff, and maintenance.

3.10. On-Premises TCO

Short description: In an on-premises environment, the organisation owns and manages the infrastructure itself, so it pays for everything below.

Category	What the organisation pays for
Hardware	Servers, storage, network devices, backup devices
Software	Operating systems, database licences, virtualization software, security software
Facility	Data centre space, electricity, cooling, UPS, physical security
Operations	IT administrators, network administrators, system administrators, maintenance

3.10.1 CAPEX and OPEX

$$\text{On-premises TCO} = \text{CAPEX} + \text{OPEX}$$

CAPEX – Capital Expenditure	OPEX – Operational Expenditure
The initial, one-time investment:	The recurring costs:
1. Servers 2. Storage 3. Network equipment 4. Data centre infrastructure 5. Software licences	1. Electricity 2. Cooling 3. Maintenance 4. IT staff 5. Internet connectivity 6. Software support

3.11. Cloud TCO

Short description: In a cloud environment the organisation generally does not purchase physical servers. Instead, it pays for the cloud resources and services it uses.

Major cloud costs: Compute, storage, database, network and data transfer.

Simplified formula:

$$\text{Cloud TCO} = \text{Cloud usage cost} + \text{Management} + \text{Migration} + \text{Other costs}$$

3.11.1 Components of cloud TCO

Component	Includes
1. Compute cost	Virtual machines, containers, serverless functions
2. Storage cost	Object storage, block storage, file storage
3. Database cost	Managed SQL databases, NoSQL databases
4. Network cost	Data transfer, load balancers, VPN, CDN

3.12. Basic TCO Formulas

On-premises TCO:

TCO = Hardware+Software+Data centre+Power/Cooling+Networking+IT staff+Maintenance

Cloud TCO:

$$\text{TCO} = \text{Compute} + \text{Storage} + \text{Data transfer} + \text{Managed services} + \text{Support}$$

Percentage difference (on-premises cost as the base):

$$\text{Percentage difference} = \frac{\text{Cloud TCO} - \text{On-premises TCO}}{\text{On-premises TCO}} \times 100$$

3.13. On-Premises versus Cloud TCO

Cost factor	On-premises	Cloud-based
Servers	High initial cost	Usage-based
Storage	Purchase hardware	Pay for storage
Data centre	Required	Provider managed
Power	Organisation pays	Mostly included in service cost
Cooling	Organisation pays	Provider managed
IT staff	Required	Reduced infrastructure management
Maintenance	Organisation	Provider manages infrastructure
Scalability	Requires planning	Usually easier
Initial investment	High	Lower
Billing	Less variable	Usage dependent

3.14. TCO Numericals

All amounts are in rupees (Rs.). One year = 12 months, so 3 years = 36 months and 4 years = 48 months.

Problem 1: Three-year cloud TCO

A company wants to run an application for 3 years. Calculate the compute cost and the total cloud TCO.

Given

Virtual machines	4, each at Rs. 8/hour
Hours per month	720
Storage	Rs. 5,000/month
Network	Rs. 3,000/month
Other services	Rs. 4,000/month
Migration (one-time)	Rs. 50,000

Step 1: Compute cost

Four VMs per hour	$4 \times 8 =$ Rs. 32/hour
Monthly	$32 \times 720 =$ Rs. 23,040/month
Annual	$23{,}040 \times 12 =$ Rs. 2,76,480
Three years	$2{,}76{,}480 \times 3 =$ **Rs. 8,29,440**

Step 2: Other costs over 36 months

Storage	$5{,}000 \times 36 =$ Rs. 1,80,000
Network	$3{,}000 \times 36 =$ Rs. 1,08,000
Other services	$4{,}000 \times 36 =$ Rs. 1,44,000
Migration	Rs. 50,000 (one-time)

Final Answer

Component	**3-year cost (Rs.)**
Compute	8,29,440
Storage	1,80,000
Network	1,08,000
Other services	1,44,000
Migration	50,000
Total cloud TCO	**13,11,440**

Problem 2: On-premises versus cloud for 5 servers

A company needs 5 servers for 3 years. Calculate the 3-year on-premises TCO, the 3-year cloud TCO, and the percentage cost difference between the two.

Given

On-premises costs	**Cloud costs**
Cost of one server = Rs. 2,00,000	Virtual machine = Rs. 12,000/month/server
Software/licence per server per year = Rs. 30,000	Storage = Rs. 8,000/month
Maintenance per server per year = Rs. 20,000	Data transfer = Rs. 5,000/month
Power and cooling per server per year = Rs. 25,000	
Initial networking = Rs. 1,00,000	

Step 1: On-premises TCO for 3 years

Cost component	Calculation	3-year cost
Server cost	$5 \times 2{,}00{,}000$	10,00,000
Software/licence	$5 \times 30{,}000 \times 3$	4,50,000
Maintenance	$5 \times 20{,}000 \times 3$	3,00,000
Power and cooling	$5 \times 25{,}000 \times 3$	3,75,000
Initial networking	One-time	1,00,000
Total on-premises TCO		**22,25,000**

Step 2: Cloud TCO for 3 years (36 months)

Cost component	Calculation	3-year cost
Virtual machines	$12{,}000 \times 5 \times 36$	21,60,000
Storage	$8{,}000 \times 36$	2,88,000
Data transfer	$5{,}000 \times 36$	1,80,000
Total cloud TCO		**26,28,000**

Step 3: Percentage cost difference

$$\text{Difference} = 26{,}28{,}000 - 22{,}25{,}000 = \text{Rs. } 4{,}03{,}000$$

$$\text{Percentage difference} = \frac{4{,}03{,}000}{22{,}25{,}000} \times 100 = 18.11\%$$

Final Answer

Term	Value
On-premises TCO	Rs. 22,25,000 (22.25 lakh)
Cloud TCO	Rs. 26,28,000 (26.28 lakh)
Result	**Cloud costs 18.11% more than on-premises for this workload**

Problem 3: College student management system

A college is evaluating whether to maintain its own server or migrate its student management system to the cloud. Calculate the 3-year TCO for both alternatives and recommend the financially preferable option.

Given

On-premises server cost	Rs. 5,00,000
Maintenance and electricity	Rs. 1,00,000/year
Cloud service cost	Rs. 1,80,000/year
Evaluation period	3 years

Step 1: On-premises TCO

$$\text{On-premises TCO} = 5{,}00{,}000 + (1{,}00{,}000 \times 3) = 5{,}00{,}000 + 3{,}00{,}000 = \text{Rs. } 8{,}00{,}000$$

Step 2: Cloud TCO

$$\text{Cloud TCO} = 1{,}80{,}000 \times 3 = \text{Rs. } 5{,}40{,}000$$

Final Answer

Term	Value
On-premises TCO	Rs. 8,00,000
Cloud TCO	Rs. 5,40,000
Saving with cloud	Rs. 2,60,000 (32.5% of the on-premises cost)
Recommendation	**Migrate to the cloud** – it avoids the Rs. 5,00,000 upfront purchase and is cheaper over the 3-year period.

Problem 4: Monthly cloud bill with a discount

A startup uses a cloud server with the pricing below and receives a 10% discount on the total monthly bill. Calculate the monthly cloud cost after the discount.

Given

VM	Rs. 8/hour, 600 hours/month
Storage	Rs. 4/GB/month, 250 GB
Data transfer	Rs. 0.50/GB, 800 GB/month
Discount	10% of the total monthly bill

Step 1: Each component

VM	$600 \times 8 = \text{Rs. } 4{,}800$
Storage	$250 \times 4 = \text{Rs. } 1{,}000$
Data transfer	$800 \times 0.50 = \text{Rs. } 400$
Total before discount	$4{,}800 + 1{,}000 + 400 = \text{Rs. } 6{,}200$

Step 2: Apply the discount

Discount $= 10\% \times 6{,}200 =$ Rs. 620 Monthly cost $= 6{,}200 - 620 =$ **Rs. 5,580**

Problem 5: Four-year TCO for a university

A university is comparing two alternatives for a 4-year application. Calculate the 4-year TCO of both options.

Given

Option A: On-premises	**Option B: Cloud**
Server = Rs. 8,00,000	Migration = Rs. 60,000
Installation = Rs. 70,000	Monthly cloud cost = Rs. 38,000
Annual maintenance = Rs. 1,20,000	Annual backup cost = Rs. 50,000
Annual electricity = Rs. 80,000	
Hardware replacement after 3 years = Rs. 2,00,000	

Step 1: Option A, on-premises

Cost component	**Calculation**	**4-year cost**
Server	One-time	8,00,000
Installation	One-time	70,000
Maintenance	$1{,}20{,}000 \times 4$	4,80,000
Electricity	$80{,}000 \times 4$	3,20,000
Hardware replacement	Once, after year 3	2,00,000
Total		**18,70,000**

Step 2: Option B, cloud

Cost component	**Calculation**	**4-year cost**
Migration	One-time	60,000
Cloud usage	$38{,}000 \times 48$	18,24,000
Backup	$50{,}000 \times 4$	2,00,000
Total		**20,84,000**

Final Answer

On-premises TCO = **Rs. 18,70,000**; cloud TCO = **Rs. 20,84,000**. Over 4 years the on-premises option is cheaper by Rs. 2,14,000, so on cost alone Option A is preferable.

Problem 6: Pay-as-you-go versus a reserved plan

A company normally runs a cloud VM for 730 hours/month at Rs. 12/hour. It is considering a reserved plan costing Rs. 65,000 for 6 months instead. Storage costs Rs. 5/GB/month and the company uses 400 GB. Calculate the monthly cost under each model and the monthly saving with the reserved plan.

Step 1: Pay-as-you-go

VM	$730 \times 12 = \text{Rs. } 8{,}760$
Storage	$400 \times 5 = \text{Rs. } 2{,}000$
Total	**Rs. 10,760/month**

Step 2: Reserved plan

VM	$65{,}000 \div 6 = \text{Rs. } 10{,}833.33/\text{month}$
Storage	Rs. 2,000
Total	**Rs. 12,833.33/month**

Step 3: Monthly saving

$$10{,}760 - 12{,}833.33 = -\text{Rs. } 2{,}073.33$$

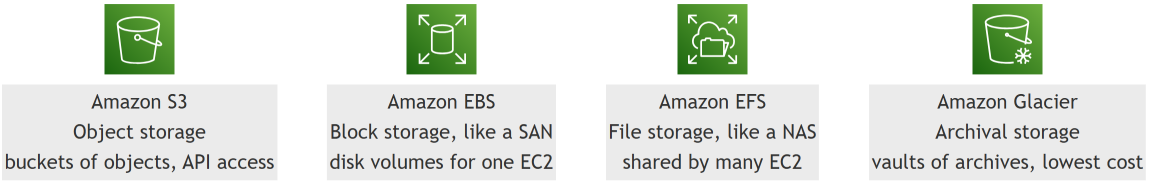
Final Answer

The "saving" is negative: the reserved plan costs **Rs. 2,073.33 more per month**, so the company should stay on pay-as-you-go. A reserved plan only pays off when its effective hourly rate is below the on-demand rate.

Unit 4: Amazon Web Services (AWS)

4.1. The AWS Storage Portfolio

Why AWS: "AWS makes it ever easier to experiment with technology and try new solutions that would be prohibitively costly or outright impossible for most companies to even consider." Many services can be tried on the AWS Free Tier (https://aws.amazon.com/free/).



Service	How data is presented and accessed
Amazon S3	Object storage: data presented as buckets of objects; accessed via APIs over the Internet.
Amazon EBS	Block storage (analogous to a SAN): data presented as disk volumes; lowest-latency access from a single EC2 instance.
Amazon Glacier	Archival storage: data presented as vaults of archives; lowest-cost storage with infrequent access via APIs over the Internet.
Amazon EFS	File storage (analogous to a NAS): data presented as a file system; shared low-latency access from multiple EC2 instances.

4.2. Amazon S3: Simple Storage Service

Short description: Amazon S3 is a fully managed **object storage** service that stores data as objects inside buckets, and is accessed over the web.

4.2.1 Overview

Object storage: S3 is object storage, not traditional file or block storage. It provides a file-system-like structure, but data is actually stored and managed as objects.

Introduced in 2006: S3 was one of the earliest services launched by AWS.

Fully managed: AWS handles the underlying storage infrastructure, maintenance and scalability.

Highly durable and scalable: Designed for high durability, security and scalability, protecting data against the loss of multiple facilities through redundant storage.

Pay-as-you-go pricing: No upfront infrastructure costs or minimum commitments; you pay based on actual usage, such as storage and data transfer.

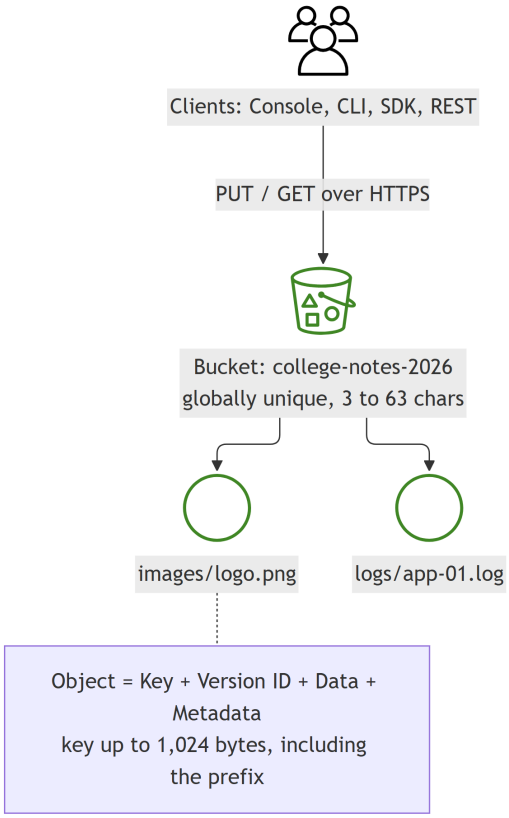
Integration with AWS services: Integrates easily with EC2 for computing, CloudFront for content delivery and RDS for database applications.

4.2.2 Access methods

Method	Description
Management Console	Web-based GUI for managing S3 resources.
AWS CLI	Command-line access on Windows, macOS, Linux and PowerShell.
API	Programmatic access using the REST API over HTTP/HTTPS.
AWS SDK	Integrate S3 into applications using Java, .NET, Python, Node.js, PHP, Ruby, Go, C++ and others.

4.2.3 Fundamental constructs

Buckets and objects: S3 stores data as **objects** within **buckets**. An object consists of a file and, optionally, any metadata that describes that file.



Buckets

- **Containers for objects** – a bucket stores and organises objects.
- **Namespace organisation** – buckets provide a unique namespace for objects.
- **Globally unique names** – each bucket name must be unique across all of S3.
- **Naming length** – a bucket name must contain 3 to 63 characters.
- **DNS-compatible naming** – bucket names must follow DNS naming conventions.
- **Bucket limit** – an account has a default service limit on the number of buckets, which can be increased when required.

Objects

- **Data and metadata** – an object is the actual data (file) plus metadata that describes it.
- **Unique identification** – each object is identified by its **key** (name) and, when versioning is enabled, a **version ID**.
- **Number of objects** – no fixed limit on objects per bucket.
- **Object size** – an individual object can be larger than 5 TB.
- **Different storage classes** – objects in the same bucket can belong to different storage classes.
- **Storage Inventory** – produces a flat-file listing of objects and their metadata, in formats such as CSV or JSON.

Keys

- **Unique identifier** – the object key identifies an object within a bucket.
- **Maximum length** – up to 1,024 bytes, including any path prefix such as `images/2026/`.

Structure of an S3 URL: `https://`*`bucket-name`*`.s3.amazonaws.com/`*`key`*, for example `https://college-no`

4.2.4 Storage classes

S3 Standard

- **Frequently accessed ("hot") data** – dynamic websites, content distribution, mobile and gaming applications, big-data analytics.
- **Low latency and high throughput.**
- **Durability** – 99.999999999% (11 nines); expected annual loss of about 0.000000001% of objects.
- **Availability** – 99.99%.
- **No minimum storage duration.**

S3 Standard-IA (Infrequent Access)

- **Infrequently accessed data that still needs high performance** – long-term storage, backups, disaster-recovery data.
- **Low latency and high throughput** when the data is needed.
- **Durability** – 11 nines. **Availability** – 99.9%.
- **Lower storage cost** than Standard, but a **retrieval fee** is charged.
- **Minimum storage duration** – 30 days.

S3 RRS (Reduced Redundancy Storage)

- **Non-critical, reproducible data** already stored safely elsewhere.
- **Protection against facility loss** – withstands the loss of data in a single facility.
- **Durability** – 99.99% (about 0.01% expected annual loss). **Availability** – 99.99%.

Amazon Glacier

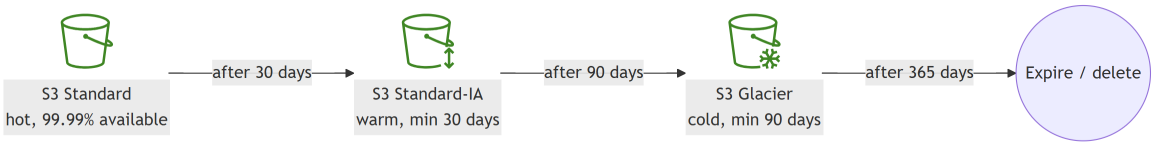
- **Cold storage** – long-term archiving of rarely accessed data.
- **Archives and vaults** – data is stored as archives, organised in vaults.
- **Durability** – 11 nines. **Availability** – 99.99%.
- **Retrieval options** – Expedited (about 1–5 minutes), Standard (about 3–5 hours), Bulk (about 5–12 hours).
- **Vault Lock** – stores data as immutable objects for compliance, preventing unauthorised change or deletion.
- **Minimum storage duration** – 90 days.

Comparison

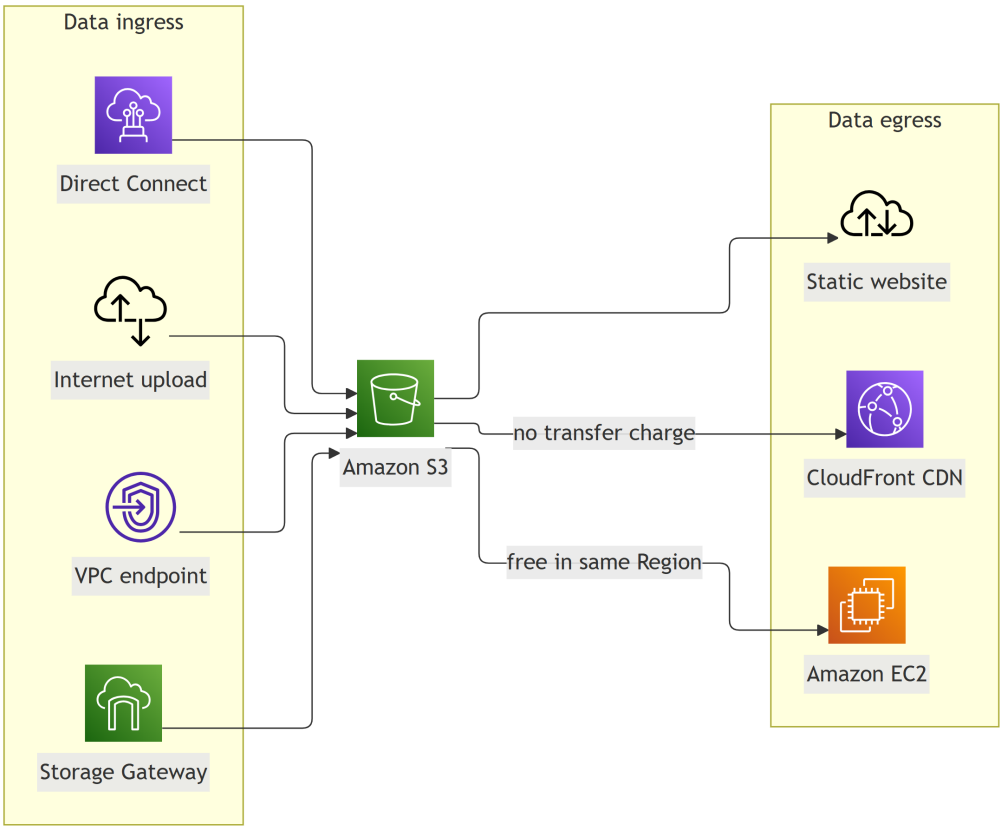
Property	Standard	Standard-IA	RRS	Glacier
Access pattern	Frequent (hot)	Infrequent (warm)	Non-critical	Rare (cold)
Durability	11 nines	11 nines	99.99%	11 nines
Availability	99.99%	99.9%	99.99%	99.99%
Minimum duration	None	30 days	–	90 days
Retrieval	Immediate	Immediate + fee	Immediate	Minutes to hours
Use it for	Reliable, durable primary storage	Backups, DR	Secondary backups at lower cost	Lowest-cost archival

4.2.5 Lifecycle policies

- Automatically **transition** objects between storage classes based on their access pattern – for example Hot (Standard) → Warm (IA) → Cold (Glacier).
- Automatically **expire** (delete) objects after a specified number of days.
- Multiple actions can be combined or staged – for example, archive first and delete later. Separate rules can be defined for current and previous object versions.
- Rules apply to an entire bucket or to objects with a given prefix, such as `logs-`.



4.2.6 Data ingress and egress



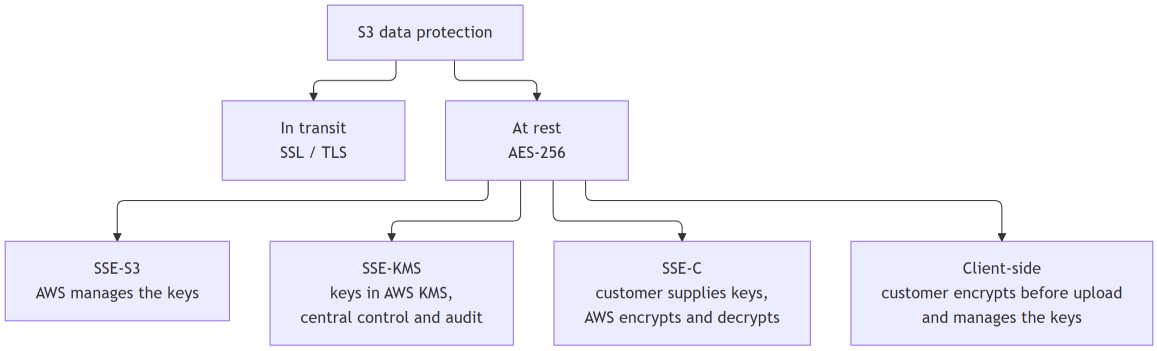
Ingress (getting data in):

- **Internet upload** – buckets can accept uploads over the Internet, either anonymously or with time-limited authentication.
- **VPC endpoints** – private transfer between a VPC and S3 without crossing the public Internet.
- **AWS Direct Connect** – a dedicated network connection between an organisation's network and AWS.
- **AWS Storage Gateway** – connects on-premises storage to AWS cloud storage.

Egress (serving data out):

- **Static website hosting** – HTML, CSS, JavaScript, images and videos.
- **Origin for CDNs** – a durable origin for CloudFront and other CDNs.
- **Transfer to CloudFront** – no charge for data transfer from S3 to CloudFront.
- **Transfer to EC2** – generally free between S3 and EC2 in the same Region.

4.2.7 Security



Mechanism	Who encrypts, who holds the keys
Encryption in transit	SSL/TLS protects data travelling to and from S3.
Encryption at rest	Stored data encrypted with AES-256 and other supported mechanisms.
SSE-S3	Server-side encryption; AWS manages the keys.
SSE-KMS	Server-side encryption with AWS Key Management Service: centralised control over keys and access permissions.
SSE-C	Server-side encryption with customer-provided keys; AWS performs the encryption and decryption.
Client-side encryption	The customer encrypts data before uploading and manages the process and keys.

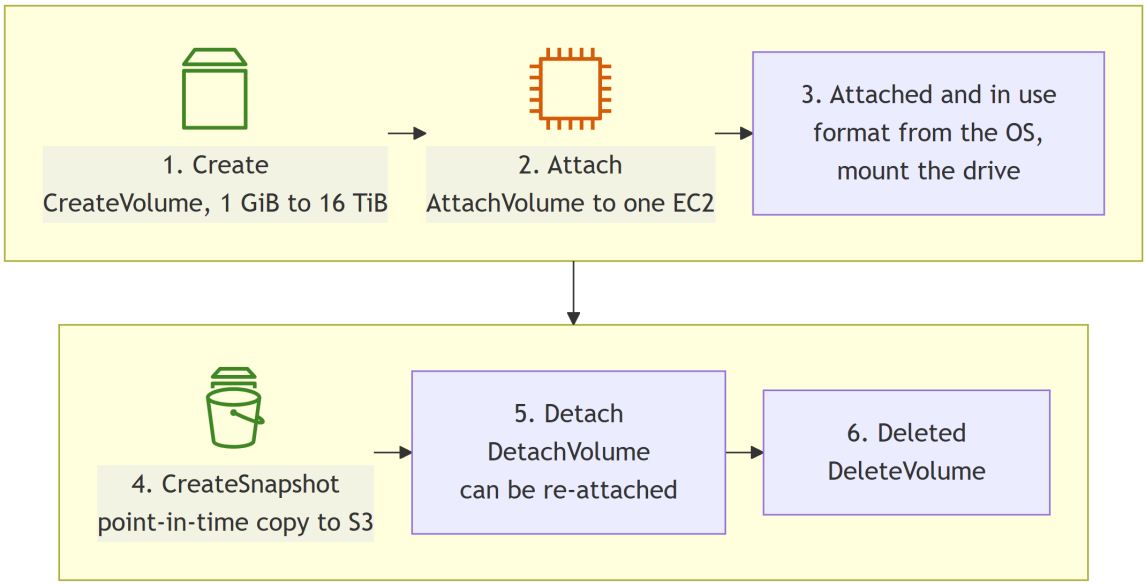
4.3. Amazon EBS: Elastic Block Store

Short description: Amazon EBS is a cloud-based **block storage** system that provides durable, persistent block-level volumes for use with EC2 instances.

4.3.1 Introduction

- Best used for storing **persistent data**.
- Volumes are created and attached through an API, and used like hard drives.
- **Availability Zone specific** – a volume lives in one AZ.
- Offers consistent, low-latency performance.
- Stored data is automatically **replicated within its Availability Zone**.

4.3.2 EBS lifecycle



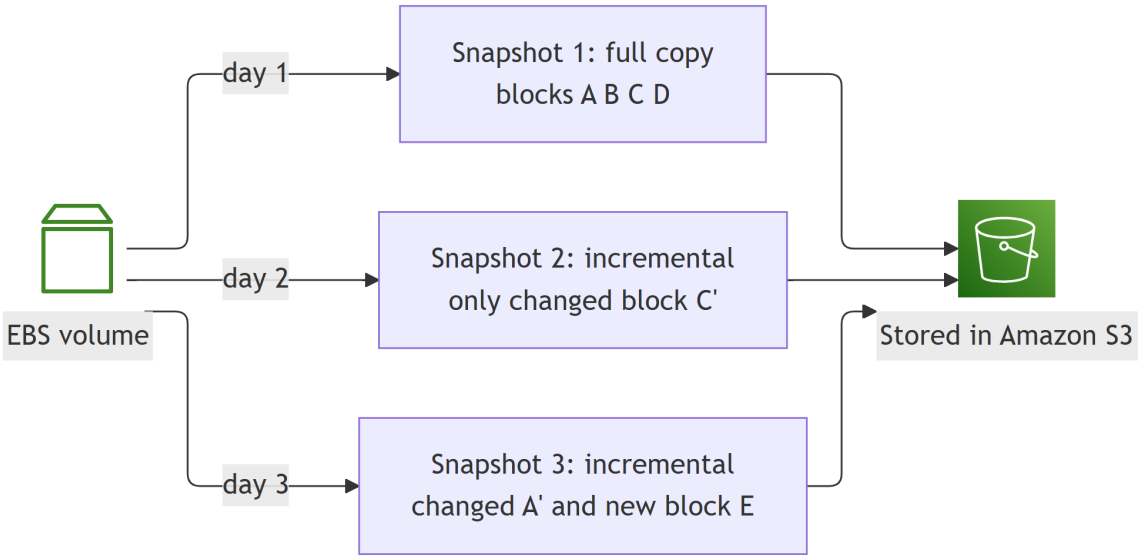
1. **Create** – call `CreateVolume` (1 GiB to 16 TiB) out of the vast pool of unused space.
2. **Attach** – call `AttachVolume` to associate it with one EC2 instance.
3. **Attached and in use** – format it from the EC2 instance's OS and mount the formatted drive.
4. **CreateSnapshot** – take a snapshot to Amazon S3.
5. **Detach** – call `DetachVolume`; the volume can be re-attached later.
6. **Deleted** – call `DeleteVolume`.

4.3.3 Volume types

Type	API	Size	Use cases	IOPS
General Purpose SSD	`gp2`	1 GB – 16 TB	Boot volumes, low-latency interactive apps; balances price and performance	100 – 16,000
Provisioned IOPS SSD	`io1`	4 GB – 16 TB	Relational databases, I/O-intensive NoSQL; highest-performance SSD	100 – 64,000
Throughput Optimized HDD	`st1`	500 GB – 16 TB	Big data, data warehouses, log processing; low cost, throughput intensive	max 500
Cold HDD	`sc1`	500 GB – 16 TB	Colder data with fewer scans per day; lowest cost; cannot be a boot volume	max 250
Magnetic (Standard)	`standard`	1 GB – 1 TB	Previous generation; infrequently accessed data	≈100, bursts to hundreds

4.3.4 EBS snapshots

- A snapshot is an **incremental** copy of data – "block-level incremental".
- A **point-in-time backup** of the modified volume blocks.
- Stored in S3, and accessed via the EBS APIs from the EC2 dashboard.
- Can be shared with specific AWS accounts or made public.
- The **first** snapshot is a full clone; **subsequent** snapshots are incremental.
- Deleting a snapshot removes only the data exclusive to that snapshot.



4.3.5 Features and benefits

Features	Benefits
Persistent storage	Reliable and secure storage
Volumes behave like raw, unformatted block devices	Optimised performance
High-performance file system, high reliability	Data backup and restore
Flexible volume size	Quickly scale up, easily scale down
Private to your instances	Consistent, low-latency performance
Replicated within an Availability Zone	Flexible to detach and attach
Snapshots for point-in-time restore	

4.4. Amazon EFS: Elastic File System

Short description: Amazon EFS is a fully managed **file storage** service: a file system that many EC2 instances can mount and share at the same time over NFS v4.

4.4.1 Simple, elastic, scalable

1. Simple	2. Elastic	3. Scalable
Fully managed: no hardware, network or file layer to run; a file system in seconds.	Grows and shrinks automatically as files are added and removed.	Grows to petabyte scale.
Works with existing tools and apps: NFS v4, standard file-system semantics and OS file APIs.	No need to provision storage capacity or performance.	Throughput and IOPS scale automatically with the file system.
Simple pricing, so simple forecasting.	Pay only for the storage used, with no minimum fee.	Consistent low latency at any size; thousands of concurrent NFS connections.

4.4.2 File systems and mounting

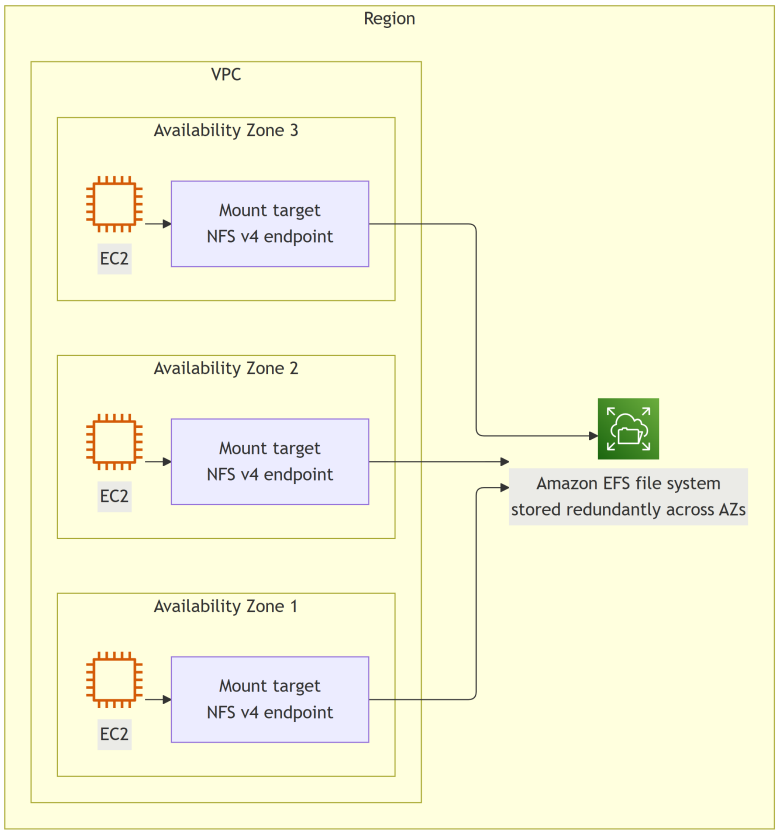
File system: The primary resource in EFS – where files and directories are stored.

Mounting: You *mount* a file system on an EC2 instance with a standard command; it then appears as a local set of directories and files. An NFS v4 client is standard on Linux distributions.

```
sudo mount -t nfs4 fs-12345678.efs.ap-south-1.amazonaws.com:/ /mnt/efs
```

Mount target: To reach the file system from instances in a VPC, you create **mount targets** in the VPC. A mount target is an NFS v4 endpoint with an IP address and a DNS name, used in the mount command – one per Availability Zone.

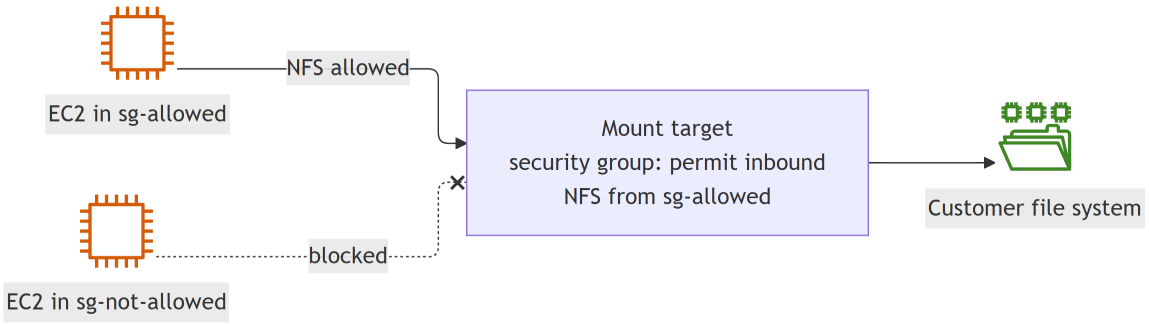
4.4.3 How it fits together



Ways to set up and manage a file system: AWS Management Console, AWS Command Line Interface (CLI), AWS Software Development Kit (SDK).

4.4.4 Securing a file system

- **VPC only** – only EC2 instances in the VPC you specify can access the file system.
- **Security groups** – control which instances in the VPC can connect to the mount targets.
- **File permissions** – user-level read-write-execute permissions for users and groups on files and directories.
- **IAM integration** – IAM policies control who may use the administrative APIs to create, manage and delete file systems, with action-level and resource-level permissions.



4.4.5 Availability and consistency

- Every file-system object (directory, file and link) is **stored redundantly across multiple AZs** in a Region, for high availability and durability.
- EC2 instances can connect from **any AZ** in the Region.
- All reads and writes are **fully consistent across AZs**: a read in one AZ is guaranteed to see the latest data, even if it is being written in another AZ.

4.5. Content Delivery Networks (CDN)

Short description: A CDN is a network of servers placed at the edge of the Internet that keeps copies of content close to end users, so pages and media load faster and the origin server carries less load.

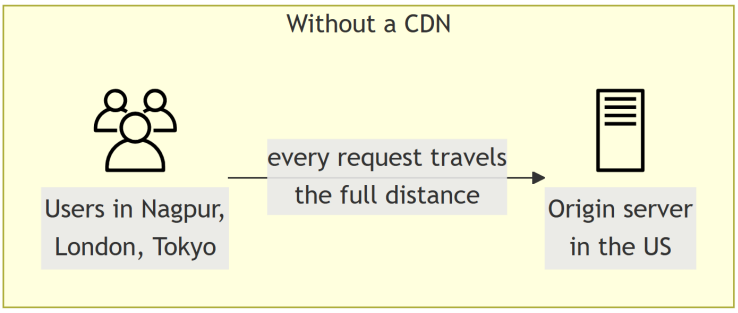
4.5.1 Why a CDN is needed

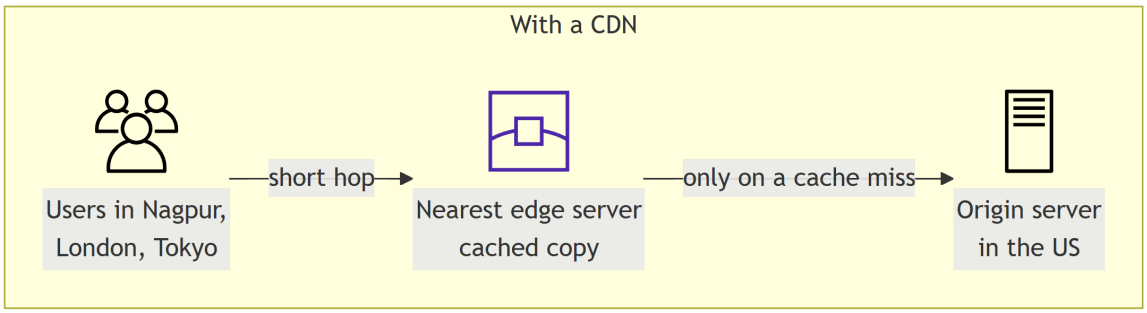
Simple on the outside: From outside, the web looks simple: content providers on one side, end users on the other, with "the Internet" between them.

Problematic on the inside: In reality a request crosses many network providers (UUNet, Qwest, AOL and others) joined at peering points (NAPs). Each hop adds delay and risk.

Issue	What the user sees
Latency	The browser takes a long time to load pages.
Packet loss	The browser hangs; the user needs to hit refresh.
Jitter	Streams are jerky.
Server load	The browser connects but does not fully load the page.
Bandwidth aggregation	Many users share congested links.
Broken or missing content	Parts of the page fail to appear.

The solution: Place servers at the network edge, close to end users, and serve content from them.





4.5.2 Passive and active CDNs

Passive	Active
Works through browser caches and proxies.	Replicates content on a distribution network.
Little investment needed if content is static.	Uses HTTP redirects or DNS-based redirection.
A significant fraction of HTTP objects is not cacheable.	The provider has control and customisation.
The provider lacks control and customisation.	Low investment from the content provider.

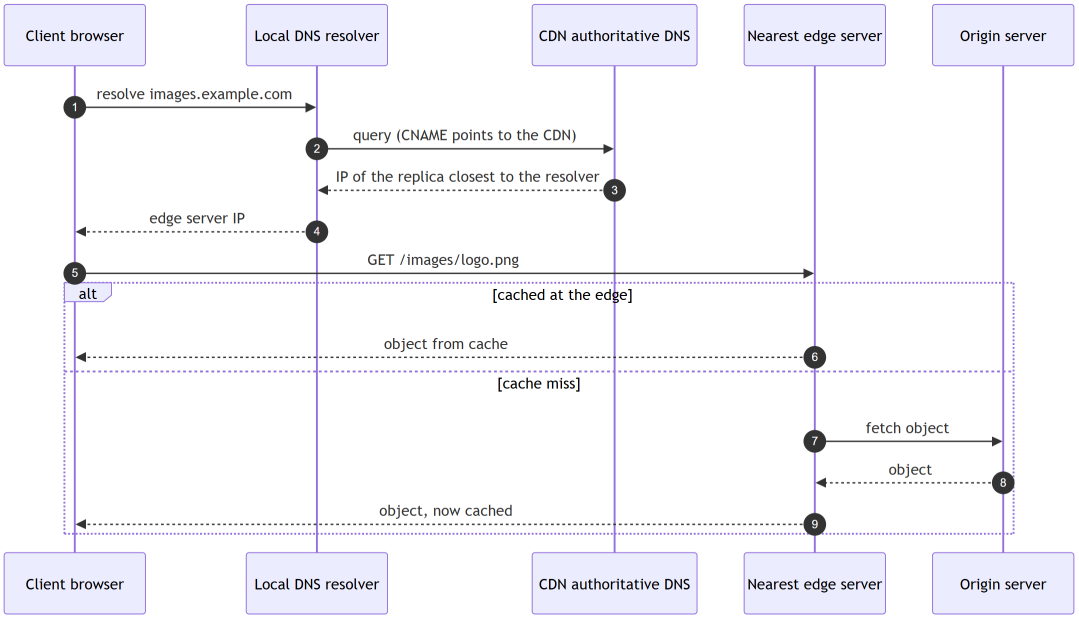
4.5.3 CDN challenges

- How to replicate content?
- Where to replicate content?
- How to find replicated content?
- How to choose among known replicas?
- How to direct clients towards a replica?

4.5.4 Directing clients to a replica

HTTP redirects (application based): The origin server rewrites its pages so that embedded content (images, scripts) is fetched from CDN URLs instead of its own.

DNS based (naming based): The content's hostname points to the CDN, whose DNS answers with the address of a nearby replica.



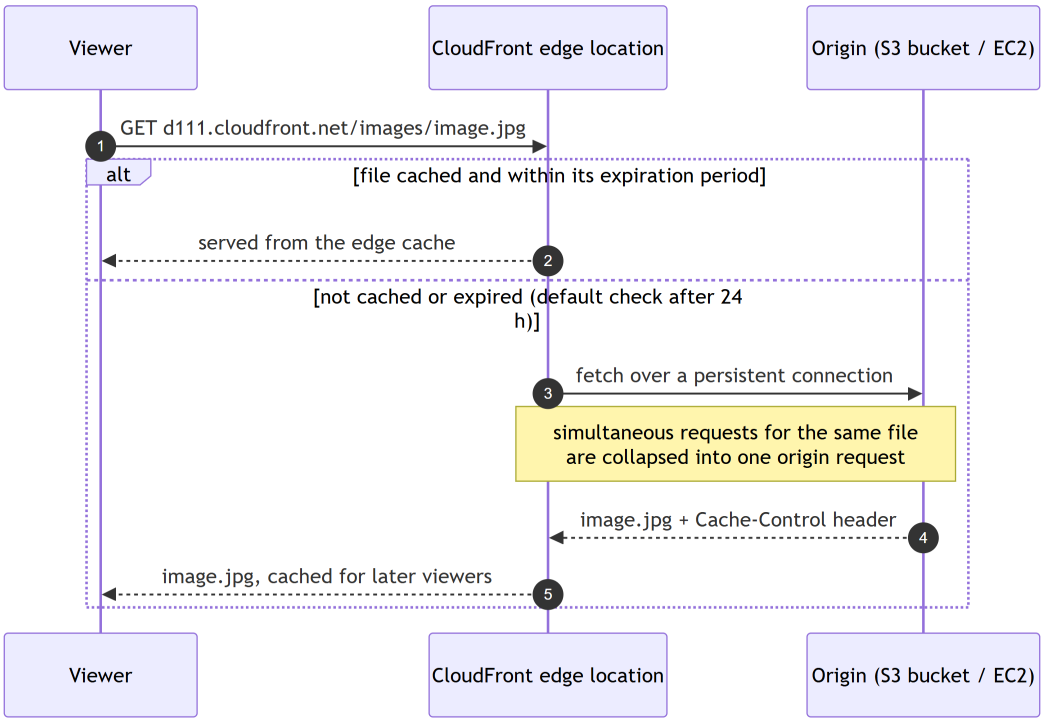
Option	Pros	Cons
HTTP redirects	Application-level, fine-grained control	Additional load and high RTTs; hard to cache
DNS selection	Well suited to caching; reduces RTTs	The request comes from the resolver, not the client; it is for a domain, not a URL; the resolver's user population hides the real load

4.6. Amazon CloudFront

Short description: Amazon CloudFront is AWS's web service for content delivery. It integrates with other AWS services to give developers and businesses an easy way to distribute content to end users with low latency, high data-transfer speeds and no commitments.

4.6.1 How CloudFront gives higher performance

- A network of **edge locations** caches copies of popular files close to viewers.
- Each request is served by the **closest edge location**, so it travels a shorter distance.
- For files not cached at the edge, CloudFront keeps **persistent connections** with the origin, so they are fetched as quickly as possible.
- Further optimisations, such as a wider TCP initial congestion window.
- **Request collapsing** – simultaneous viewer requests for the same file at an edge are merged into one origin request, reducing origin load and cost.



4.6.2 Pricing

Charges are based on actual usage in three areas:

Charge	Basis
Data transfer	Volume of data transferred out of the edge locations, measured in GB.
HTTP/HTTPS requests	Number of requests made to CloudFront for your content.
Invalidation requests	Up to 1,000 files per month free; beyond that, a per-file charge for each file listed in invalidation requests.

4.6.3 Edge locations

The original edge locations (the number in brackets is the count at that city):

United States	Europe	Asia	South America
Ashburn, VA (2); Dallas/Fort Worth, TX (2); Jacksonville, FL; Los Angeles, CA (2); Miami, FL; New York, NY (2); Newark, NJ; Palo Alto, CA; San Jose, CA; Seattle, WA; South Bend, IN; St. Louis, MO	Amsterdam; Dublin; Frankfurt (2); London (2); Milan; Paris (2); Stockholm	Hong Kong; Osaka; Singapore (2); Sydney; Tokyo	São Paulo

4.6.4 Speeding up an entire website

- CloudFront uses the standard **cache-control headers** you set on your files to tell static content from dynamic content.
- With AWS origins you also gain route tracking, health monitoring and fast response to issues.
- Different origins can serve different content on one site – S3 for static objects, EC2 for dynamic content, custom origins for third-party content – paying only for what you use.

4.6.5 How long files stay at the edge

- With no cache-control header, an edge location re-checks the origin for a newer version when it gets a request more than **24 hours** after its last check. This is the **expiration period**.
- The expiration period can be set anywhere from 0 seconds upward with cache-control headers on the origin's files.
- At 0 seconds, CloudFront revalidates every request with the origin.
- For files that rarely change, set a long expiration period and use a **versioning** scheme in file names to roll out updates.

4.6.6 Distributions

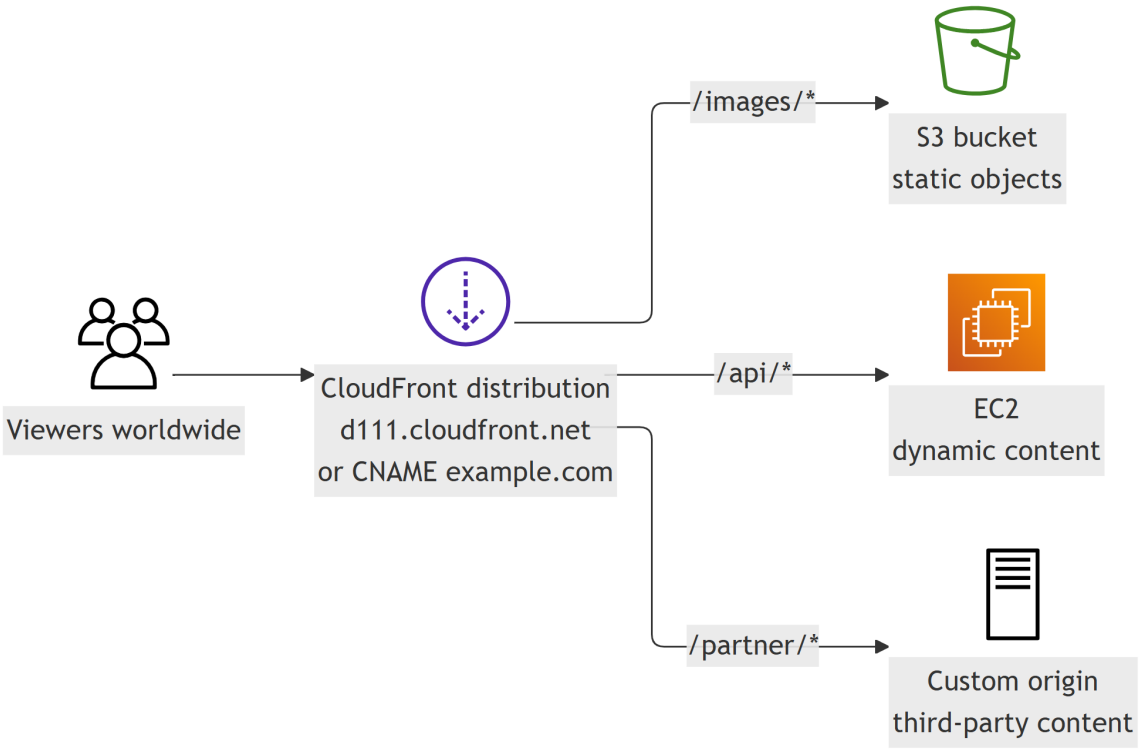
Distribution: After storing objects on an origin server, you create a **distribution**, which tells CloudFront where the objects are.

Example: For an S3 bucket `myawsbucket` holding `images/image.jpg`:

Directly from S3	`http://myawsbucket.s3.amazonaws.com/images/image.jpg`
Through CloudFront	`http://myawsbucket.cloudfront.net/images/image.jpg`
With a CNAME	`http://example.com/images/image.jpg`

Alternate domain names: Up to 10 alternate domain names (CNAMEs), such as `example.com`, can be attached to a distribution.

Types: A **download distribution** delivers files over HTTP/HTTPS. A **streaming distribution** delivered media over RTMP (Adobe Flash Media Server); AWS has since retired RTMP distributions, so streaming now uses ordinary web distributions.



4.7. AWS Snowball and Snowmobile

4.7.1 The problem: moving large data sets

Cloud computing lets organisations store and process data in S3, EC2 and analytics services, but moving a large amount of existing data into the cloud can be difficult. Consider a university with **500 TB** of research data – medical images, videos, student project datasets and simulation results. Uploading it over a normal Internet connection may take weeks or months. The main challenges:

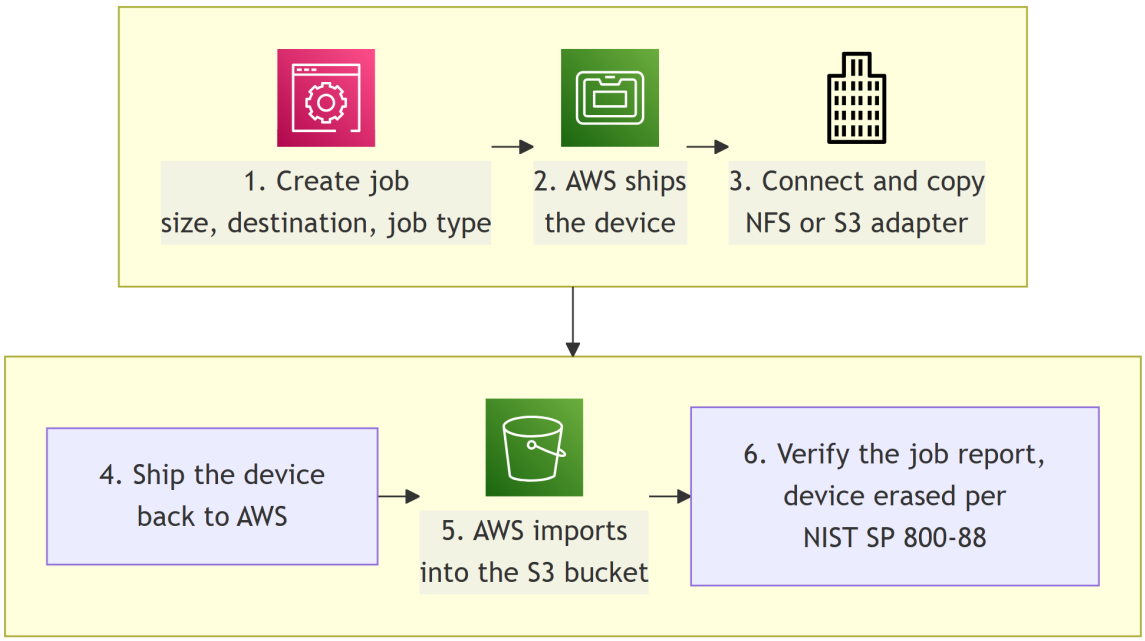
- **Limited bandwidth** – the connection may not support high-speed uploads.
- **Long transfer time** – large datasets take substantial time to upload.
- **Network costs** – data transfer and dedicated connectivity can be expensive.
- **Reliability** – network interruptions can delay the migration.
- **Security** – sensitive data must be protected during transfer.

4.7.2 AWS Snowball and Snowball Edge

Snowball: A secure, ruggedised physical device used to transfer large amounts of data between an organisation's site and AWS, and for supported edge-computing workloads. Suitable for terabytes to hundreds of terabytes, depending on device capacity and configuration.

Snowball Edge: A physical device with both **storage and computing** capabilities. It moves data to or from AWS without depending entirely on an Internet connection. The device is connected to the local network, data is copied onto it, and it is shipped to AWS, which imports the data into the specified S3 destination.

4.7.3 How Snowball works



1. **Create a migration job** – define data volume, destination, device configuration and job type in the AWS Snow Family management interface.
2. **Receive the device** – AWS prepares and ships the configured device.
3. **Connect the device** – the IT team connects it to the local network and authenticates with the job's security credentials.
4. **Copy the data** – through a supported interface such as NFS or the S3 adapter.
5. **Return the device** – ship it back to AWS.
6. **Import into AWS** – AWS imports the data into the specified S3 bucket.
7. **Verify the migration** – check the job completion report and success/failure logs before removing the original copies.

4.7.4 Types of Snowball jobs

Job type	Purpose	Example
Import job	Transfer local data into AWS	Upload university research files to S3
Export job	Transfer data from AWS to a local site	Bring archived datasets from S3 to a data centre
Local compute and storage	Process or store data locally on supported devices	Process sensor data at a remote site

4.7.5 Features, advantages and limitations

Feature	Description
Offline data transfer	Reduces dependence on Internet upload speed.
Large storage capacity	Configurations with about 39.5 TB or 210 TB of usable storage, depending on device type.
Security	Data is encrypted; completed jobs include device erasure following NIST SP 800-88.
Edge computing	Supported configurations run workloads locally.
Scalability	Multiple devices and jobs can be coordinated for larger migrations.
S3 integration	Files and datasets migrate into cloud object storage.

Advantages	Limitations
Suits large datasets and slow or unreliable Internet connections.	Shipping and physical handling add time.
Secure physical transport.	Device capacity and number of jobs must be planned.
Less time spent uploading over a constrained network.	Data must be copied, tracked and verified carefully.
Local processing in suitable configurations.	Not meant for continuously syncing small, frequently changing datasets.

Problem: upload time for 400 TB

A university needs to migrate 400 TB of research data with an upload speed of 100 Mbps. Estimate the minimum upload time over the Internet (decimal units, constant speed, no network overhead).

Solution

Step 1: data in bits $400 \text{ TB} \times 8 = 3{,}200 \text{ terabits}$

Step 2: divide by speed $\dfrac{3{,}200 \text{ Tb}}{0.1 \text{ Gbps}} = \dfrac{3{,}200 \times 10^{12}}{10^{8}} = 32{,}000{,}000 \text{ s}$

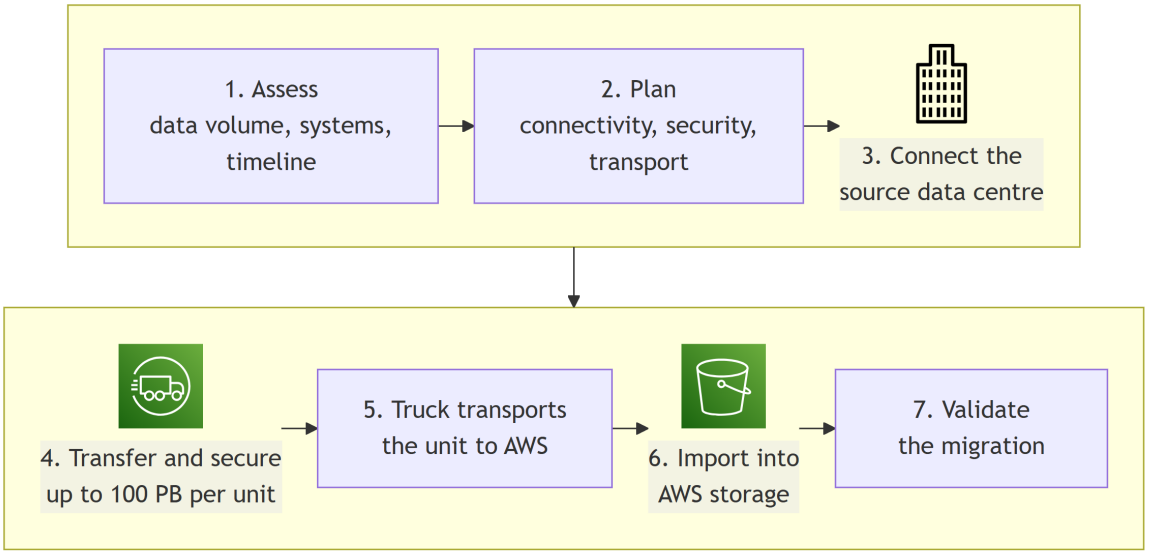
Step 3: seconds to days $\dfrac{32{,}000{,}000}{86{,}400} \approx \mathbf{370.4 \text{ days}}$

Conclusion: More than a year of continuous uploading – a strong case for Snowball.

4.7.6 AWS Snowmobile

Short description: Snowmobile was a large-scale data-migration solution: a secure, shipping-container-sized unit transported by truck, designed for petabyte-scale migrations where Internet transfer or many smaller devices would be impractical. AWS historically described it as carrying

up to **100 PB per unit**. This is a historical specification, not a guarantee of current availability.



1. **Assess the data volume** – total data, storage systems, network requirements and timeline.
2. **Plan the migration infrastructure** – connection between the data centre and the unit, security and transport.
3. **Connect the source systems** – high-speed networking from the customer's storage into the unit.
4. **Transfer and secure the data** – copy using the agreed technical and security procedures.
5. **Transport the unit** – to the designated AWS facility.
6. **Import into AWS** – ingest into the target storage, then validate the migration.

Important characteristics: Extremely high capacity (up to 100 PB per unit); physical bulk transfer that avoids the public Internet; enterprise-scale data-centre migration; specialised security and logistics; useful for moving historical archives and large enterprise datasets.

Advantages	Limitations
Suits exceptionally large data volumes.	Needs specialised transport and planning.
Avoids relying on Internet bandwidth alone.	Transport, setup and ingestion still take time.
Cuts migrations that would otherwise take months or years.	Security, access, logistics and validation are more complex.
Supports major data-centre migrations.	Not for small datasets or continuous sync; its historical availability must not be confused with current offerings.

Benefits of the Snow family: High speed, extremely scalable, tamper resistant and secure, simple and compatible, low cost, easy data retrieval.

Problem: transfer time for 50 PB

An organisation has 50 PB of data and 10 Gbps of upload bandwidth. Estimate the transfer time under ideal conditions (decimal units, always-on connection, no protocol overhead).

Solution

Step 1: data in bits $50 \text{ PB} \times 8 = 400 \text{ petabits} = 4 \times 10^{17} \text{ bits}$

Step 2: divide by speed $\dfrac{4 \times 10^{17}}{10 \times 10^{9}} = 4 \times 10^{7} \text{ s}$

Step 3: seconds to days $\dfrac{40{,}000{,}000}{86{,}400} \approx 463 \text{ days} \approx \mathbf{1.27 \text{ years}}$

4.8. Amazon Route 53: DNS Service

Short description: Route 53 is AWS's Domain Name System (DNS) service. It translates domain names into IP addresses and decides, through routing policies, which resource answers each request.

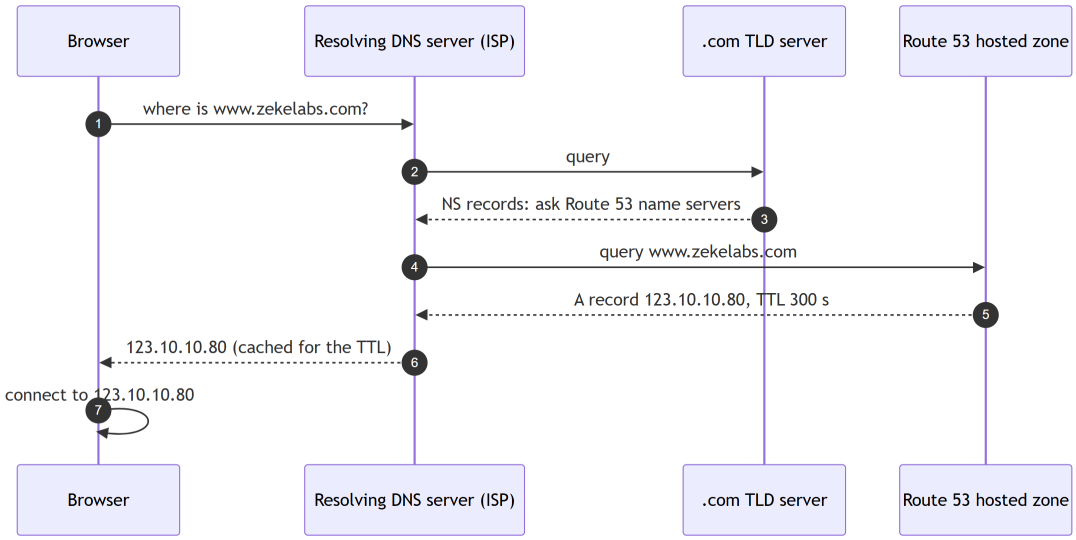
4.8.1 DNS background

IPv4 versus IPv6: IPv4 is a 32-bit field with over 4 billion addresses ($256^4 = 4{,}294{,}967{,}296$, from `0.0.0.0` to `255.255.255.255`, divided into classes A, B and C). IPv6 was created to solve the depletion of this space and uses 128-bit addresses.

Record	Meaning
NS record	**Name Server** record. Used by top-level-domain servers to direct traffic to the content DNS server holding the authoritative records.
A record	**Address** record, the fundamental type. Translates a domain name to an IP address, e.g. `www.zekelabs.com` → `123.10.10.80`.
TTL	**Time To Live**, in seconds: how long a record is cached on the resolving server or the user's PC. A lower TTL makes record changes propagate faster.
CNAME record	**Canonical Name**: resolves one domain name to another, e.g. `mobile.google.com` → `m.google.com`.
Alias record	Maps a record set to an ELB, a CloudFront distribution or an S3 website bucket. Works like a CNAME, but can be used at the **zone apex** (naked domain such as `example.com`), which a CNAME cannot.

Why alias records save time: Route 53 automatically tracks changes in the resource an alias points to. If an alias for `zekelabs.com` points to the load balancer `elb1-1234.us-east.elb.amazonaws.com` and the balancer's IP address changes, Route 53 reflects the change in its DNS answers without any edit to the hosted zone.

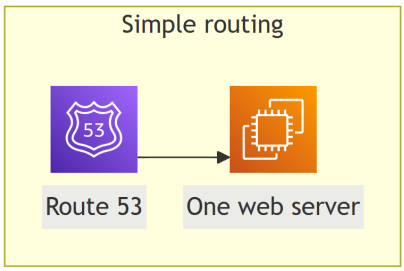
4.8.2 How a name is resolved

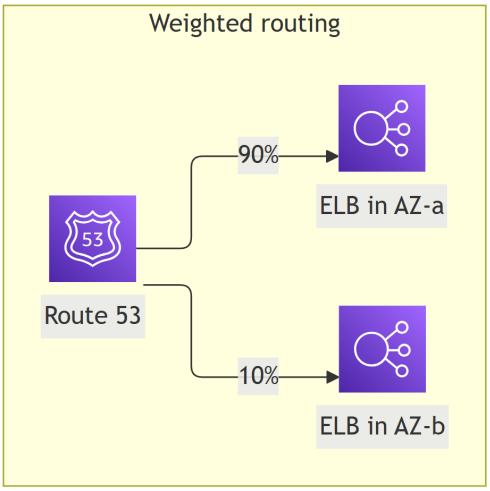


4.8.3 Routing policies

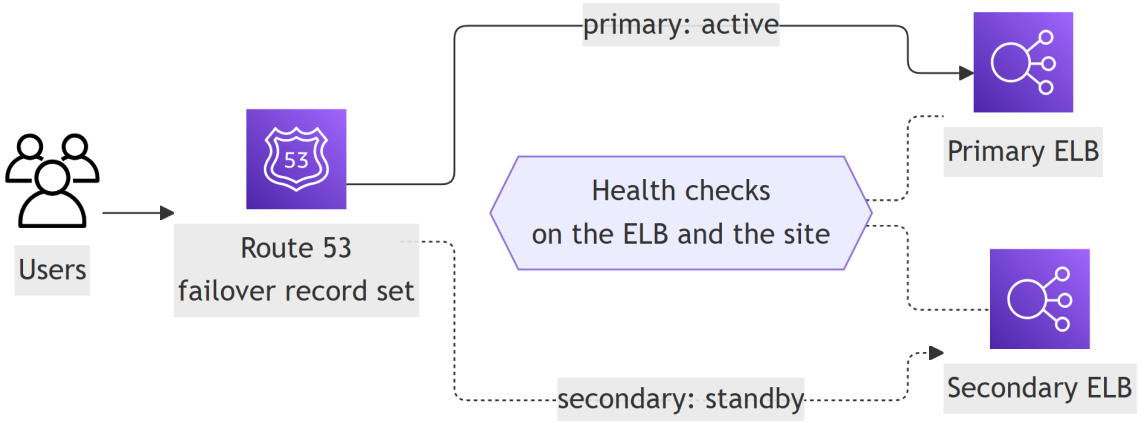
Policy	How it routes
Simple	The default. One resource performs the function for the domain, e.g. one web server serving `zekelabs.com`.
Weighted	Splits traffic by assigned weights, e.g. 10% to one AZ and 90% to another, over the course of the day.
Failover	Active–passive high availability: one ELB is primary, the other secondary. Health checks (one on the ELB, one on the whole site) trigger an automatic switch.
Latency	Sends each user to the Region with the lowest network latency. You create a latency record set for the EC2/ELB resource in each Region; Route 53 answers with the lowest-latency one.
Geolocation	Routes by the location the DNS request comes from, configured with record sets.

Simple and weighted

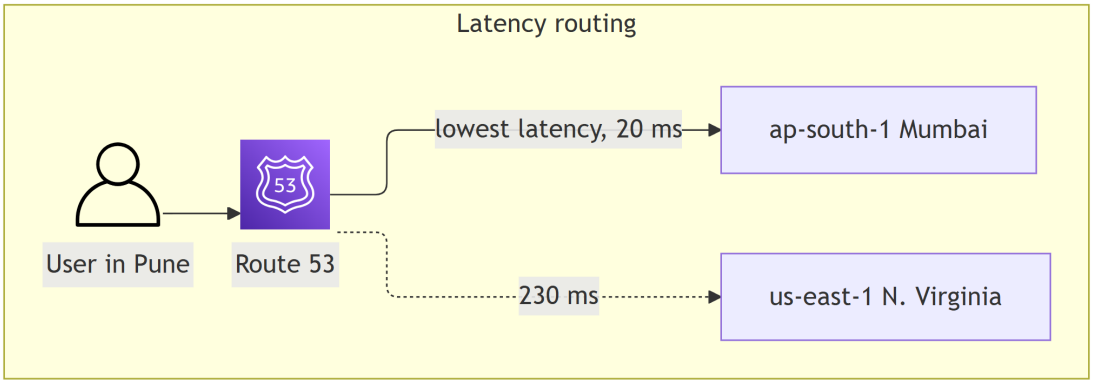


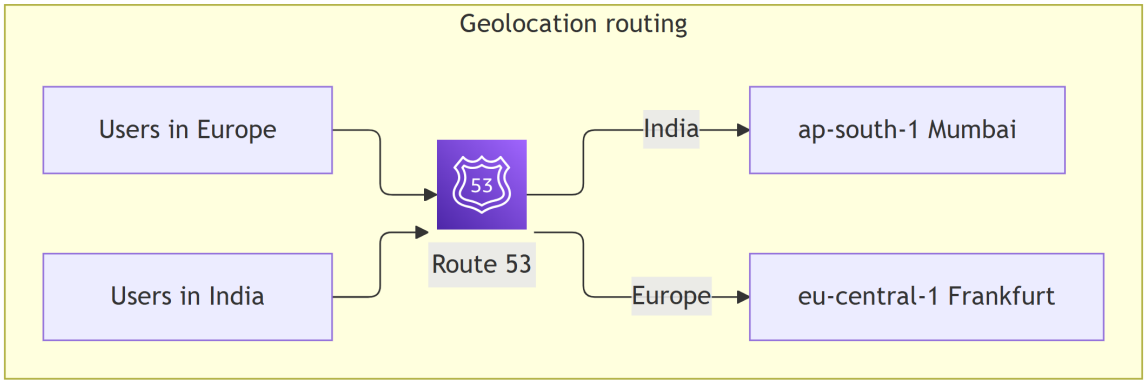


Failover



Latency and geolocation





Unit 5: Introduction to Microsoft Azure

5.1. Introduction to Microsoft Azure

Short description: Microsoft Azure is Microsoft's cloud computing platform. It provides computing power, storage, networking, databases, analytics, artificial intelligence and other services over the Internet.

The idea: Instead of purchasing and maintaining physical servers, organisations rent computing resources from Azure and pay according to their usage and pricing arrangement.

5.1.1 Why we need Azure

Need	What Azure provides
Scalability	Increase or decrease resources according to demand.
Cost efficiency	Avoid large upfront investment in physical infrastructure.
Availability	Keep applications reachable with suitable redundancy and recovery configurations.
Security	Identity management, access controls, encryption and security monitoring.
Global reach	Deploy applications in Azure regions around the world.
Flexibility	Choose services to fit the application's requirements.

5.2. Cloud Service Models in Azure

Model	What it provides	Example	Use case
IaaS	Virtual machines, networking and storage; the customer manages the OS and applications.	Azure Virtual Machines	Hosting a custom web server
PaaS	A managed platform to build and deploy applications without managing servers.	Azure App Service	Deploying a student attendance web app
SaaS	Ready-to-use software over the Internet; users do not manage the infrastructure.	Microsoft 365	Online email, documents and collaboration

5.3. Major Azure Services

Category	Purpose	Examples
Compute	Run workloads on VMs, containers or serverless platforms	Azure Virtual Machines, Azure Functions, Azure Kubernetes Service (AKS)
Storage	Store images, videos, documents, backups and application data	Azure Blob Storage
Database	Store and retrieve structured and other application data	Azure SQL Database, Azure Cosmos DB
Networking	Connect cloud resources and control traffic	Azure Virtual Network, Azure Load Balancer, Azure VPN Gateway
AI and analytics	Build intelligent applications and analyse data	Azure Machine Learning, Microsoft Foundry, Azure Databricks

5.4. Azure Subscription

Short description: An Azure subscription is a logical boundary used to organise Azure resources, manage billing and control access to cloud services. Resources such as virtual machines, databases, storage accounts and networks are created inside a subscription.

5.4.1 Key functions

- **Billing** – tracks the costs of resources in the subscription.
- **Resource management** – provides a scope for managing resources.
- **Access control** – supports role-based access control (RBAC).
- **Governance** – policies, quotas and management rules can be applied.
- **Organisation** – separates resources for departments, projects or environments.

Example: A college uses Azure for a Student Management System and for an Artificial Intelligence Research Project. It may create a separate subscription for each, to manage their budgets, access permissions and resources independently.

5.5. Azure Resource Group

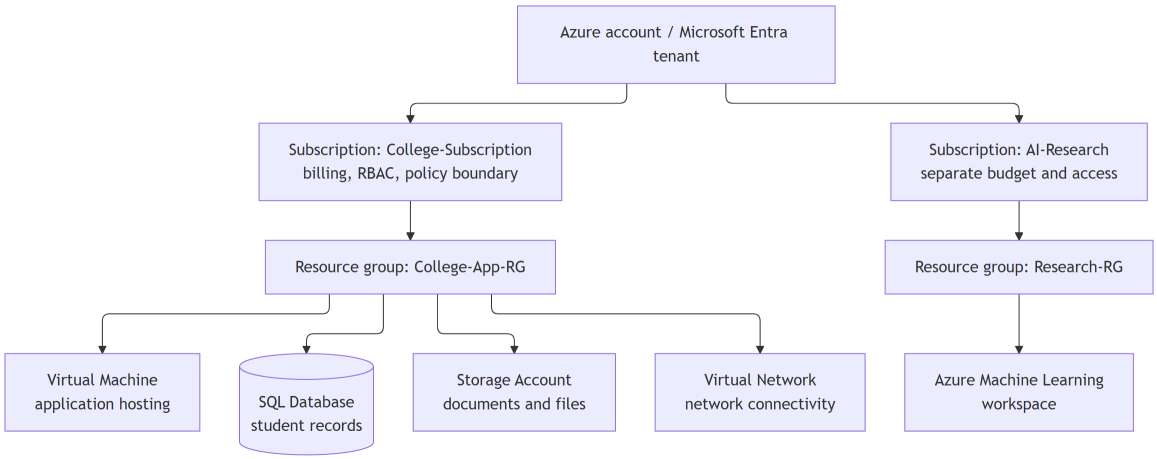
Short description: A resource group is a logical container that holds related Azure resources so that they can be managed together. An online college application might need a VM or App Service, a database, a storage account and networking – all placed in one resource group.

5.5.1 Key functions

- Groups related resources for a project or application.
- Helps organise and manage resources.
- Supports access control and Azure Policy at the resource-group scope.

- Lets resources be deployed and managed together.
- Simplifies clean-up when a project ends – but deleting a resource group also deletes its resources.

5.5.2 Example: college management system



5.5.3 Important points

- Every Azure resource that supports resource groups belongs to exactly one resource group.
- A resource group belongs to one subscription.
- Resources in a resource group can be located in different Azure regions.
- Deleting a resource group normally deletes everything in it, so do it carefully.
- Resources can depend on other resources; check dependencies before moving or deleting one.

5.6. Azure Resource Deployment Models

Two meanings: In Azure, "deployment model" can mean the **resource management model** (Azure Resource Manager versus Classic) or the general **cloud deployment model** (public, private, hybrid). Both are covered here.

5.6.1 Azure Resource Manager (ARM)

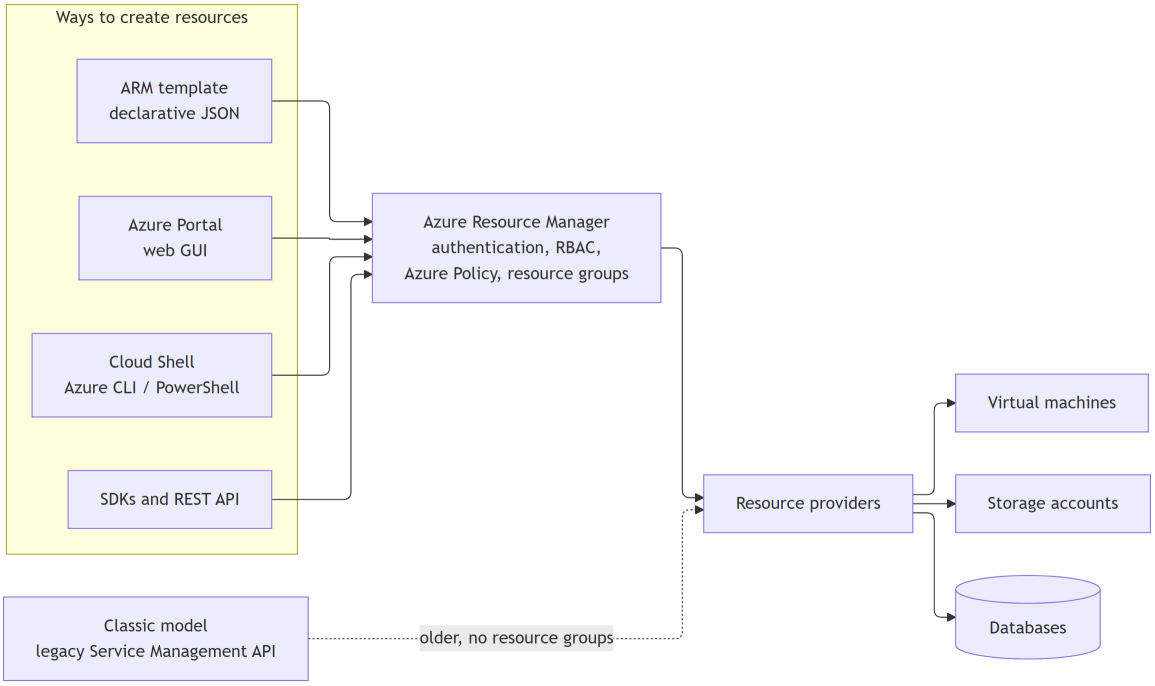
Short description: ARM is Azure's modern management and deployment model. Resources are deployed, organised into resource groups and managed through one consistent control plane.

- Supports resource groups.
- Supports declarative deployments with ARM templates and other infrastructure-as-code tools.
- Provides role-based access control and policy integration.
- Supports coordinated deployment of related resources.

5.6.2 Classic deployment model

Short description: The older Azure management model. It used the older Service Management API and did not use the ARM resource-group structure.

- Uses legacy management mechanisms.
- Has fewer integrated management capabilities than ARM.
- Is associated with older Azure resources and deployments.



5.6.3 ARM versus Classic

Aspect	Azure Resource Manager	Classic
Status	Modern, current model	Older, legacy model
API	ARM control plane	Service Management API
Resource groups	Yes	No
Templates / IaC	ARM templates, Bicep	Limited
Access control	RBAC and Azure Policy	Fewer integrated capabilities

5.6.4 Cloud deployment models with Azure examples

Model	Meaning	Example
Public cloud	Infrastructure provided by a cloud provider and accessed over a network	Hosting a college website on Azure
Private cloud	Infrastructure dedicated to a single organisation	A university's dedicated private cloud for sensitive research
Hybrid cloud	On-premises or private infrastructure combined with public cloud services	A college keeps certain records on-premises and hosts its student portal on Azure

5.7. Methods of Creating Resources in Azure

Resource: An individual cloud service you create and manage, such as a virtual machine, storage account, database or virtual network.

Methods covered: The Azure Portal, Azure Cloud Shell and ARM templates. All three go through Azure Resource Manager to deploy and manage resources.

5.7.1 Method 1: Azure Portal

Short description: A web-based graphical user interface for creating, configuring, monitoring and deleting Azure resources.

Steps – creating a storage account:

1. Open the Azure Portal and sign in.
2. Search for **Storage accounts** in the search bar.
3. Click **Create**.
4. Select the subscription, and an existing resource group or create a new one.
5. Enter the storage account name and choose the region and configuration.
6. Review the configuration and click **Create**.
7. When deployment completes, open the resource to explore its settings.

Advantages	Limitations
Easy for beginners; no programming needed.	Repeating a deployment by hand is time-consuming.
Visual configuration options.	Manual configuration can introduce inconsistencies.
Shows deployment status and resource properties.	Less convenient for large-scale automation.
Useful for demonstrations and small deployments.	

5.7.2 Method 2: Azure Cloud Shell

Short description: A browser-based command-line environment for managing Azure with the Azure CLI (Bash) or Azure PowerShell, without installing either tool locally.

Example: The storage account above, created from Cloud Shell with the Azure CLI:

```
az group create --name College-App-RG --location centralindia
az storage account create --name collegestore2026 \
    --resource-group College-App-RG --location centralindia --sku Standard_LRS
```

5.7.3 Method 3: ARM templates

Short description: An ARM template is a JSON file that defines the Azure resources to deploy and their configuration.

Declarative approach: You state *what* resources you want, not every step needed to create them. This makes templates suited to repeatable, automated deployments.

```
{
  "$schema": "https://schema.management.azure.com/schemas/2019-04-01/deploymentTemplate.json#",
  "contentVersion": "1.0.0.0",
  "resources": [
    {
      "type": "Microsoft.Storage/storageAccounts",
      "apiVersion": "2023-01-01",
      "name": "collegestore2026",
      "location": "centralindia",
      "sku": { "name": "Standard_LRS" },
      "kind": "StorageV2"
    }
  ]
}
```

5.8. Virtual Networking in the Cloud

Short description: Virtual networking is creating and managing software-defined networks that connect cloud resources such as VMs, databases and applications. In Azure the main component is the **Azure Virtual Network (VNet)** – an isolated private network for Azure resources, similar to an organisation's own network.

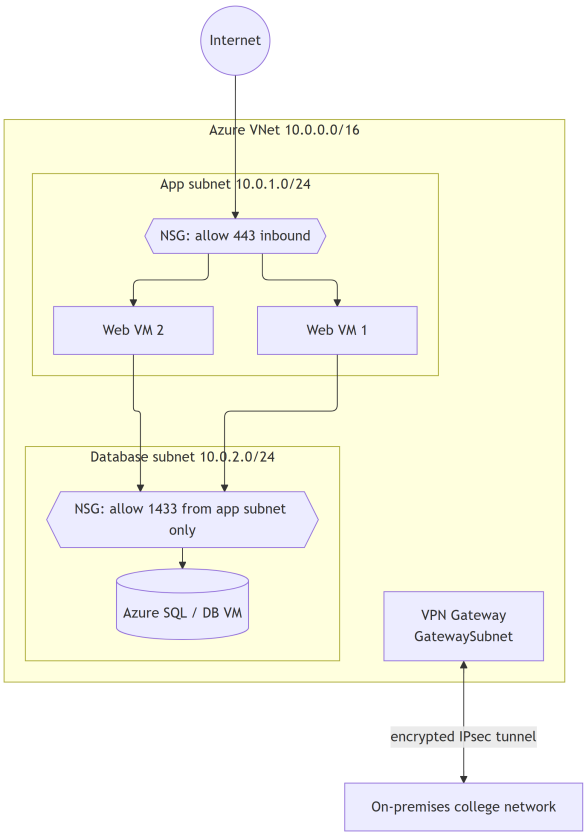
Why it is required:

- To let cloud resources communicate with each other.
- To isolate workloads from other networks.
- To control inbound and outbound traffic.
- To connect cloud resources to on-premises networks.
- To support secure, scalable cloud applications.

5.8.1 Main components

Component	Description
Virtual Network (VNet)	An isolated private network that defines the address space for connected resources, e.g. `10.0.0.0/16`.
Subnet	A subdivision of a VNet that organises resources by function or security boundary, e.g. an application subnet and a database subnet.
Network Security Group (NSG)	Rules that allow or deny inbound and outbound traffic for network interfaces and subnets.
VPN Gateway	Encrypted VPN connections between Azure VNets and remote or on-premises networks.

5.8.2 Architecture diagram

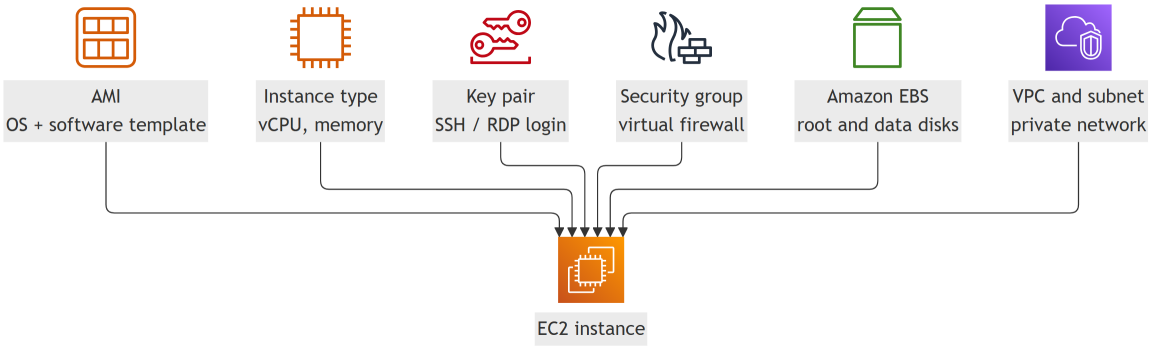


5.9. Compute Services in AWS and Azure

Short description: Compute services provide the processing power to run applications, websites, databases and other software without owning physical servers. AWS and Azure both offer virtual machines: **Amazon EC2** and **Azure Virtual Machines**.

5.9.1 Amazon EC2 (Elastic Compute Cloud)

Short description: An AWS service for launching and managing virtual servers called **instances**. The user chooses the operating system, instance type, storage and networking.



Component	Role
Amazon Machine Image (AMI)	Template holding the OS and pre-installed software used to launch the instance.
Instance type	The hardware profile: number of vCPUs, memory and network performance (e.g. `t3.micro`).
Key pair	Public/private key used to log in securely (SSH for Linux, password decryption for Windows RDP).
Security group	A virtual firewall of allow rules for inbound and outbound traffic.
Amazon EBS	Block-storage volumes used as the instance's root and data disks.
VPC	The isolated virtual network and subnet the instance is launched into.

5.9.2 Creating a Linux VM in Azure

Azure Virtual Machines provide on-demand Windows or Linux computing. Through the Azure Portal:

1. Sign in to the Azure Portal and choose **Virtual machines** → **Create** → **Azure virtual machine**.
2. **Basics:** choose the subscription and resource group, enter the VM name, and pick the region and an image such as Ubuntu Server.
3. Choose a **size** (vCPUs and memory).
4. **Administrator account:** choose SSH public key authentication and a username.
5. **Inbound ports:** allow SSH (22) so you can connect.
6. **Disks** and **Networking:** keep the defaults (managed OS disk, a new VNet, subnet and public IP) or adjust them.
7. **Review + create**, then **Create**; download the private key when prompted.
8. When deployment finishes, copy the public IP and connect with `ssh -i key.pem azureuser@<public-ip>`.

5.9.3 AWS EC2 versus Azure Virtual Machines

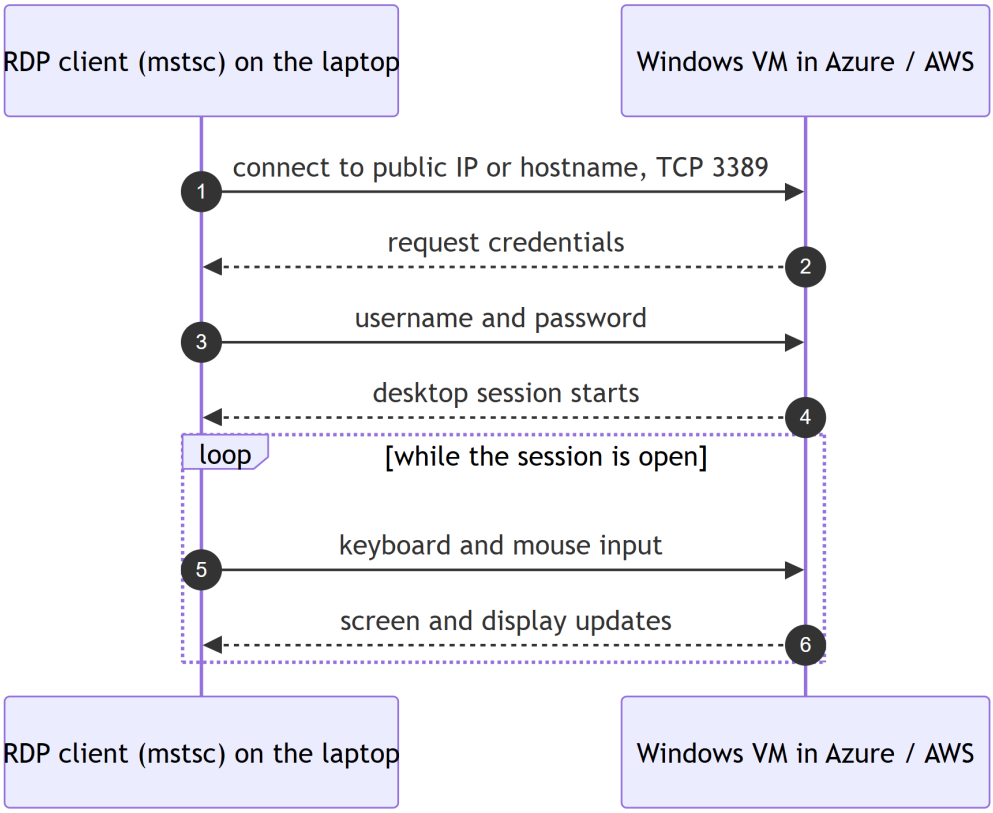
Feature	AWS EC2	Azure Virtual Machines
Cloud provider	Amazon Web Services	Microsoft Azure
Virtual server	EC2 instance	Azure VM
Machine image	Amazon Machine Image (AMI)	Azure VM image
Networking	Amazon VPC and subnets	Azure Virtual Network and subnets
Firewall rules	Security groups	Network Security Groups
Virtual disk	Amazon EBS	Azure Managed Disks
Resource organisation	AWS resources, tags and account structure	Subscription and resource group
Management interface	AWS Management Console	Azure Portal
Automation	AWS CLI, SDKs, CloudFormation	Azure CLI, SDKs, ARM templates and Bicep

5.10. Remote Desktop Protocol (RDP)

Short description: RDP is a Microsoft network protocol that lets a user remotely access and control another computer over a network or the Internet – seeing its desktop, opening applications and managing files as if sitting in front of it. It is commonly used to manage virtual machines on Azure and AWS.

5.10.1 How RDP works

RDP follows a **client–server** architecture.



1. The user opens an RDP client on the local computer.
2. The client connects to the remote computer by IP address or hostname.
3. The user authenticates with valid credentials.
4. The remote computer provides a desktop session.
5. The user sends keyboard and mouse input.
6. The remote computer executes it and sends display updates back.

RDP in cloud computing: Especially useful for Windows VMs: an Azure administrator can create a Windows VM and connect to it remotely over RDP, without physical access to the server.

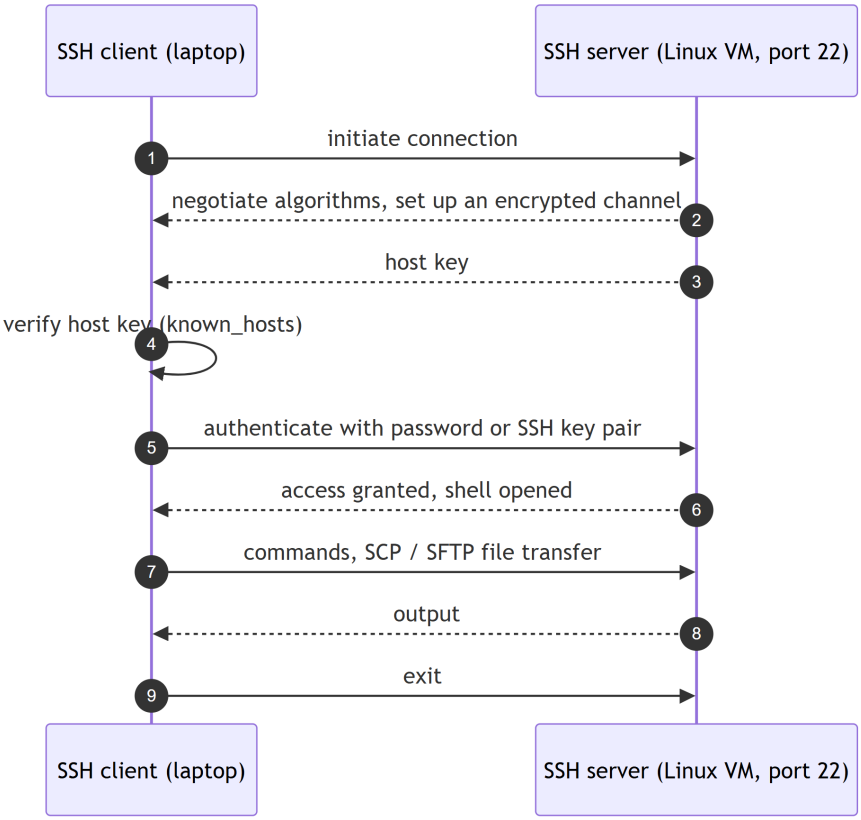
Advantages	Limitations
Remote access from different locations.	Requires network connectivity.
Centralised management of cloud servers.	Performance depends on network speed and latency.
Use applications and resources hosted elsewhere.	Wrong security settings can expose systems to attack.
No physical presence needed.	Mainly for Windows graphical access, though clients exist for other platforms.
Cost-efficient remote administration of cloud VMs.	Some advanced features need extra configuration or licensing.

5.11. Secure Shell (SSH)

Short description: SSH is a cryptographic network protocol for securely accessing and managing remote computers over an unsecured network such as the Internet. It is used to run commands, transfer files, configure servers and manage cloud resources, especially Linux VMs on AWS, Azure and Google Cloud.

5.11.1 How SSH works

The SSH **client** runs on the user's computer; the SSH **server** runs on the remote machine.



1. **Initiate connection** – the user runs an SSH client.
2. **Establish secure communication** – client and server negotiate algorithms and set up an encrypted connection.
3. **Verify server identity** – the client checks the server's host key to prevent impersonation.
4. **Authenticate the user** – with a password or an SSH key pair.
5. **Execute commands** on the remote system.
6. **Terminate** – type `exit` to close the session.

```
ssh -i college-key.pem azureuser@20.193.45.10     # log in with a key pair
scp -i college-key.pem report.pdf azureuser@20.193.45.10:~/   # copy a file
```

5.11.2 Important features

Feature	Description
Encryption	Protects data between client and server.
Authentication	Verifies the server's identity and the user's credentials.
Data integrity	Detects unauthorised modification of transmitted data.
Remote command execution	Manage remote systems from the command line.
Secure file transfer	SCP and SFTP.
Port forwarding	Securely tunnels other network connections.

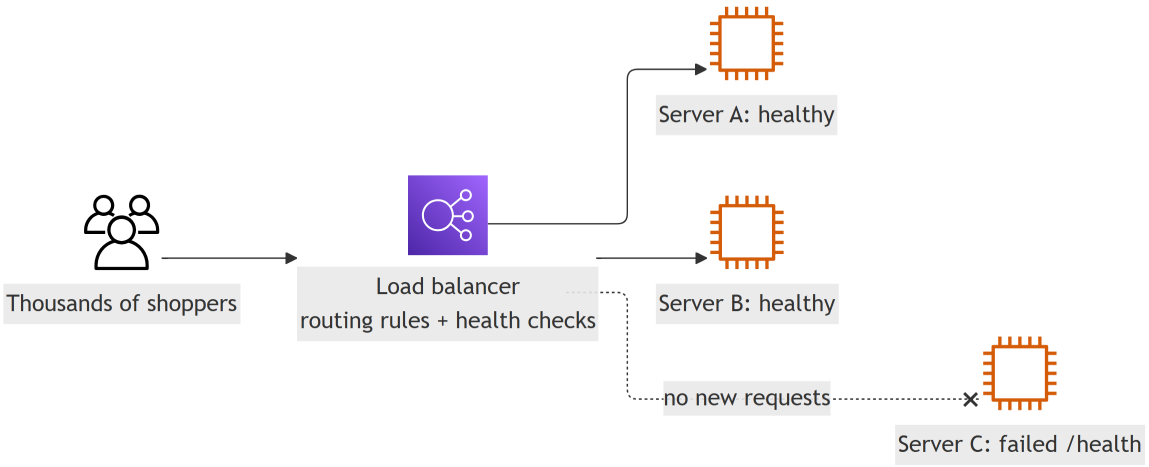
5.11.3 RDP versus SSH

Aspect	RDP	SSH
Interface	Graphical desktop	Command line
Typical OS	Windows VMs	Linux VMs
Default port	3389	22
Authentication	Username and password	Password or key pair
File transfer	Clipboard / drive redirection	SCP, SFTP

5.12. Load Balancer

Short description: A load balancer distributes incoming network traffic across multiple servers or VMs to improve performance, availability, scalability and reliability. If every request went to a single server it would become overloaded; the load balancer spreads the requests.

Example: During a major sale, thousands of customers browse and order at the same time on a shopping website. A load balancer spreads their requests across several application servers so no single server carries all the traffic.



5.12.1 How a request is handled

1. A user sends a request to the website or application.
2. The request reaches the load balancer.
3. The load balancer checks its routing rules and server health.
4. It selects a suitable healthy server.
5. The selected server processes the request.
6. The response is returned to the user.

5.12.2 Objectives

- **Scalability** – distributes traffic as the number of users grows.
- **High availability** – keeps routing to healthy servers when one fails.
- **Performance** – no single server becomes a bottleneck.
- **Fault tolerance** – reduces the impact of individual server failures.
- **Efficient resource utilisation** – spreads work across available servers.
- **Maintenance support** – servers can be taken out of service for maintenance.

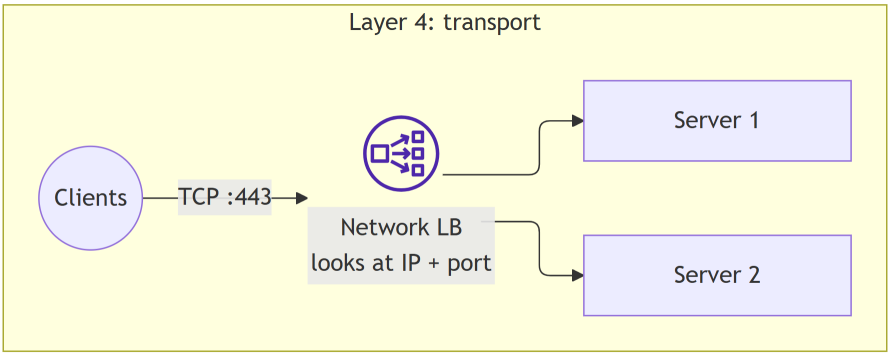
5.12.3 Types of load balancers

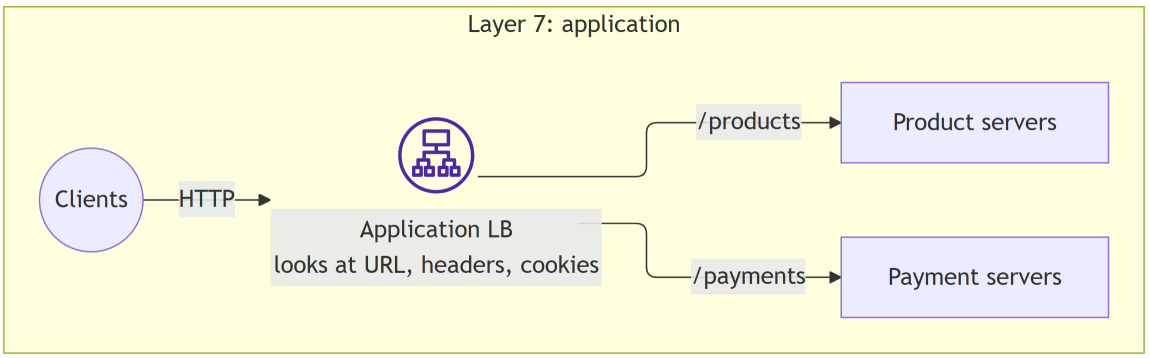
By deployment:

Type	Description
Hardware	A physical appliance in a data centre; common where dedicated network equipment is required.
Software	An application that distributes traffic, running on VMs or containers, e.g. HAProxy and NGINX.
Cloud	A managed cloud service that distributes traffic without the user managing any load-balancing hardware.

By network layer:

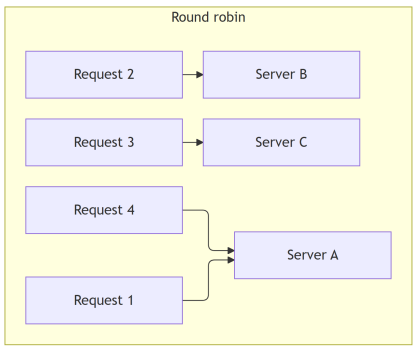
Layer	Decides using	Example and typical use
Layer 4 (transport)	TCP/UDP connection information: IP addresses and port numbers	Send TCP traffic on port 443 to a backend server; high-performance traffic distribution
Layer 7 (application)	HTTP headers, hostnames, URL paths and cookies	Route `/products` to product servers and `/payments` to payment servers; websites, APIs and microservices

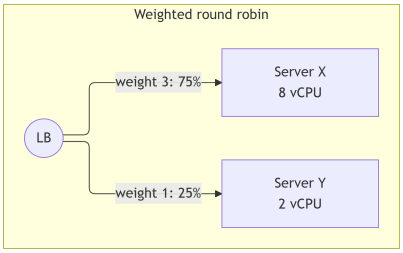




5.12.4 Load-balancing algorithms

Algorithm	How it works	Advantage
Round robin	Requests go to servers in turn: 1 → A, 2 → B, 3 → C, then repeat	Simple and easy to understand
Least connections	A new request goes to the eligible server with the fewest active connections	Good when requests take different times to process
Weighted round robin	Servers get traffic in proportion to assigned weights; bigger servers get more	Handles servers of different capacity
IP hash	A hash of the client IP selects the backend server	Consistent routing for a client, depending on configuration





5.12.5 Health checks

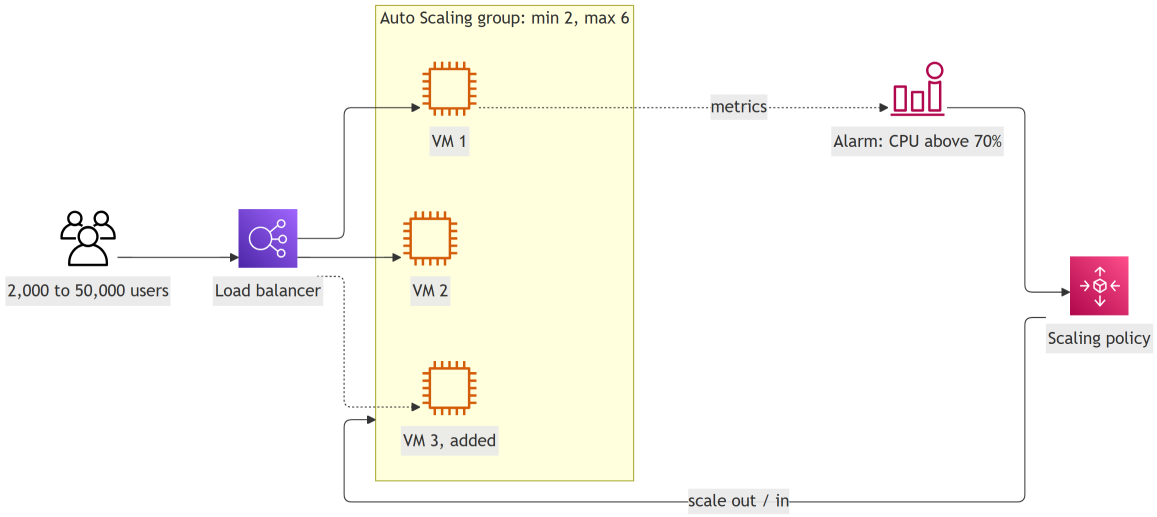
Short description: A health check tests whether a backend server is working correctly.

- The load balancer periodically sends a request, for example HTTP `GET /health`.
- If Server A responds successfully, it stays eligible for traffic.
- If Server B fails the configured check, it receives no new requests.
- When Server B is healthy again, it is returned to service.
- Health checks improve availability but do not prove that every function of the application works – the health endpoint must be designed carefully.

5.13. Auto Scaling

Short description: Auto Scaling automatically increases or decreases the computing resources allocated to an application according to its workload, keeping performance up while controlling cost.

Example: A shopping website has 2,000 users on a normal day and 50,000 during a major sale. Auto Scaling adds VM instances to handle the extra traffic and removes them when demand falls.



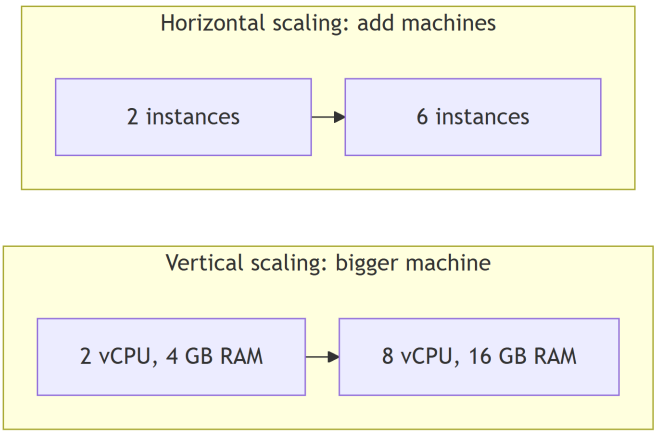
5.13.1 Why we need it

Without Auto Scaling, administrators must add or remove resources by hand, risking poor performance or wasted spending. Its objectives:

- **High availability** – keeps applications running as demand changes.
- **Better performance** – adds resources when the workload rises.
- **Cost optimisation** – removes unneeded resources when demand falls.
- **Automatic resource management** – less manual intervention.
- **Elasticity** – resources adapt to workload changes.
- **Fault tolerance** – can replace unhealthy instances.

5.13.2 Types of scaling

Type	What changes	Example
Horizontal	Adds or removes VMs / instances	A web app grows from 2 to 6 EC2 instances
Vertical	Resources of an existing machine	Upgrade a VM from 2 vCPU / 4 GB to 8 vCPU / 16 GB
Scheduled	Capacity on a predefined schedule	Add capacity before online examination hours
Dynamic	Capacity in response to real-time metrics	Add instances while CPU stays above 70%



5.13.3 Load balancing and Auto Scaling together

Load balancing	**Auto Scaling**
Distributes incoming traffic	Adds or removes computing instances
Selects eligible backend servers	Changes the size of the server fleet
Responds to traffic-distribution needs	Responds to configured scaling policies
Helps avoid overloaded servers	Helps match capacity to demand

5.14. Load Balancing and Auto Scaling Numericals

Problem 1: Can four servers handle the load?

An e-commerce website receives 12,000 requests per minute. A load balancer distributes them equally among 4 web servers; each server can handle 3,500 requests per minute. Find the requests per server, the total capacity, and whether the servers can cope.

Solution

Requests per server	$\dfrac{12{,}000}{4} = 3{,}000$ per minute
Total capacity	$4 \times 3{,}500 = 14{,}000$ per minute
Can they cope?	Yes: capacity $14{,}000 >$ traffic $12{,}000$
Unused capacity	$14{,}000 - 12{,}000 =$ **2,000 requests/minute**

Problem 2: Instances needed at peak

An application runs on 3 VM instances, each handling 2,000 requests per minute. At peak, traffic rises to 15,000 requests per minute and Auto Scaling adds instances as required. Find the initial capacity, the minimum instances for peak traffic, and the instances Auto Scaling must add.

Solution

Initial capacity	$3 \times 2{,}000 = 6{,}000$ per minute
Minimum instances	$\frac{15{,}000}{2{,}000} = 7.5$, rounded **up** to **8** instances (instances are whole numbers)
Additional instances	$8 - 3 =$ **5 instances**

Problem 3: Load balancer plus Auto Scaling

An application receives 24,000 requests per minute. Initially 4 servers are available, each handling 4,000 requests per minute. The load balancer distributes requests equally, and Auto Scaling adds servers of the same capacity when needed. Find the requests per server before scaling, the initial capacity, the minimum total servers, and the additional servers.

Solution

Requests per server before scaling	$\frac{24{,}000}{4} = 6{,}000$ per minute (over the 4,000 limit)
Initial capacity	$4 \times 4{,}000 = 16{,}000$ per minute
Minimum total servers	$\frac{24{,}000}{4{,}000} =$ **6**
Additional servers	$6 - 4 =$ **2 servers**